Chapter 4 **Data Warehousing and Online Analytical Processing 125**

4.1 **Data Warehouse: Basic Concepts 125**

4.1.1 What Is a Data Warehouse? 126

4.1.2 Differences between Operational Database Systems and Data Warehouses 128

4.1.3 But, Why Have a Separate Data Warehouse? 129

4.1.4 Data Warehousing: A Multitiered Architecture 130
4.1.5 Data Warehouse Models: Enterprise Warehouse, Data Mart, and Virtual Warehouse 132
4.1.6 Extraction, Transformation, and Loading 134
4.1.7 Metadata Repository 134

4.2 **Data Warehouse Modeling: Data Cube and OLAP 135**
4.2.1 Data Cube: A Multidimensional Data Model 136
4.2.2 Stars, Snowflakes, and Fact Constellations: Schemas for Multidimensional Data Models 139
4.2.3 Dimensions: The Role of Concept Hierarchies 142
4.2.4 Measures: Their Categorization and Computation 144
4.2.5 Typical OLAP Operations 146
4.2.6 A Starnet Query Model for Querying Multidimensional Databases 149

4.3 **Data Warehouse Design and Usage 150**
4.3.1 A Business Analysis Framework for Data Warehouse Design 150
4.3.2 Data Warehouse Design Process 151
4.3.3 Data Warehouse Usage for Information Processing 153
4.3.4 From Online Analytical Processing to Multidimensional Data Mining 155

4.4 **Data Warehouse Implementation 156**
4.4.1 Efficient Data Cube Computation: An Overview 156
4.4.2 Indexing OLAP Data: Bitmap Index and Join Index 160
4.4.3 Efficient Processing of OLAP Queries 163
4.4.4 OLAP Server Architectures: ROLAP versus MOLAP versus HOLAP 164

4.5 **Data Generalization by Attribute-Oriented Induction 166**
4.5.1 Attribute-Oriented Induction for Data Characterization 167
4.5.2 Efficient Implementation of Attribute-Oriented Induction 172
4.5.3 Attribute-Oriented Induction for Class Comparisons 175

4.6 **Summary 178**

4.7 **Exercises 180**

4.8 **Bibliographic Notes 184**

Chapter 5 **Data Cube Technology 187**

5.1 **Data Cube Computation: Preliminary Concepts 188**
5.1.1 Cube Materialization: Full Cube, Iceberg Cube, Closed Cube, and Cube Shell 188
5.1.2 General Strategies for Data Cube Computation 192

5.2 **Data Cube Computation Methods 194**
5.2.1 Multiway Array Aggregation for Full Cube Computation 195

5.2.2 BUC: Computing Iceberg Cubes from the Apex Cuboid Downward 200
5.2.3 Star-Cubing: Computing Iceberg Cubes Using a Dynamic Star-Tree Structure 204
5.2.4 Precomputing Shell Fragments for Fast High-Dimensional OLAP 210

5.3 **Processing Advanced Kinds of Queries by Exploring Cube Technology 218**
5.3.1 Sampling Cubes: OLAP-Based Mining on Sampling Data 218
5.3.2 Ranking Cubes: Efficient Computation of Top-*k* Queries 225

5.4 **Multidimensional Data Analysis in Cube Space 227**
5.4.1 Prediction Cubes: Prediction Mining in Cube Space 227
5.4.2 Multifeature Cubes: Complex Aggregation at Multiple Granularities 230
5.4.3 Exception-Based, Discovery-Driven Cube Space Exploration 231

5.5 **Summary 234**

5.6 **Exercises 235**

5.7 **Bibliographic Notes 240**

4 Data Warehousing and Online Analytical Processing

Data warehouses generalize and consolidate data in multidimensional space. The construction of data warehouses involves data cleaning, data integration, and data transformation, and can be viewed as an important preprocessing step for data mining. Moreover, data warehouses provide *online analytical processing (OLAP)* tools for the interactive analysis of multidimensional data of varied granularities, which facilitates effective data generalization and data mining. Many other data mining functions, such as association, classification, prediction, and clustering, can be integrated with OLAP operations to enhance interactive mining of knowledge at multiple levels of abstraction. Hence, the data warehouse has become an increasingly important platform for data analysis and OLAP and will provide an effective platform for data mining. Therefore, data warehousing and OLAP form an essential step in the knowledge discovery process. This chapter presents an overview of data warehouse and OLAP technology. This overview is essential for understanding the overall data mining and knowledge discovery process.

In this chapter, we study a well-accepted definition of the data warehouse and see why more and more organizations are building data warehouses for the analysis of their data (Section 4.1). In particular, we study the *data cube*, a multidimensional data model for data warehouses and OLAP, as well as OLAP operations such as roll-up, drill-down, slicing, and dicing (Section 4.2). We also look at data warehouse design and usage (Section 4.3). In addition, we discuss *multidimensional data mining*, a powerful paradigm that integrates data warehouse and OLAP technology with that of data mining. An overview of data warehouse implementation examines general strategies for efficient data cube computation, OLAP data indexing, and OLAP query processing (Section 4.4). Finally, we study data generalization by attribute-oriented induction (Section 4.5). This method uses concept hierarchies to generalize data to multiple levels of abstraction.

4.1 Data Warehouse: Basic Concepts

This section gives an introduction to data warehouses. We begin with a definition of the data warehouse (Section 4.1.1). We outline the differences between operational database

© 2012 Elsevier Inc. All rights reserved.

systems and data warehouses (Section 4.1.2), then explain the need for using data warehouses for data analysis, rather than performing the analysis directly on traditional databases (Section 4.1.3). This is followed by a presentation of data warehouse architecture (Section 4.1.4). Next, we study three data warehouse models—an enterprise model, a data mart, and a virtual warehouse (Section 4.1.5). Section 4.1.6 describes back-end utilities for data warehousing, such as extraction, transformation, and loading. Finally, Section 4.1.7 presents the metadata repository, which stores data about data.

1. What Is a Data Warehouse(DW)? Discuss key features of DW.

4.1.1 What Is a Data Warehouse?

Data warehousing provides architectures and tools for business executives to systematically organize, understand, and use their data to make strategic decisions. Data warehouse systems are valuable tools in today's competitive, fast-evolving world. In the last several years, many firms have spent millions of dollars in building enterprise-wide data warehouses. Many people feel that with competition mounting in every industry, data warehousing is the latest must-have marketing weapon—a way to retain customers by learning more about their needs.

"*Then, what exactly is a data warehouse?*" Data warehouses have been defined in many ways, making it difficult to formulate a rigorous definition. Loosely speaking, a data warehouse refers to a data repository that is maintained separately from an organization's operational databases. Data warehouse systems allow for integration of a variety of application systems. They support information processing by providing a solid platform of consolidated historic data for analysis.

According to William H. Inmon, a leading architect in the construction of data warehouse systems, "A data warehouse is a subject-oriented, integrated, time-variant, and nonvolatile collection of data in support of management's decision making process" [Inm96]. This short but comprehensive definition presents the major features of a data warehouse. The four keywords—*subject-oriented, integrated, time-variant*, and *nonvolatile*—distinguish data warehouses from other data repository systems, such as relational database systems, transaction processing systems, and file systems.

Let's take a closer look at each of these key features.

- **Subject-oriented:** A data warehouse is organized around major subjects such as customer, supplier, product, and sales. Rather than concentrating on the day-to-day operations and transaction processing of an organization, a data warehouse focuses on the modeling and analysis of data for decision makers. Hence, data warehouses typically provide a simple and concise view of particular subject issues by excluding data that are not useful in the decision support process.
- **Integrated:** A data warehouse is usually constructed by integrating multiple heterogeneous sources, such as relational databases, flat files, and online transaction records. Data cleaning and data integration techniques are applied to ensure consistency in naming conventions, encoding structures, attribute measures, and so on.

- **Time-variant:** Data are stored to provide information from an historic perspective (e.g., the past 5–10 years). Every key structure in the data warehouse contains, either implicitly or explicitly, a time element.
- **Nonvolatile:** A data warehouse is always a physically separate store of data transformed from the application data found in the operational environment. Due to this separation, a data warehouse does not require transaction processing, recovery, and concurrency control mechanisms. It usually requires only two operations in data accessing: *initial loading of data* and *access of data*.

In sum, a data warehouse is a semantically consistent data store that serves as a physical implementation of a decision support data model. It stores the information an enterprise needs to make strategic decisions. A data warehouse is also often viewed as an architecture, constructed by integrating data from multiple heterogeneous sources to support structured and/or ad hoc queries, analytical reporting, and decision making.

Based on this information, we view **data warehousing** as the process of constructing and using data warehouses. The construction of a data warehouse requires data cleaning, data integration, and data consolidation. The utilization of a data warehouse often necessitates a collection of *decision support* technologies. This allows "knowledge workers" (e.g., managers, analysts, and executives) to use the warehouse to quickly and conveniently obtain an overview of the data, and to make sound decisions based on information in the warehouse. Some authors use the term *data warehousing* to refer only to the process of data warehouse *construction*, while the term *warehouse DBMS* is used to refer to the *management and utilization* of data warehouses. We will not make this distinction here.

"*How are organizations using the information from data warehouses?*" Many organizations use this information to support business decision-making activities, including (1) increasing customer focus, which includes the analysis of customer buying patterns (such as buying preference, buying time, budget cycles, and appetites for spending); (2) repositioning products and managing product portfolios by comparing the performance of sales by quarter, by year, and by geographic regions in order to fine-tune production strategies; (3) analyzing operations and looking for sources of profit; and (4) managing customer relationships, making environmental corrections, and managing the cost of corporate assets.

Data warehousing is also very useful from the point of view of *heterogeneous database integration*. Organizations typically collect diverse kinds of data and maintain large databases from multiple, heterogeneous, autonomous, and distributed information sources. It is highly desirable, yet challenging, to integrate such data and provide easy and efficient access to it. Much effort has been spent in the database industry and research community toward achieving this goal.

The traditional database approach to heterogeneous database integration is to build **wrappers** and **integrators** (or **mediators**) on top of multiple, heterogeneous databases. When a query is posed to a client site, a metadata dictionary is used to translate the query into queries appropriate for the individual heterogeneous sites involved. These

queries are then mapped and sent to local query processors. The results returned from the different sites are integrated into a global answer set. This **query-driven approach** requires complex information filtering and integration processes, and competes with local sites for processing resources. It is inefficient and potentially expensive for frequent queries, especially queries requiring aggregations.

Data warehousing provides an interesting alternative to this traditional approach. Rather than using a query-driven approach, data warehousing employs an **update-driven** approach in which information from multiple, heterogeneous sources is integrated in advance and stored in a warehouse for direct querying and analysis. Unlike online transaction processing databases, data warehouses do not contain the most current information. However, a data warehouse brings high performance to the integrated heterogeneous database system because data are copied, preprocessed, integrated, annotated, summarized, and restructured into one semantic data store. Furthermore, query processing in data warehouses does not interfere with the processing at local sources. Moreover, data warehouses can store and integrate historic information and support complex multidimensional queries. As a result, data warehousing has become popular in industry.

2. Explain the Differences between Operational Database Systems and Data Warehouses.

4.1.2 Differences between Operational Database Systems and Data Warehouses

Because most people are familiar with commercial relational database systems, it is easy to understand what a data warehouse is by comparing these two kinds of systems.

The major task of online operational database systems is to perform online transaction and query processing. These systems are called **online transaction processing (OLTP)** systems. They cover most of the day-to-day operations of an organization such as purchasing, inventory, manufacturing, banking, payroll, registration, and accounting. Data warehouse systems, on the other hand, serve users or knowledge workers in the role of data analysis and decision making. Such systems can organize and present data in various formats in order to accommodate the diverse needs of different users. These systems are known as **online analytical processing (OLAP)** systems.

The major distinguishing features of OLTP and OLAP are summarized as follows:

- **Users and system orientation**: An OLTP system is *customer-oriented* and is used for transaction and query processing by clerks, clients, and information technology professionals. An OLAP system is *market-oriented* and is used for data analysis by knowledge workers, including managers, executives, and analysts.
- **Data contents**: An OLTP system manages current data that, typically, are too detailed to be easily used for decision making. An OLAP system manages large amounts of historic data, provides facilities for summarization and aggregation, and stores and manages information at different levels of granularity. These features make the data easier to use for informed decision making.

- **Database design**: An OLTP system usually adopts an entity-relationship (ER) data model and an application-oriented database design. An OLAP system typically adopts either a *star* or a *snowflake* model (see Section 4.2.2) and a subject-oriented database design.
- **View**: An OLTP system focuses mainly on the current data within an enterprise or department, without referring to historic data or data in different organizations. In contrast, an OLAP system often spans multiple versions of a database schema, due to the evolutionary process of an organization. OLAP systems also deal with information that originates from different organizations, integrating information from many data stores. Because of their huge volume, OLAP data are stored on multiple storage media.
- **Access patterns**: The access patterns of an OLTP system consist mainly of short, atomic transactions. Such a system requires concurrency control and recovery mechanisms. However, accesses to OLAP systems are mostly read-only operations (because most data warehouses store historic rather than up-to-date information), although many could be complex queries.

Other features that distinguish between OLTP and OLAP systems include database size, frequency of operations, and performance metrics. These are summarized in Table 4.1.

4.1.3 But, Why Have a Separate Data Warehouse?

Because operational databases store huge amounts of data, you may wonder, "*Why not perform online analytical processing directly on such databases instead of spending additional time and resources to construct a separate data warehouse?*" A major reason for such a separation is to help promote the *high performance of both systems*. An operational database is designed and tuned from known tasks and workloads like indexing and hashing using primary keys, searching for particular records, and optimizing "canned" queries. On the other hand, data warehouse queries are often complex. They involve the computation of large data groups at summarized levels, and may require the use of special data organization, access, and implementation methods based on multidimensional views. Processing OLAP queries in operational databases would substantially degrade the performance of operational tasks.

Moreover, an operational database supports the concurrent processing of multiple transactions. Concurrency control and recovery mechanisms (e.g., locking and logging) are required to ensure the consistency and robustness of transactions. An OLAP query often needs read-only access of data records for summarization and aggregation. Concurrency control and recovery mechanisms, if applied for such OLAP operations, may jeopardize the execution of concurrent transactions and thus substantially reduce the throughput of an OLTP system.

Finally, the separation of operational databases from data warehouses is based on the different structures, contents, and uses of the data in these two systems. Decision

2. Explain the Differences between Operational Database Systems and Data Warehouses.

Table 4.1 Comparison of OLTP and OLAP Systems

Feature	*OLTP*	*OLAP*
Characteristic	operational processing	informational processing
Orientation	transaction	analysis
User	clerk, DBA, database professional	knowledge worker (e.g., manager, executive, analyst)
Function	day-to-day operations	long-term informational requirements decision support
DB design	ER-based, application-oriented	star/snowflake, subject-oriented
Data	current, guaranteed up-to-date	historic, accuracy maintained over time
Summarization	primitive, highly detailed	summarized, consolidated
View	detailed, flat relational	summarized, multidimensional
Unit of work	short, simple transaction	complex query
Access	read/write	mostly read
Focus	data in	information out
Operations	index/hash on primary key	lots of scans
Number of records accessed	tens	millions
Number of users	thousands	hundreds
DB size	GB to high-order GB	$\geq$ TB
Priority	high performance, high availability	high flexibility, end-user autonomy
Metric	transaction throughput	query throughput, response time

Note: Table is partially based on Chaudhuri and Dayal [CD97].

support requires historic data, whereas operational databases do not typically maintain historic data. In this context, the data in operational databases, though abundant, are usually far from complete for decision making. Decision support requires consolidation (e.g., aggregation and summarization) of data from heterogeneous sources, resulting in high-quality, clean, integrated data. In contrast, operational databases contain only detailed raw data, such as transactions, which need to be consolidated before analysis. Because the two systems provide quite different functionalities and require different kinds of data, it is presently necessary to maintain separate databases. However, many vendors of operational relational database management systems are beginning to optimize such systems to support OLAP queries. As this trend continues, the separation between OLTP and OLAP systems is expected to decrease.

3. Discuss three-tier data warehousing architecture.

4.1.4 Data Warehousing: A Multitiered Architecture

Data warehouses often adopt a three-tier architecture, as presented in Figure 4.1.

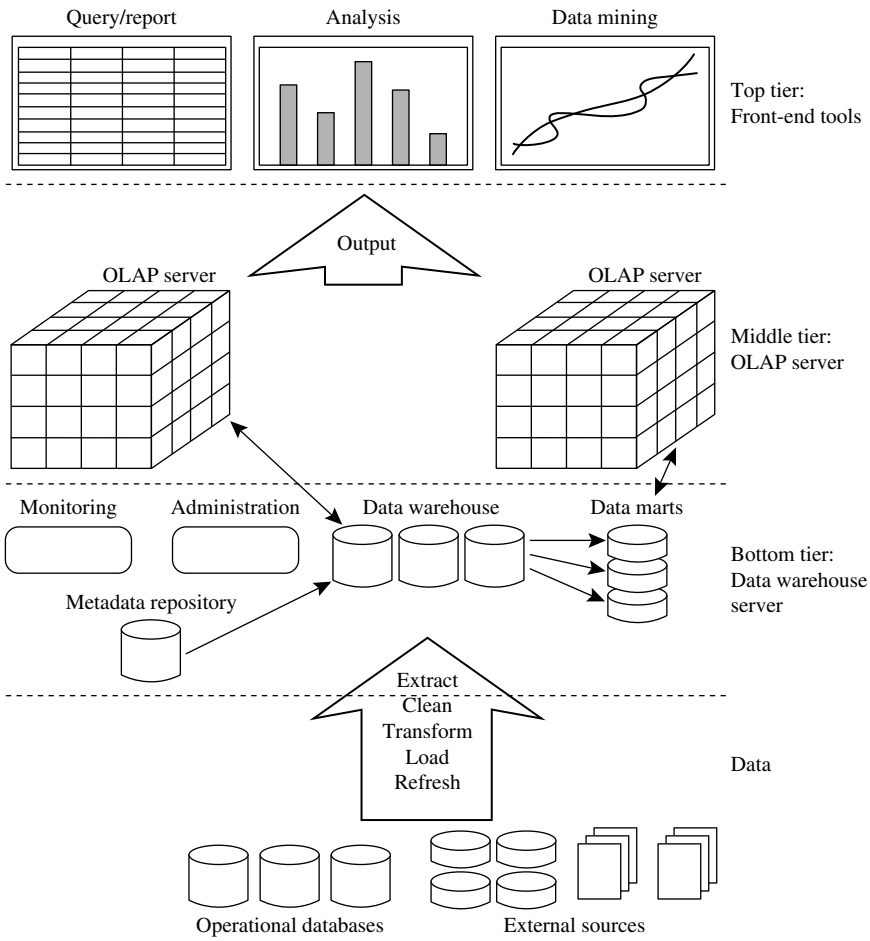


Figure 4.1 A three-tier data warehousing architecture.

1. The bottom tier is a **warehouse database server** that is almost always a relational database system. Back-end tools and utilities are used to feed data into the bottom tier from operational databases or other external sources (e.g., customer profile information provided by external consultants). These tools and utilities perform data extraction, cleaning, and transformation (e.g., to merge similar data from different sources into a unified format), as well as load and refresh functions to update the data warehouse (see Section 4.1.6). The data are extracted using application program interfaces known as **gateways**. A gateway is supported by the underlying DBMS and allows client programs to generate SQL code to be executed at a server. Examples of gateways include ODBC (Open Database Connection) and OLEDB (Object

Linking and Embedding Database) by Microsoft and JDBC (Java Database Connection). This tier also contains a metadata repository, which stores information about the data warehouse and its contents. The metadata repository is further described in Section 4.1.7.

2. The middle tier is an **OLAP server** that is typically implemented using either (1) a **relational OLAP (ROLAP)** model (i.e., an extended relational DBMS that maps operations on multidimensional data to standard relational operations); or (2) a **multidimensional OLAP (MOLAP)** model (i.e., a special-purpose server that directly implements multidimensional data and operations). OLAP servers are discussed in Section 4.4.4.

3. The top tier is a **front-end client layer**, which contains query and reporting tools, analysis tools, and/or data mining tools (e.g., trend analysis, prediction, and so on).

4. Explain the following models: Enterprise Warehouse, Data Mart, and Virtual Warehouse

4.1.5 Data Warehouse Models: Enterprise Warehouse, Data Mart, and Virtual Warehouse

From the architecture point of view, there are three data warehouse models: the *enterprise warehouse*, the *data mart*, and the *virtual warehouse*.

Enterprise warehouse: An enterprise warehouse collects all of the information about subjects spanning the entire organization. It provides corporate-wide data integration, usually from one or more operational systems or external information providers, and is cross-functional in scope. It typically contains detailed data as well as summarized data, and can range in size from a few gigabytes to hundreds of gigabytes, terabytes, or beyond. An enterprise data warehouse may be implemented on traditional mainframes, computer superservers, or parallel architecture platforms. It requires extensive business modeling and may take years to design and build.

Data mart: A data mart contains a subset of corporate-wide data that is of value to a specific group of users. The scope is confined to specific selected subjects. For example, a marketing data mart may confine its subjects to customer, item, and sales. The data contained in data marts tend to be summarized.

Data marts are usually implemented on low-cost departmental servers that are Unix/Linux or Windows based. The implementation cycle of a data mart is more likely to be measured in weeks rather than months or years. However, it may involve complex integration in the long run if its design and planning were not enterprise-wide.

Depending on the source of data, data marts can be categorized as independent or dependent. *Independent* data marts are sourced from data captured from one or more operational systems or external information providers, or from data generated locally within a particular department or geographic area. *Dependent* data marts are sourced directly from enterprise data warehouses.

Virtual warehouse: A virtual warehouse is a set of views over operational databases. For efficient query processing, only some of the possible summary views may be materialized. A virtual warehouse is easy to build but requires excess capacity on operational database servers.

"*What are the pros and cons of the top-down and bottom-up approaches to data warehouse development?*" The top-down development of an enterprise warehouse serves as a systematic solution and minimizes integration problems. However, it is expensive, takes a long time to develop, and lacks flexibility due to the difficulty in achieving consistency and consensus for a common data model for the entire organization. The bottom-up approach to the design, development, and deployment of independent data marts provides flexibility, low cost, and rapid return of investment. It, however, can lead to problems when integrating various disparate data marts into a consistent enterprise data warehouse.

A recommended method for the development of data warehouse systems is to implement the warehouse in an incremental and evolutionary manner, as shown in Figure 4.2. First, a high-level corporate data model is defined within a reasonably short period (such as one or two months) that provides a corporate-wide, consistent, integrated view of data among different subjects and potential usages. This high-level model, although it will need to be refined in the further development of enterprise data warehouses and departmental data marts, will greatly reduce future integration problems. Second, independent data marts can be implemented in parallel with the enterprise

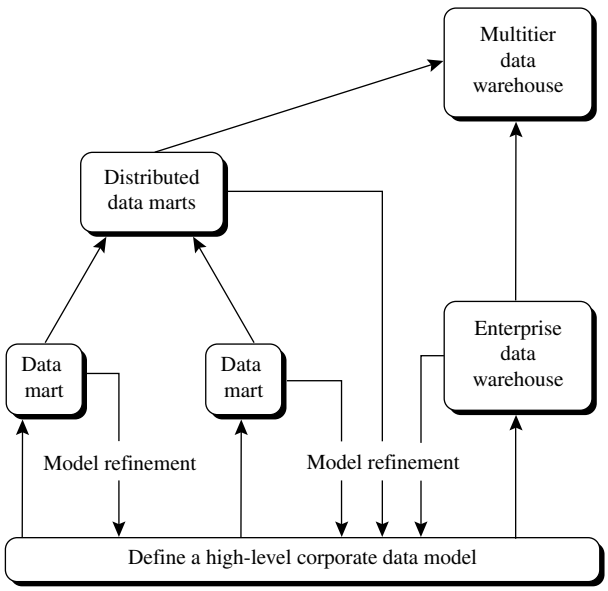


Figure 4.2 A recommended approach for data warehouse development.

warehouse based on the same corporate data model set noted before. Third, distributed data marts can be constructed to integrate different data marts via hub servers. Finally, a **multitier data warehouse** is constructed where the enterprise warehouse is the sole custodian of all warehouse data, which is then distributed to the various dependent data marts.

5. Data cleaning, data transformation, refresh

4.1.6 Extraction, Transformation, and Loading

Data warehouse systems use back-end tools and utilities to populate and refresh their data (Figure 4.1). These tools and utilities include the following functions:

- **Data extraction**, which typically gathers data from multiple, heterogeneous, and external sources.
- **Data cleaning**, which detects errors in the data and rectifies them when possible.
- **Data transformation**, which converts data from legacy or host format to warehouse format.
- **Load**, which sorts, summarizes, consolidates, computes views, checks integrity, and builds indices and partitions.
- **Refresh**, which propagates the updates from the data sources to the warehouse.

Besides cleaning, loading, refreshing, and metadata definition tools, data warehouse systems usually provide a good set of data warehouse management tools.

Data cleaning and data transformation are important steps in improving the data quality and, subsequently, the data mining results (see Chapter 3). Because we are mostly interested in the aspects of data warehousing technology related to data mining, we will not get into the details of the remaining tools, and recommend interested readers to consult books dedicated to data warehousing technology.

6. Explain Metadata Repository of DW.

4.1.7 Metadata Repository

Metadata are data about data. When used in a data warehouse, metadata are the data that define warehouse objects. Figure 4.1 showed a metadata repository within the bottom tier of the data warehousing architecture. Metadata are created for the data names and definitions of the given warehouse. Additional metadata are created and captured for timestamping any extracted data, the source of the extracted data, and missing fields that have been added by data cleaning or integration processes.

A metadata repository should contain the following:

- A description of the *data warehouse structure*, which includes the warehouse schema, view, dimensions, hierarchies, and derived data definitions, as well as data mart locations and contents.

- *Operational metadata*, which include data lineage (history of migrated data and the sequence of transformations applied to it), currency of data (active, archived, or purged), and monitoring information (warehouse usage statistics, error reports, and audit trails).
- The *algorithms used for summarization*, which include measure and dimension definition algorithms, data on granularity, partitions, subject areas, aggregation, summarization, and predefined queries and reports.
- *Mapping from the operational environment to the data warehouse*, which includes source databases and their contents, gateway descriptions, data partitions, data extraction, cleaning, transformation rules and defaults, data refresh and purging rules, and security (user authorization and access control).
- *Data related to system performance*, which include indices and profiles that improve data access and retrieval performance, in addition to rules for the timing and scheduling of refresh, update, and replication cycles.
- *Business metadata*, which include business terms and definitions, data ownership information, and charging policies.

A data warehouse contains different levels of summarization, of which metadata is one. Other types include current detailed data (which are almost always on disk), older detailed data (which are usually on tertiary storage), lightly summarized data, and highly summarized data (which may or may not be physically housed).

Metadata play a very different role than other data warehouse data and are important for many reasons. For example, metadata are used as a directory to help the decision support system analyst locate the contents of the data warehouse, and as a guide to the data mapping when data are transformed from the operational environment to the data warehouse environment. Metadata also serve as a guide to the algorithms used for summarization between the current detailed data and the lightly summarized data, and between the lightly summarized data and the highly summarized data. Metadata should be stored and managed persistently (i.e., on disk).

4.2 Data Warehouse Modeling: Data Cube and OLAP

Data warehouses and OLAP tools are based on a **multidimensional data model**. This model views data in the form of a *data cube*. In this section, you will learn how data cubes model *n*-dimensional data (Section 4.2.1). In Section 4.2.2, various multidimensional models are shown: star schema, snowflake schema, and fact constellation. You will also learn about concept hierarchies (Section 4.2.3) and measures (Section 4.2.4) and how they can be used in basic OLAP operations to allow interactive mining at multiple levels of abstraction. Typical OLAP operations such as drill-down and roll-up are illustrated

(Section 4.2.5). Finally, the starnet model for querying multidimensional databases is presented (Section 4.2.6).

7. What is a Data Cube? Explain Multidimensional Data Cube Model

4.2.1 Data Cube: A Multidimensional Data Model

"What is a data cube?" A **data cube** allows data to be modeled and viewed in multiple dimensions. It is defined by dimensions and facts.

In general terms, **dimensions** are the perspectives or entities with respect to which an organization wants to keep records. For example, *AllElectronics* may create a *sales* data warehouse in order to keep records of the store's sales with respect to the dimensions *time*, *item*, *branch*, and *location*. These dimensions allow the store to keep track of things like monthly sales of items and the branches and locations at which the items were sold. Each dimension may have a table associated with it, called a **dimension table**, which further describes the dimension. For example, a dimension table for *item* may contain the attributes *item_name, brand*, and *type*. Dimension tables can be specified by users or experts, or automatically generated and adjusted based on data distributions.

A multidimensional data model is typically organized around a central theme, such as *sales*. This theme is represented by a fact table. **Facts** are numeric measures. Think of them as the quantities by which we want to analyze relationships between dimensions. Examples of facts for a sales data warehouse include *dollars_sold* (sales amount in dollars), *units_sold* (number of units sold), and *amount_budgeted*. The **fact table** contains the names of the *facts*, or measures, as well as keys to each of the related dimension tables. You will soon get a clearer picture of how this works when we look at multidimensional schemas.

Although we usually think of cubes as 3-D geometric structures, in data warehousing the data cube is *n*-dimensional. To gain a better understanding of data cubes and the multidimensional data model, let's start by looking at a simple 2-D data cube that is, in fact, a table or spreadsheet for sales data from *AllElectronics*. In particular, we will look at the *AllElectronics* sales data for items sold per quarter in the city of Vancouver. These data are shown in Table 4.2. In this 2-D representation, the sales for Vancouver are shown with respect to the *time* dimension (organized in quarters) and the *item* dimension (organized according to the types of items sold). The fact or measure displayed is *dollars_sold* (in thousands).

Now, suppose that we would like to view the sales data with a third dimension. For instance, suppose we would like to view the data according to *time* and *item*, as well as *location*, for the cities Chicago, New York, Toronto, and Vancouver. These 3-D data are shown in Table 4.3. The 3-D data in the table are represented as a series of 2-D tables. Conceptually, we may also represent the same data in the form of a 3-D data cube, as in Figure 4.3.

Suppose that we would now like to view our sales data with an additional fourth dimension such as *supplier*. Viewing things in 4-D becomes tricky. However, we can think of a 4-D cube as being a series of 3-D cubes, as shown in Figure 4.4. If we continue

Table 4.2 2-D View of Sales Data for *AllElectronics* According to *time* and *item*

	location = *"Vancouver"*			
	item *(type)*			
	home			
time *(quarter)*	*entertainment*	*computer*	*phone*	*security*
Q1	605	825	14	400
Q2	680	952	31	512
Q3	812	1023	30	501
Q4	927	1038	38	580

Note: The sales are from branches located in the city of Vancouver. The measure displayed is *dollars_sold* (in thousands).

Table 4.3 3-D View of Sales Data for *AllElectronics* According to *time*, *item*, and *location*

	location = "Chicago"				***location*** = "New York"				***location*** = "Toronto"				***location*** = "Vancouver"			
	item				***item***				***item***				***item***			
	home				*home*				*home*				*home*			
time	*ent.*	*comp.*	*phone*	*sec.*	*ent.*	*comp.*	*phone*	*sec.*	*ent.*	*comp.*	*phone*	*sec.*	*ent.*	*comp.*	*phone*	*sec.*
Q1	854	882	89	623	1087	968	38	872	818	746	43	591	605	825	14	400
Q2	943	890	64	698	1130	1024	41	925	894	769	52	682	680	952	31	512
Q3	1032	924	59	789	1034	1048	45	1002	940	795	58	728	812	1023	30	501
Q4	1129	992	63	870	1142	1091	54	984	978	864	59	784	927	1038	38	580

Note: The measure displayed is *dollars_sold* (in thousands).

in this way, we may display any n-dimensional data as a series of $(n-1)$-dimensional "cubes." The data cube is a metaphor for multidimensional data storage. The actual physical storage of such data may differ from its logical representation. The important thing to remember is that data cubes are n-dimensional and do not confine data to 3-D.

Tables 4.2 and 4.3 show the data at different degrees of summarization. In the data warehousing research literature, a data cube like those shown in Figures 4.3 and 4.4 is often referred to as a **cuboid**. Given a set of dimensions, we can generate a cuboid for each of the possible subsets of the given dimensions. The result would form a *lattice* of cuboids, each showing the data at a different level of summarization, or group-by. The lattice of cuboids is then referred to as a data cube. Figure 4.5 shows a lattice of cuboids forming a data cube for the dimensions *time*, *item*, *location*, and *supplier*.

The cuboid that holds the lowest level of summarization is called the **base cuboid**. For example, the 4-D cuboid in Figure 4.4 is the base cuboid for the given *time*, *item*, *location*, and *supplier* dimensions. Figure 4.3 is a 3-D (nonbase) cuboid for *time*, *item*,

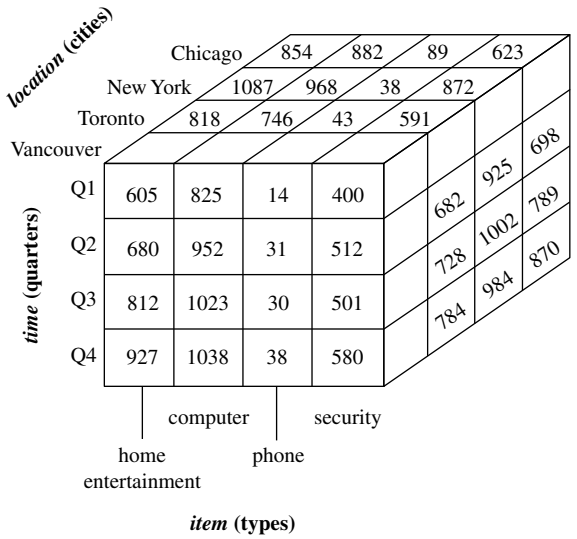


Figure 4.3 A 3-D data cube representation of the data in Table 4.3, according to *time*, *item*, and *location*. The measure displayed is *dollars_sold* (in thousands).

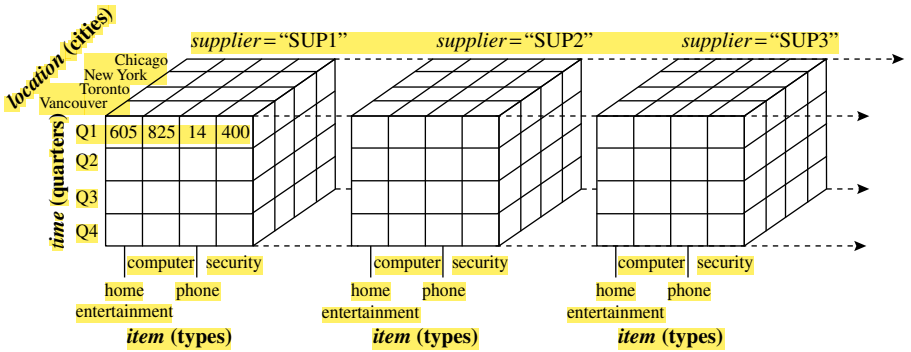


Figure 4.4 A 4-D data cube representation of sales data, according to *time*, *item*, *location*, and *supplier*. The measure displayed is *dollars_sold* (in thousands). For improved readability, only some of the cube values are shown.

and *location*, summarized for all suppliers. The 0-D cuboid, which holds the highest level of summarization, is called the **apex cuboid**. In our example, this is the total sales, or *dollars_sold*, summarized over all four dimensions. The apex cuboid is typically denoted by all.

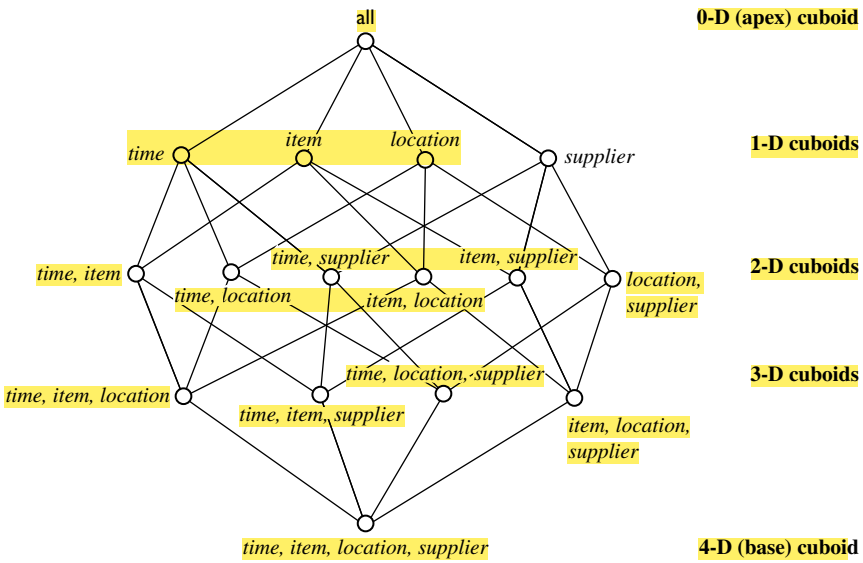


Figure 4.5 Lattice of cuboids, making up a 4-D data cube for *time*, *item*, *location*, and *supplier*. Each cuboid represents a different degree of summarization.

8. Discuss Snowflake schema, fact constellation, starnet query model of Multidimensional Data model.

4.2.2 Stars, Snowflakes, and Fact Constellations: Schemas for Multidimensional Data Models

The entity-relationship data model is commonly used in the design of relational databases, where a database schema consists of a set of entities and the relationships between them. Such a data model is appropriate for online transaction processing. A data warehouse, however, requires a concise, subject-oriented schema that facilitates online data analysis.

The most popular data model for a data warehouse is a **multidimensional model**, which can exist in the form of a **star schema**, a **snowflake schema**, or a **fact constellation schema**. Let's look at each of these.

Star schema: The most common modeling paradigm is the star schema, in which the data warehouse contains (1) a large central table (**fact table**) containing the bulk of the data, with no redundancy, and (2) a set of smaller attendant tables (**dimension tables**), one for each dimension. The schema graph resembles a starburst, with the dimension tables displayed in a radial pattern around the central fact table.

Example 4.1 **Star schema.** A star schema for *AllElectronics* sales is shown in Figure 4.6. Sales are considered along four dimensions: *time, item, branch*, and *location*. The schema contains a central fact table for *sales* that contains keys to each of the four dimensions, along

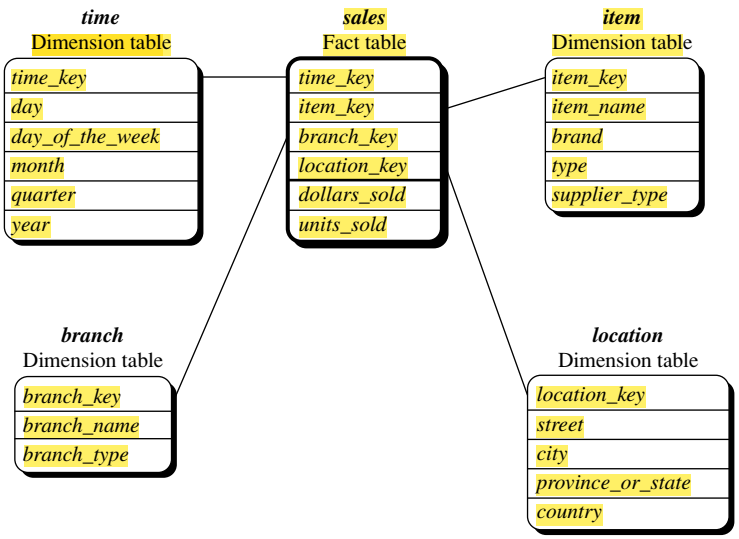


Figure 4.6 Star schema of *sales* data warehouse.

with two measures: *dollars_sold* and *units_sold*. To minimize the size of the fact table, dimension identifiers (e.g., *time_key* and *item_key*) are system-generated identifiers. ■

Notice that in the star schema, each dimension is represented by only one table, and each table contains a set of attributes. For example, the *location* dimension table contains the attribute set {*location_key, street, city, province_or_state, country*}. This constraint may introduce some redundancy. For example, "Urbana" and "Chicago" are both cities in the state of Illinois, USA. Entries for such cities in the *location* dimension table will create redundancy among the attributes *province_or_state* and *country*; that is, (..., Urbana, IL, USA) and (..., Chicago, IL, USA). Moreover, the attributes within a dimension table may form either a hierarchy (total order) or a lattice (partial order).

Snowflake schema: The snowflake schema is a variant of the star schema model, where some dimension tables are *normalized*, thereby further splitting the data into additional tables. The resulting schema graph forms a shape similar to a snowflake.

The major difference between the snowflake and star schema models is that the dimension tables of the snowflake model may be kept in normalized form to reduce redundancies. Such a table is easy to maintain and saves storage space. However, this space savings is negligible in comparison to the typical magnitude of the fact table. Furthermore, the snowflake structure can reduce the effectiveness of browsing, since more joins will be needed to execute a query. Consequently, the system performance may be adversely impacted. Hence, although the snowflake schema reduces redundancy, it is not as popular as the star schema in data warehouse design.

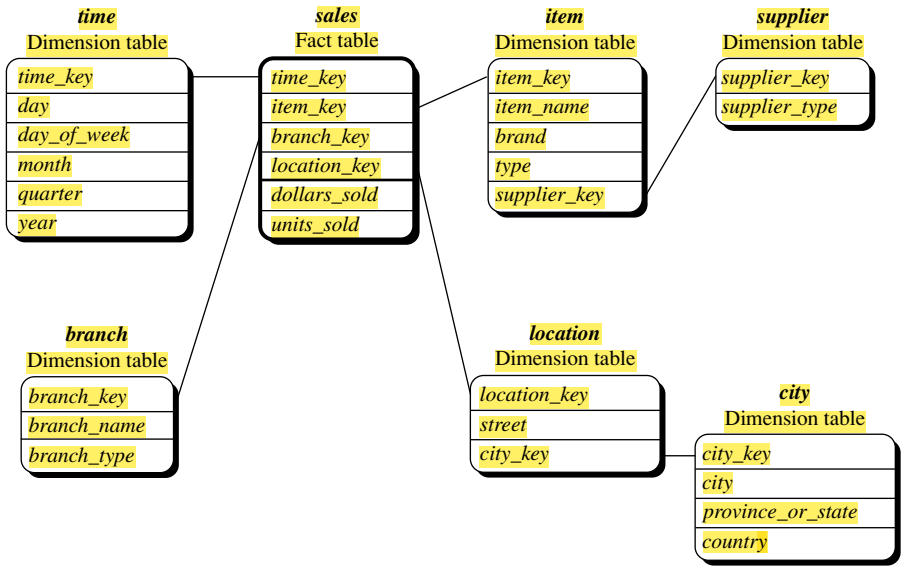


Figure 4.7 Snowflake schema of a *sales* data warehouse.

Example 4.2 **Snowflake schema.** A snowflake schema for *AllElectronics* sales is given in Figure 4.7. Here, the *sales* fact table is identical to that of the star schema in Figure 4.6. The main difference between the two schemas is in the definition of dimension tables. The single dimension table for *item* in the star schema is normalized in the snowflake schema, resulting in new *item* and *supplier* tables. For example, the *item* dimension table now contains the attributes *item_key, item_name, brand, type*, and *supplier_key*, where *supplier_key* is linked to the *supplier* dimension table, containing *supplier_key* and *supplier_type* information. Similarly, the single dimension table for *location* in the star schema can be normalized into two new tables: *location* and *city*. The *city_key* in the new *location* table links to the *city* dimension. Notice that, when desirable, further normalization can be performed on *province_or_state* and *country* in the snowflake schema shown in Figure 4.7. ■

Fact constellation: Sophisticated applications may require multiple fact tables to *share* dimension tables. This kind of schema can be viewed as a collection of stars, and hence is called a **galaxy schema** or a **fact constellation**.

Example 4.3 **Fact constellation.** A fact constellation schema is shown in Figure 4.8. This schema specifies two fact tables, *sales* and *shipping*. The *sales* table definition is identical to that of the star schema (Figure 4.6). The *shipping* table has five dimensions, or keys—*item_key, time_key, shipper_key, from_location*, and *to_location*—and two measures—*dollars_cost*

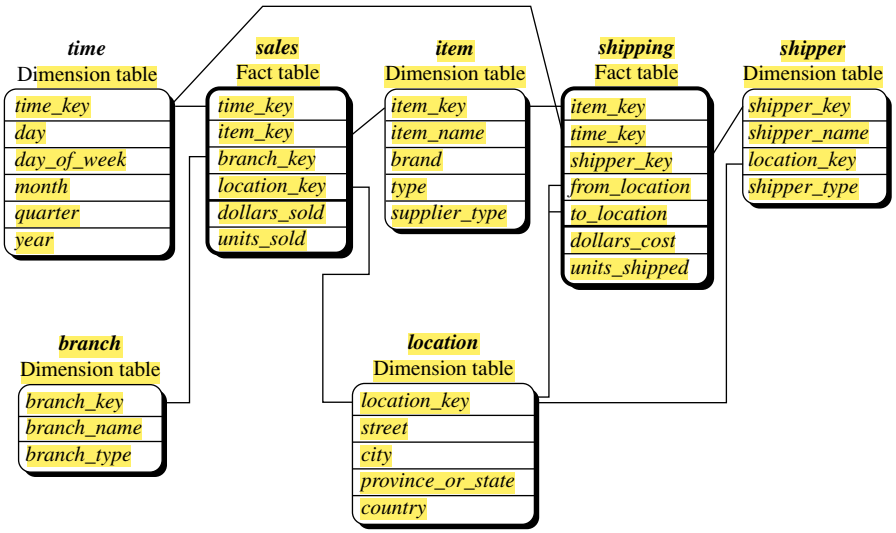


Figure 4.8 Fact constellation schema of a sales and shipping data warehouse.

and *units_shipped*. A fact constellation schema allows dimension tables to be shared between fact tables. For example, the dimensions tables for *time, item*, and *location* are shared between the *sales* and *shipping* fact tables. ■

In data warehousing, there is a distinction between a data warehouse and a data mart. A data warehouse collects information about subjects that span the *entire organization*, such as *customers, items, sales, assets*, and *personnel*, and thus its scope is *enterprise-wide*. For data warehouses, the fact constellation schema is commonly used, since it can model multiple, interrelated subjects. A **data mart**, on the other hand, is a department subset of the data warehouse that focuses on selected subjects, and thus its scope is *department-wide*. For data marts, the *star* or *snowflake* schema is commonly used, since both are geared toward modeling single subjects, although the star schema is more popular and efficient.

9. Explain the Role of Concept Hierarchies in DW

4.2.3 Dimensions: The Role of Concept Hierarchies

A **concept hierarchy** defines a sequence of mappings from a set of low-level concepts to higher-level, more general concepts. Consider a concept hierarchy for the dimension *location*. City values for *location* include Vancouver, Toronto, New York, and Chicago. Each city, however, can be mapped to the province or state to which it belongs. For example, Vancouver can be mapped to British Columbia, and Chicago to Illinois. The provinces and states can in turn be mapped to the country (e.g., Canada or the United States) to which they belong. These mappings form a concept hierarchy for the

dimension *location*, mapping a set of low-level concepts (i.e., cities) to higher-level, more general concepts (i.e., countries). This concept hierarchy is illustrated in Figure 4.9.

Many concept hierarchies are implicit within the database schema. For example, suppose that the dimension *location* is described by the attributes *number, street, city, province_or_state, zip_code*, and *country*. These attributes are related by a total order, forming a concept hierarchy such as "*street* < *city* < *province_or_state* < *country*." This hierarchy is shown in Figure 4.10(a). Alternatively, the attributes of a dimension may

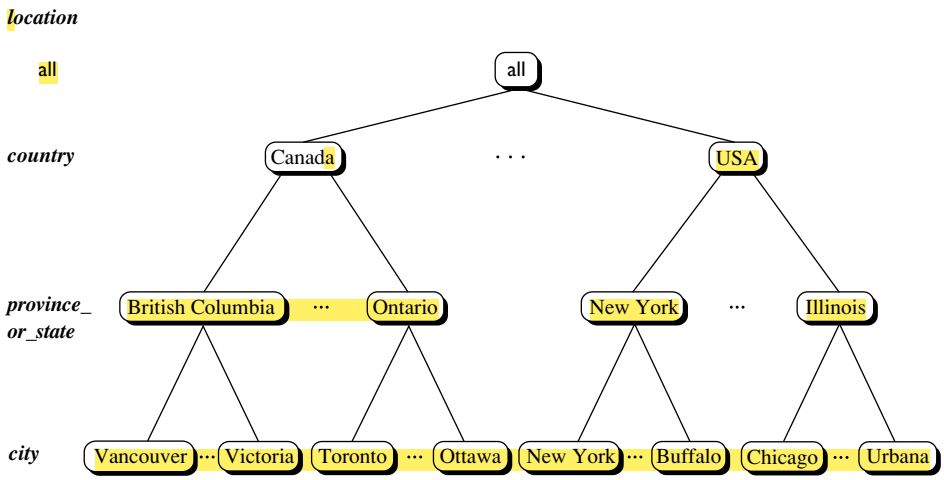


Figure 4.9 A concept hierarchy for *location*. Due to space limitations, not all of the hierarchy nodes are shown, indicated by ellipses between nodes.

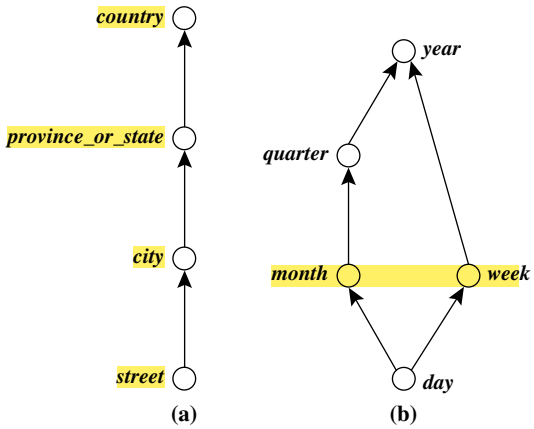


Figure 4.10 Hierarchical and lattice structures of attributes in warehouse dimensions: (a) a hierarchy for *location* and (b) a lattice for *time*.

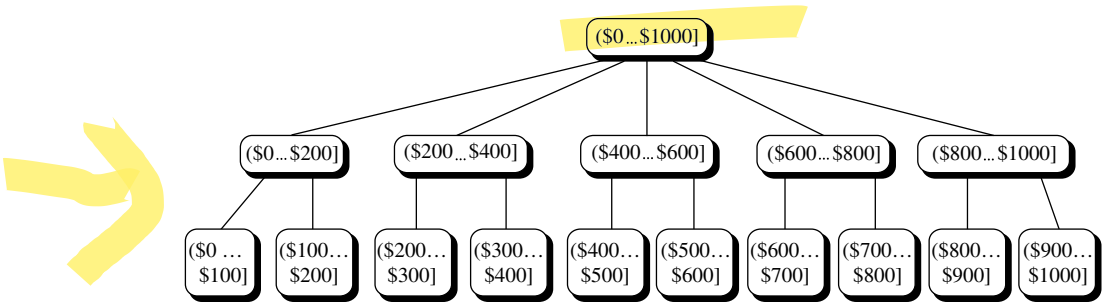


Figure 4.11 A concept hierarchy for *price*.

be organized in a partial order, forming a lattice. An example of a partial order for the *time* dimension based on the attributes *day, week, month, quarter*, and *year* is "*day* < {*month* < *quarter; week*} < *year*."[1] This lattice structure is shown in Figure 4.10(b). A concept hierarchy that is a total or partial order among attributes in a database schema is called a **schema hierarchy**. Concept hierarchies that are common to many applications (e.g., *for time*) may be predefined in the data mining system. Data mining systems should provide users with the flexibility to tailor predefined hierarchies according to their particular needs. For example, users may want to define a fiscal year starting on April 1 or an academic year starting on September 1.

Concept hierarchies may also be defined by discretizing or grouping values for a given dimension or attribute, resulting in a **set-grouping hierarchy**. A total or partial order can be defined among groups of values. An example of a set-grouping hierarchy is shown in Figure 4.11 for the dimension *price*, where an interval ($\$X \ldots \$Y]$ denotes the range from $\$X$ (exclusive) to $\$Y$ (inclusive).

There may be more than one concept hierarchy for a given attribute or dimension, based on different user viewpoints. For instance, a user may prefer to organize *price* by defining ranges for *inexpensive, moderately_priced*, and *expensive*.

Concept hierarchies may be provided manually by system users, domain experts, or knowledge engineers, or may be automatically generated based on statistical analysis of the data distribution. The automatic generation of concept hierarchies is discussed in Chapter 3 as a preprocessing step in preparation for data mining.

Concept hierarchies allow data to be handled at varying levels of abstraction, as we will see in Section 4.2.4.

4.2.4 Measures: Their Categorization and Computation

"*How are measures computed?*" To answer this question, we first study how measures can be categorized. Note that a *multidimensional point* in the data cube space can be defined

[1]Since a *week* often crosses the boundary of two consecutive months, it is usually not treated as a lower abstraction of *month*. Instead, it is often treated as a lower abstraction of *year*, since a year contains approximately 52 weeks.

10. Explain different aggregate functions used to measure of data in multidimensional way.

by a set of dimension–value pairs; for example, ⟨*time* = "Q1", *location* = "Vancouver", *item* = "computer"⟩. A data cube **measure** is a numeric function that can be evaluated at each point in the data cube space. A measure value is computed for a given point by aggregating the data corresponding to the respective dimension–value pairs defining the given point. We will look at concrete examples of this shortly.

Measures can be organized into three categories—distributive, algebraic, and holistic—based on the kind of aggregate functions used.

Distributive: An aggregate function is *distributive* if it can be computed in a distributed manner as follows. Suppose the data are partitioned into n sets. We apply the function to each partition, resulting in n aggregate values. If the result derived by applying the function to the n aggregate values is the same as that derived by applying the function to the entire data set (without partitioning), the function can be computed in a distributed manner. For example, sum() can be computed for a data cube by first partitioning the cube into a set of subcubes, computing sum() for each subcube, and then summing up the counts obtained for each subcube. Hence, sum() is a distributive aggregate function.

For the same reason, count(), min(), and max() are distributive aggregate functions. By treating the count value of each nonempty base cell as 1 by default, count() of any cell in a cube can be viewed as the sum of the count values of all of its corresponding child cells in its subcube. Thus, count() is distributive. A measure is *distributive* if it is obtained by applying a distributive aggregate function. Distributive measures can be computed efficiently because of the way the computation can be partitioned.

Algebraic: An aggregate function is *algebraic* if it can be computed by an algebraic function with M arguments (where M is a bounded positive integer), each of which is obtained by applying a distributive aggregate function. For example, avg() (average) can be computed by sum()/count(), where both sum() and count() are distributive aggregate functions. Similarly, it can be shown that min_N() and max_N() (which find the N minimum and N maximum values, respectively, in a given set) and standard_deviation() are algebraic aggregate functions. A measure is *algebraic* if it is obtained by applying an algebraic aggregate function.

Holistic: An aggregate function is *holistic* if there is no constant bound on the storage size needed to describe a subaggregate. That is, there does not exist an algebraic function with M arguments (where M is a constant) that characterizes the computation. Common examples of holistic functions include median(), mode(), and rank(). A measure is *holistic* if it is obtained by applying a holistic aggregate function.

Most large data cube applications require efficient computation of distributive and algebraic measures. Many efficient techniques for this exist. In contrast, it is difficult to compute holistic measures efficiently. Efficient techniques to *approximate* the computation of some holistic measures, however, do exist. For example, rather than computing the exact median(), Equation (2.3) of Chapter 2 can be used to estimate the approximate median value for a large data set. In many cases, such techniques are sufficient to overcome the difficulties of efficient computation of holistic measures.

Various methods for computing different measures in data cube construction are discussed in depth in Chapter 5. Notice that most of the current data cube technology confines the measures of multidimensional databases to *numeric data*. However, measures can also be applied to other kinds of data, such as spatial, multimedia, or text data.

11. Explain OLAP operations

4.2.5 Typical OLAP Operations

"*How are concept hierarchies useful in OLAP?*" In the multidimensional model, data are organized into multiple dimensions, and each dimension contains multiple levels of abstraction defined by concept hierarchies. This organization provides users with the flexibility to view data from different perspectives. A number of OLAP data cube operations exist to materialize these different views, allowing interactive querying and analysis of the data at hand. Hence, OLAP provides a user-friendly environment for interactive data analysis.

Example 4.4 **OLAP operations.** Let's look at some typical OLAP operations for multidimensional data. Each of the following operations described is illustrated in Figure 4.12. At the center of the figure is a data cube for *AllElectronics* sales. The cube contains the dimensions *location, time*, and *item*, where *location* is aggregated with respect to city values, *time* is aggregated with respect to quarters, and *item* is aggregated with respect to item types. To aid in our explanation, we refer to this cube as the central cube. The measure displayed is *dollars_sold* (in thousands). (For improved readability, only some of the cubes' cell values are shown.) The data examined are for the cities Chicago, New York, Toronto, and Vancouver.

Roll-up: The roll-up operation (also called the *drill-up* operation by some vendors) performs aggregation on a data cube, either by *climbing up a concept hierarchy* for a dimension or by *dimension reduction*. Figure 4.12 shows the result of a roll-up operation performed on the central cube by climbing up the concept hierarchy for *location* given in Figure 4.9. This hierarchy was defined as the total order "*street* < *city* < *province_or_state* < *country*." The roll-up operation shown aggregates the data by ascending the *location* hierarchy from the level of *city* to the level of *country*. In other words, rather than grouping the data by city, the resulting cube groups the data by country.

When roll-up is performed by dimension reduction, one or more dimensions are removed from the given cube. For example, consider a sales data cube containing only the *location* and *time* dimensions. Roll-up may be performed by removing, say, the *time* dimension, resulting in an aggregation of the total sales by location, rather than by location and by time.

Drill-down: Drill-down is the reverse of roll-up. It navigates from less detailed data to more detailed data. Drill-down can be realized by either *stepping down a concept hierarchy* for a dimension or *introducing additional dimensions*. Figure 4.12 shows the result of a drill-down operation performed on the central cube by stepping down a

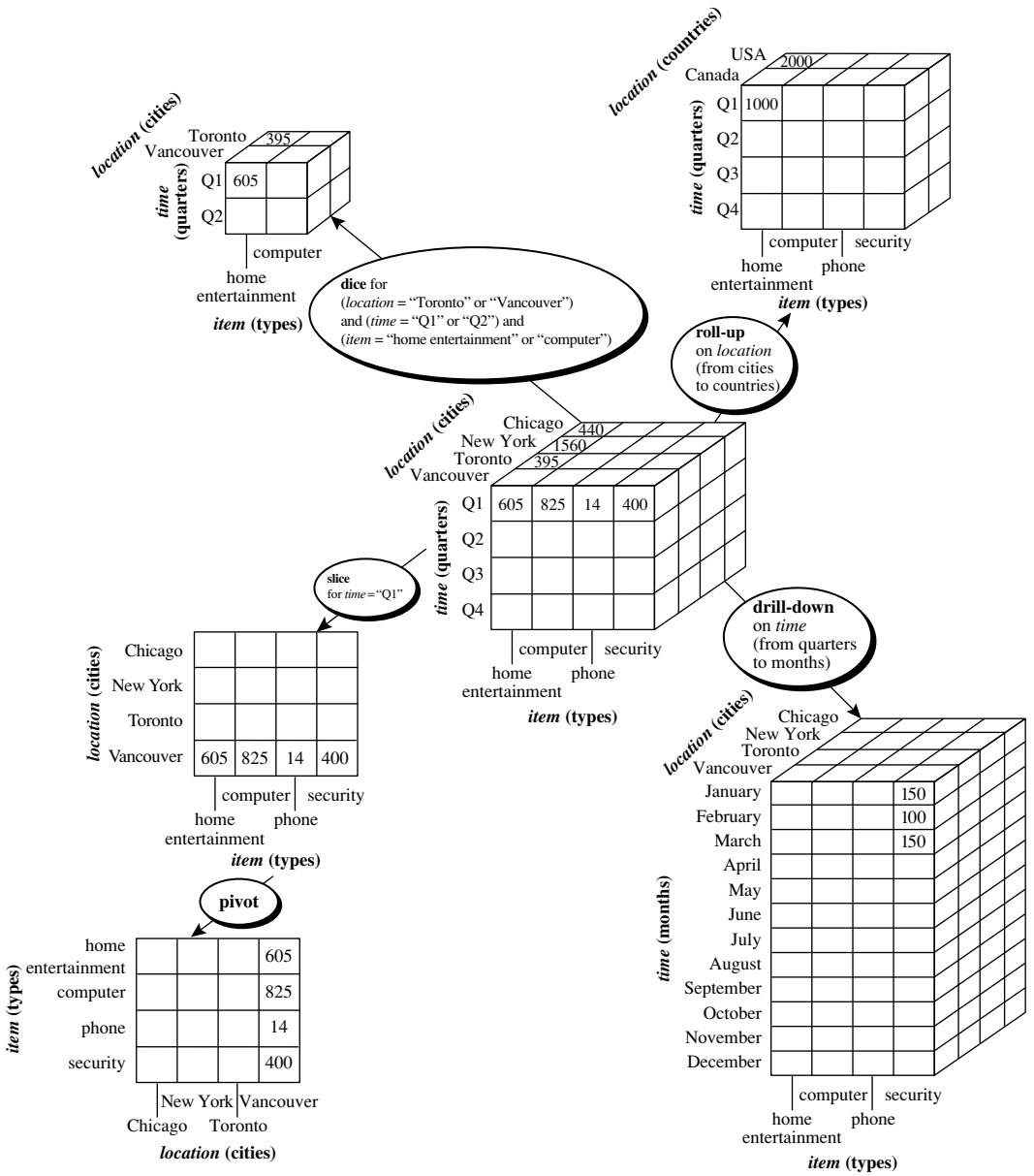


Figure 4.12 Examples of typical OLAP operations on multidimensional data.

concept hierarchy for *time* defined as "*day* < *month* < *quarter* < *year*." Drill-down occurs by descending the *time* hierarchy from the level of *quarter* to the more detailed level of *month*. The resulting data cube details the total sales per month rather than summarizing them by quarter.

Because a drill-down adds more detail to the given data, it can also be performed by adding new dimensions to a cube. For example, a drill-down on the central cube of Figure 4.12 can occur by introducing an additional dimension, such as *customer_group*.

Slice and dice: The *slice* operation performs a selection on one dimension of the given cube, resulting in a subcube. Figure 4.12 shows a slice operation where the sales data are selected from the central cube for the dimension *time* using the criterion *time* = "Q1." The *dice* operation defines a subcube by performing a selection on two or more dimensions. Figure 4.12 shows a dice operation on the central cube based on the following selection criteria that involve three dimensions: (*location* = "Toronto" or "Vancouver") and (*time* = "Q1" or "Q2") and (item = "home entertainment" or "computer").

Pivot (rotate): *Pivot* (also called *rotate*) is a visualization operation that rotates the data axes in view to provide an alternative data presentation. Figure 4.12 shows a pivot operation where the *item* and *location* axes in a 2-D slice are rotated. Other examples include rotating the axes in a 3-D cube, or transforming a 3-D cube into a series of 2-D planes.

Other OLAP operations: Some OLAP systems offer additional drilling operations. For example, **drill-across** executes queries involving (i.e., across) more than one fact table. The **drill-through** operation uses relational SQL facilities to drill through the bottom level of a data cube down to its back-end relational tables.

Other OLAP operations may include ranking the top N or bottom N items in lists, as well as computing moving averages, growth rates, interests, internal return rates, depreciation, currency conversions, and statistical functions. ■

OLAP offers analytical modeling capabilities, including a calculation engine for deriving ratios, variance, and so on, and for computing measures across multiple dimensions. It can generate summarizations, aggregations, and hierarchies at each granularity level and at every dimension intersection. OLAP also supports functional models for forecasting, trend analysis, and statistical analysis. In this context, an OLAP engine is a powerful data analysis tool.

OLAP Systems versus Statistical Databases

Many OLAP systems' characteristics (e.g., the use of a multidimensional data model and concept hierarchies, the association of measures with dimensions, and the notions of roll-up and drill-down) also exist in earlier work on statistical databases (SDBs). A **statistical database** is a database system that is designed to support statistical applications. Similarities between the two types of systems are rarely discussed, mainly due to differences in terminology and application domains.

OLAP and SDB systems, however, have distinguishing differences. While SDBs tend to focus on socioeconomic applications, OLAP has been targeted for business applications. Privacy issues regarding concept hierarchies are a major concern for SDBs. For example, given summarized socioeconomic data, it is controversial to allow users to view the corresponding low-level data. Finally, unlike SDBs, OLAP systems are designed for efficiently handling huge amounts of data.

10. starnet query model of Multidimensional

4.2.6 A Starnet Query Model for Querying Multidimensional Databases

The querying of multidimensional databases can be based on a **starnet model**, which consists of radial lines emanating from a central point, where each line represents a concept hierarchy for a dimension. Each abstraction level in the hierarchy is called a **footprint**. These represent the granularities available for use by OLAP operations such as drill-down and roll-up.

Example 4.5 **Starnet.** A starnet query model for the *AllElectronics* data warehouse is shown in Figure 4.13. This starnet consists of four radial lines, representing concept hierarchies for the dimensions *location*, *customer*, *item*, and *time*, respectively. Each line consists of footprints representing abstraction levels of the dimension. For example, the *time* line has four footprints: "day," "month," "quarter," and "year." A concept hierarchy may involve a single attribute (e.g., *date* for the *time* hierarchy) or several attributes (e.g., the concept hierarchy for *location* involves the attributes *street, city, province_or_state*, and *country*). In order to examine the item sales at *AllElectronics*, users can roll up along the

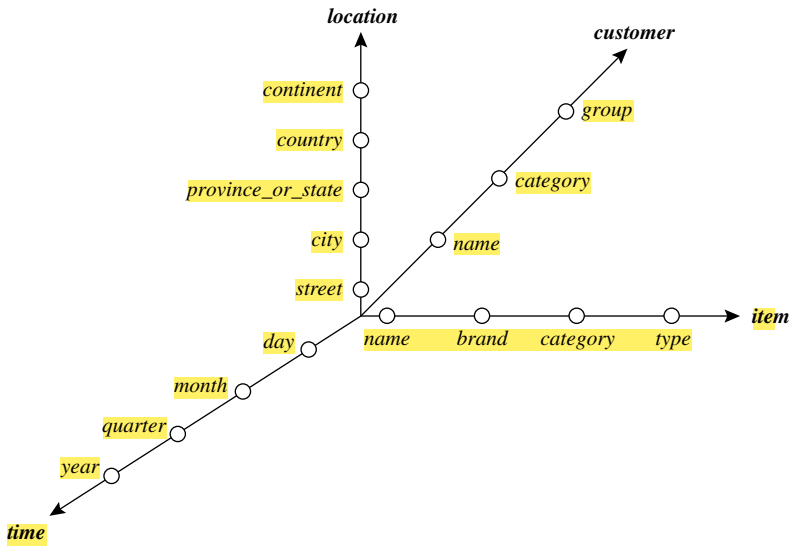


Figure 4.13 A starnet model of business queries.

time dimension from *month* to *quarter*, or, say, drill down along the *location* dimension from *country* to *city*.

Concept hierarchies can be used to **generalize** data by replacing low-level values (such as "day" for the *time* dimension) by higher-level abstractions (such as "year"), or to **specialize** data by replacing higher-level abstractions with lower-level values. ■

4.3 Data Warehouse Design and Usage

"What goes into a data warehouse design? How are data warehouses used? How do data warehousing and OLAP relate to data mining?" This section tackles these questions. We study the design and usage of data warehousing for information processing, analytical processing, and data mining. We begin by presenting a business analysis framework for data warehouse design (Section 4.3.1). Section 4.3.2 looks at the design process, while Section 4.3.3 studies data warehouse usage. Finally, Section 4.3.4 describes *multidimensional data mining*, a powerful paradigm that integrates OLAP with data mining technology.

4.3.1 A Business Analysis Framework for Data Warehouse Design

"*What can business analysts gain from having a data warehouse?*" First, having a data warehouse may provide a *competitive advantage* by presenting relevant information from which to measure performance and make critical adjustments to help win over competitors. Second, a data warehouse can enhance business *productivity* because it is able to quickly and efficiently gather information that accurately describes the organization. Third, a data warehouse facilitates *customer relationship management* because it provides a consistent view of customers and items across all lines of business, all departments, and all markets. Finally, a data warehouse may bring about *cost reduction* by tracking trends, patterns, and exceptions over long periods in a consistent and reliable manner.

To design an effective data warehouse we need to understand and analyze business needs and construct a *business analysis framework*. The construction of a large and complex information system can be viewed as the construction of a large and complex building, for which the owner, architect, and builder have different views. These views are combined to form a complex framework that represents the top-down, business-driven, or owner's perspective, as well as the bottom-up, builder-driven, or implementor's view of the information system.

Four different views regarding a data warehouse design must be considered: the *top-down view*, the *data source view*, the *data warehouse view*, and the *business query view*.

- The **top-down view** allows the selection of the relevant information necessary for the data warehouse. This information matches current and future business needs.
- The **data source view** exposes the information being captured, stored, and managed by operational systems. This information may be documented at various levels of detail and accuracy, from individual data source tables to integrated data source tables. Data sources are often modeled by traditional data modeling techniques, such as the entity-relationship model or CASE (computer-aided software engineering) tools.
- The **data warehouse view** includes fact tables and dimension tables. It represents the information that is stored inside the data warehouse, including precalculated totals and counts, as well as information regarding the source, date, and time of origin, added to provide historical context.
- Finally, the **business query view** is the data perspective in the data warehouse from the end-user's viewpoint.

Building and using a data warehouse is a complex task because it requires *business skills, technology skills*, and *program management skills.* Regarding *business skills*, building a data warehouse involves understanding how systems store and manage their data, how to build **extractors** that transfer data from the operational system to the data warehouse, and how to build **warehouse refresh software** that keeps the data warehouse reasonably up-to-date with the operational system's data. Using a data warehouse involves understanding the significance of the data it contains, as well as understanding and translating the business requirements into queries that can be satisfied by the data warehouse.

Regarding *technology skills*, data analysts are required to understand how to make assessments from quantitative information and derive facts based on conclusions from historic information in the data warehouse. These skills include the ability to discover patterns and trends, to extrapolate trends based on history and look for anomalies or paradigm shifts, and to present coherent managerial recommendations based on such analysis. Finally, *program management skills* involve the need to interface with many technologies, vendors, and end-users in order to deliver results in a timely and cost-effective manner.

4.3.2 Data Warehouse Design Process

Let's look at various approaches to the data warehouse design process and the steps involved.

A data warehouse can be built using a *top-down approach*, a *bottom-up approach*, or a *combination of both.* The **top-down approach** starts with overall design and planning. It is useful in cases where the technology is mature and well known, and where the business problems that must be solved are clear and well understood. The **bottom-up approach** starts with experiments and prototypes. This is useful in the early stage of business modeling and technology development. It allows an organization to move

forward at considerably less expense and to evaluate the technological benefits before making significant commitments. In the **combined approach**, an organization can exploit the planned and strategic nature of the top-down approach while retaining the rapid implementation and opportunistic application of the bottom-up approach.

From the software engineering point of view, the design and construction of a data warehouse may consist of the following steps: *planning, requirements study, problem analysis, warehouse design, data integration and testing*, and finally *deployment of the data warehouse*. Large software systems can be developed using one of two methodologies: the *waterfall method* or the *spiral method*. The **waterfall method** performs a structured and systematic analysis at each step before proceeding to the next, which is like a waterfall, falling from one step to the next. The **spiral method** involves the rapid generation of increasingly functional systems, with short intervals between successive releases. This is considered a good choice for data warehouse development, especially for data marts, because the turnaround time is short, modifications can be done quickly, and new designs and technologies can be adapted in a timely manner.

In general, the warehouse design process consists of the following steps:

1. Choose a *business process* to model (e.g., orders, invoices, shipments, inventory, account administration, sales, or the general ledger). If the business process is organizational and involves multiple complex object collections, a data warehouse model should be followed. However, if the process is departmental and focuses on the analysis of one kind of business process, a data mart model should be chosen.
2. Choose the business process *grain*, which is the fundamental, atomic level of data to be represented in the fact table for this process (e.g., individual transactions, individual daily snapshots, and so on).
3. Choose the *dimensions* that will apply to each fact table record. Typical dimensions are time, item, customer, supplier, warehouse, transaction type, and status.
4. Choose the *measures* that will populate each fact table record. Typical measures are numeric additive quantities like *dollars_sold* and *units_sold*.

Because data warehouse construction is a difficult and long-term task, its implementation scope should be clearly defined. The goals of an initial data warehouse implementation should be *specific, achievable*, and *measurable*. This involves determining the time and budget allocations, the subset of the organization that is to be modeled, the number of data sources selected, and the number and types of departments to be served.

Once a data warehouse is designed and constructed, the initial deployment of the warehouse includes initial installation, roll-out planning, training, and orientation. Platform upgrades and maintenance must also be considered. Data warehouse administration includes data refreshment, data source synchronization, planning for disaster recovery, managing access control and security, managing data growth, managing database performance, and data warehouse enhancement and extension. Scope

management includes controlling the number and range of queries, dimensions, and reports; limiting the data warehouse's size; or limiting the schedule, budget, or resources.

Various kinds of data warehouse design tools are available. **Data warehouse development tools** provide functions to define and edit metadata repository contents (e.g., schemas, scripts, or rules), answer queries, output reports, and ship metadata to and from relational database system catalogs. **Planning and analysis tools** study the impact of schema changes and of refresh performance when changing refresh rates or time windows.

4.3.3 Data Warehouse Usage for Information Processing

Data warehouses and data marts are used in a wide range of applications. Business executives use the data in data warehouses and data marts to perform data analysis and make strategic decisions. In many firms, data warehouses are used as an integral part of a *plan-execute-assess* "closed-loop" feedback system for enterprise management. Data warehouses are used extensively in banking and financial services, consumer goods and retail distribution sectors, and controlled manufacturing such as demand-based production.

Typically, the longer a data warehouse has been in use, the more it will have evolved. This evolution takes place throughout a number of phases. Initially, the data warehouse is mainly used for generating reports and answering predefined queries. Progressively, it is used to analyze summarized and detailed data, where the results are presented in the form of reports and charts. Later, the data warehouse is used for strategic purposes, performing multidimensional analysis and sophisticated slice-and-dice operations. Finally, the data warehouse may be employed for knowledge discovery and strategic decision making using data mining tools. In this context, the tools for data warehousing can be categorized into *access and retrieval tools*, *database reporting tools*, *data analysis tools*, and *data mining tools*.

17. What are the differences between the three main types of data warehouse usage: information processing, analytical processing, and data mining? Discuss the OLAP
mining (OLAM).

Business users need to have the means to know what exists in the data warehouse (through metadata), how to access the contents of the data warehouse, how to examine the contents using analysis tools, and how to present the results of such analysis.

There are three kinds of data warehouse applications: *information processing*, *analytical processing*, and *data mining*.

- **Information processing** supports querying, basic statistical analysis, and reporting using crosstabs, tables, charts, or graphs. A current trend in data warehouse information processing is to construct low-cost web-based accessing tools that are then integrated with web browsers.
- **Analytical processing** supports basic OLAP operations, including slice-and-dice, drill-down, roll-up, and pivoting. It generally operates on historic data in both summarized and detailed forms. The major strength of online analytical processing over information processing is the multidimensional data analysis of data warehouse data.

- **Data mining** supports knowledge discovery by finding hidden patterns and associations, constructing analytical models, performing classification and prediction, and presenting the mining results using visualization tools.

"*How does data mining relate to information processing and online analytical processing?*" Information processing, based on queries, can find useful information. However, answers to such queries reflect the information directly stored in databases or computable by aggregate functions. They do not reflect sophisticated patterns or regularities buried in the database. Therefore, information processing is not data mining.

Online analytical processing comes a step closer to data mining because it can derive information summarized at multiple granularities from user-specified subsets of a data warehouse. Such descriptions are equivalent to the class/concept descriptions discussed in Chapter 1. Because data mining systems can also mine generalized class/concept descriptions, this raises some interesting questions: "*Do OLAP systems perform data mining? Are OLAP systems actually data mining systems?*"

The functionalities of OLAP and data mining can be viewed as disjoint: OLAP is a data summarization/aggregation *tool* that helps simplify data analysis, while data mining allows the *automated discovery* of implicit patterns and interesting knowledge hidden in large amounts of data. OLAP tools are targeted toward simplifying and supporting interactive data analysis, whereas the goal of data mining tools is to automate as much of the process as possible, while still allowing users to guide the process. In this sense, data mining goes one step beyond traditional online analytical processing.

An alternative and broader view of data mining may be adopted in which data mining covers both data description and data modeling. Because OLAP systems can present general descriptions of data from data warehouses, OLAP functions are essentially for user-directed data summarization and comparison (by drilling, pivoting, slicing, dicing, and other operations). These are, though limited, data mining functionalities. Yet according to this view, data mining covers a much broader spectrum than simple OLAP operations, because it performs not only data summarization and comparison but also association, classification, prediction, clustering, time-series analysis, and other data analysis tasks.

Data mining is not confined to the analysis of data stored in data warehouses. It may analyze data existing at more detailed granularities than the summarized data provided in a data warehouse. It may also analyze transactional, spatial, textual, and multimedia data that are difficult to model with current multidimensional database technology. In this context, data mining covers a broader spectrum than OLAP with respect to data mining functionality and the complexity of the data handled.

Because data mining involves more automated and deeper analysis than OLAP, it is expected to have broader applications. Data mining can help business managers find and reach more suitable customers, as well as gain critical business insights that may help drive market share and raise profits. In addition, data mining can help managers understand customer group characteristics and develop optimal pricing strategies accordingly. It can correct item bundling based not on intuition but on actual item groups derived from customer purchase patterns, reduce promotional spending, and at the same time increase the overall net effectiveness of promotions.

4.3.4 From Online Analytical Processing to Multidimensional Data Mining

The data mining field has conducted substantial research regarding mining on various data types, including relational data, data from data warehouses, transaction data, time-series data, spatial data, text data, and flat files. **Multidimensional data mining** (also known as *exploratory multidimensional data mining*, **online analytical mining**, or **OLAM**) integrates OLAP with data mining to uncover knowledge in multidimensional databases. Among the many different paradigms and architectures of data mining systems, multidimensional data mining is particularly important for the following reasons:

- **High quality of data in data warehouses:** Most data mining tools need to work on integrated, consistent, and cleaned data, which requires costly data cleaning, data integration, and data transformation as preprocessing steps. A data warehouse constructed by such preprocessing serves as a valuable source of high-quality data for OLAP as well as for data mining. Notice that data mining may serve as a valuable tool for data cleaning and data integration as well.
- **Available information processing infrastructure surrounding data warehouses:** Comprehensive information processing and data analysis infrastructures have been or will be systematically constructed surrounding data warehouses, which include accessing, integration, consolidation, and transformation of multiple heterogeneous databases, ODBC/OLEDB connections, Web accessing and service facilities, and reporting and OLAP analysis tools. It is prudent to make the best use of the available infrastructures rather than constructing everything from scratch.
- **OLAP-based exploration of multidimensional data:** Effective data mining needs exploratory data analysis. A user will often want to traverse through a database, select portions of relevant data, analyze them at different granularities, and present knowledge/results in different forms. Multidimensional data mining provides facilities for mining on different subsets of data and at varying levels of abstraction—by drilling, pivoting, filtering, dicing, and slicing on a data cube and/or intermediate data mining results. This, together with data/knowledge visualization tools, greatly enhances the power and flexibility of data mining.
- **Online selection of data mining functions:** Users may not always know the specific kinds of knowledge they want to mine. By integrating OLAP with various data mining functions, multidimensional data mining provides users with the flexibility to select desired data mining functions and swap data mining tasks dynamically.

Chapter 5 describes data warehouses on a finer level by exploring implementation issues such as data cube computation, OLAP query answering strategies, and multidimensional data mining. The chapters following it are devoted to the study of data mining techniques. As we have seen, the introduction to data warehousing and OLAP technology presented in this chapter is essential to our study of data mining. This is because data warehousing provides users with large amounts of clean, organized,

and summarized data, which greatly facilitates data mining. For example, rather than storing the details of each sales transaction, a data warehouse may store a summary of the transactions per item type for each branch or, summarized to a higher level, for each country. The capability of OLAP to provide multiple and dynamic views of summarized data in a data warehouse sets a solid foundation for successful data mining.

Moreover, we also believe that data mining should be a human-centered process. Rather than asking a data mining system to generate patterns and knowledge automatically, a user will often need to interact with the system to perform exploratory data analysis. OLAP sets a good example for interactive data analysis and provides the necessary preparations for exploratory data mining. Consider the discovery of association patterns, for example. Instead of mining associations at a primitive (i.e., low) data level among transactions, users should be allowed to specify roll-up operations along any dimension.

For example, a user may want to roll up on the *item* dimension to go from viewing the data for particular TV sets that were purchased to viewing the brands of these TVs (e.g., SONY or Toshiba). Users may also navigate from the transaction level to the customer or customer-type level in the search for interesting associations. Such an OLAP data mining style is characteristic of multidimensional data mining. In our study of the principles of data mining in this book, we place particular emphasis on multidimensional data mining, that is, on the *integration of data mining and OLAP technology*.

4.4 Data Warehouse Implementation

Data warehouses contain huge volumes of data. OLAP servers demand that decision support queries be answered in the order of seconds. Therefore, it is crucial for data warehouse systems to support highly efficient cube computation techniques, access methods, and query processing techniques. In this section, we present an overview of methods for the efficient implementation of data warehouse systems. Section 4.4.1 explores how to compute data cubes efficiently. Section 4.4.2 shows how OLAP data can be indexed, using either bitmap or join indices. Next, we study how OLAP queries are processed (Section 4.4.3). Finally, Section 4.4.4 presents various types of warehouse servers for OLAP processing.

4.4.1 Efficient Data Cube Computation: An Overview

At the core of multidimensional data analysis is the efficient computation of aggregations across many sets of dimensions. In SQL terms, these aggregations are referred to as group-by's. Each group-by can be represented by a *cuboid*, where the set of group-by's forms a lattice of cuboids defining a data cube. In this subsection, we explore issues relating to the efficient computation of data cubes.

The compute cube Operator and the Curse of Dimensionality

One approach to cube computation extends SQL so as to include a compute cube operator. The compute cube operator computes aggregates over all subsets of the dimensions specified in the operation. This can require excessive storage space, especially for large numbers of dimensions. We start with an intuitive look at what is involved in the efficient computation of data cubes.

Example 4.6 **A data cube is a lattice of cuboids.** Suppose that you want to create a data cube for *AllElectronics* sales that contains the following: *city, item, year*, and *sales_in_dollars*. You want to be able to analyze the data, with queries such as the following:

- "*Compute the sum of sales, grouping by city and item.*"
- "*Compute the sum of sales, grouping by city.*"
- "*Compute the sum of sales, grouping by item.*"

What is the total number of cuboids, or group-by's, that can be computed for this data cube? Taking the three attributes, *city, item*, and *year*, as the dimensions for the data cube, and *sales_in_dollars* as the measure, the total number of cuboids, or group-by's, that can be computed for this data cube is $2^3 = 8$. The possible group-by's are the following: {(*city, item, year*), (*city, item*), (*city, year*), (*item, year*), (*city*), (*item*), (*year*), ()}, where () means that the group-by is empty (i.e., the dimensions are not grouped). These group-by's form a lattice of cuboids for the data cube, as shown in Figure 4.14.

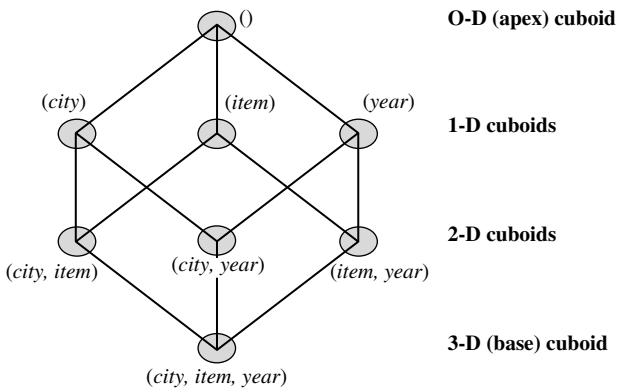


Figure 4.14 Lattice of cuboids, making up a 3-D data cube. Each cuboid represents a different group-by. The base cuboid contains *city, item*, and *year* dimensions.

The **base cuboid** contains all three dimensions, *city, item*, and *year*. It can return the total sales for any combination of the three dimensions. The **apex cuboid**, or 0-D cuboid, refers to the case where the group-by is empty. It contains the total sum of all sales. The base cuboid is the least generalized (most specific) of the cuboids. The apex cuboid is the most generalized (least specific) of the cuboids, and is often denoted as all. If we start at the apex cuboid and explore downward in the lattice, this is equivalent to drilling down within the data cube. If we start at the base cuboid and explore upward, this is akin to rolling up. ■

An SQL query containing no group-by (e.g., "*compute the sum of total sales*") is a *zero-dimensional operation*. An SQL query containing one group-by (e.g., "*compute the sum of sales, group-by city*") is a *one-dimensional operation*. A cube operator on *n* dimensions is equivalent to a collection of group-by statements, one for each subset of the *n* dimensions. Therefore, the cube operator is the *n*-dimensional generalization of the group-by operator.

Similar to the SQL syntax, the data cube in Example 4.1 could be defined as

```
define cube sales_cube [city, item, year]: sum(sales_in_dollars)
```

For a cube with *n* dimensions, there are a total of 2^n cuboids, including the base cuboid. A statement such as

```
compute cube sales_cube
```

would explicitly instruct the system to compute the sales aggregate cuboids for all eight subsets of the set {*city, item, year*}, including the empty subset. A cube computation operator was first proposed and studied by Gray et al. [GCB+97].

Online analytical processing may need to access different cuboids for different queries. Therefore, it may seem like a good idea to compute in advance all or at least some of the cuboids in a data cube. Precomputation leads to fast response time and avoids some redundant computation. Most, if not all, OLAP products resort to some degree of precomputation of multidimensional aggregates.

A major challenge related to this precomputation, however, is that the required storage space may explode if all the cuboids in a data cube are precomputed, especially when the cube has many dimensions. The storage requirements are even more excessive when many of the dimensions have associated concept hierarchies, each with multiple levels. This problem is referred to as the **curse of dimensionality**. The extent of the curse of dimensionality is illustrated here.

"*How many cuboids are there in an* n*-dimensional data cube?*" If there were no hierarchies associated with each dimension, then the total number of cuboids for an *n*-dimensional data cube, as we have seen, is 2^n. However, in practice, many dimensions do have hierarchies. For example, *time* is usually explored not at only one conceptual level (e.g., *year*), but rather at multiple conceptual levels such as in the hierarchy "*day* < *month* < *quarter* < *year*." For an *n*-dimensional data cube, the total number of cuboids

that can be generated (including the cuboids generated by climbing up the hierarchies along each dimension) is

$$Total\ number\ of\ cuboids = \prod_{i=1}^{n}(L_i + 1), \tag{4.1}$$

where L_i is the number of levels associated with dimension i. One is added to L_i in Eq. (4.1) to include the *virtual* top level, **all**. (Note that generalizing to **all** is equivalent to the removal of the dimension.)

This formula is based on the fact that, at most, one abstraction level in each dimension will appear in a cuboid. For example, the time dimension as specified before has four conceptual levels, or five if we include the virtual level **all**. If the cube has 10 dimensions and each dimension has five levels (including **all**), the total number of cuboids that can be generated is $5^{10} \approx 9.8 \times 10^6$. The size of each cuboid also depends on the *cardinality* (i.e., number of distinct values) of each dimension. For example, if the *AllElectronics* branch in each city sold every item, there would be $|city| \times |item|$ tuples in the *city_item* group-by alone. As the number of dimensions, number of conceptual hierarchies, or cardinality increases, the storage space required for many of the group-by's will grossly exceed the (fixed) size of the input relation.

By now, you probably realize that it is unrealistic to precompute and materialize all of the cuboids that can possibly be generated for a data cube (i.e., from a base cuboid). If there are many cuboids, and these cuboids are large in size, a more reasonable option is *partial materialization*; that is, to materialize only *some* of the possible cuboids that can be generated.

13. What is Materialization? Explain materialized views (using cuboids) in DW.

Partial Materialization: Selected Computation of Cuboids

There are three choices for data cube materialization given a base cuboid:

1. **No materialization**: Do not precompute any of the "nonbase" cuboids. This leads to computing expensive multidimensional aggregates on-the-fly, which can be extremely slow.
2. **Full materialization**: Precompute all of the cuboids. The resulting lattice of computed cuboids is referred to as the *full cube*. This choice typically requires huge amounts of memory space in order to store all of the precomputed cuboids.
3. **Partial materialization**: Selectively compute a proper subset of the whole set of possible cuboids. Alternatively, we may compute a subset of the cube, which contains only those cells that satisfy some user-specified criterion, such as where the tuple count of each cell is above some threshold. We will use the term *subcube* to refer to the latter case, where only some of the cells may be precomputed for various cuboids. Partial materialization represents an interesting trade-off between storage space and response time.

The partial materialization of cuboids or subcubes should consider three factors: (1) identify the subset of cuboids or subcubes to materialize; (2) exploit the materialized cuboids or subcubes during query processing; and (3) efficiently update the materialized cuboids or subcubes during load and refresh.

The selection of the subset of cuboids or subcubes to materialize should take into account the queries in the workload, their frequencies, and their accessing costs. In addition, it should consider workload characteristics, the cost for incremental updates, and the total storage requirements. The selection must also consider the broad context of physical database design such as the generation and selection of indices. Several OLAP products have adopted heuristic approaches for cuboid and subcube selection. A popular approach is to materialize the cuboids set on which other frequently referenced cuboids are based. Alternatively, we can compute an **iceberg cube**, which is a data cube that stores only those cube cells with an aggregate value (e.g., count) that is above some minimum support threshold.

Another common strategy is to materialize a *shell cube*. This involves precomputing the cuboids for only a small number of dimensions (e.g., three to five) of a data cube. Queries on additional combinations of the dimensions can be computed on-the-fly. Because our aim in this chapter is to provide a solid introduction and overview of data warehousing for data mining, we defer our detailed discussion of cuboid selection and computation to Chapter 5, which studies various data cube computation methods in greater depth.

Once the selected cuboids have been materialized, it is important to take advantage of them during query processing. This involves several issues, such as how to determine the relevant cuboid(s) from among the candidate materialized cuboids, how to use available index structures on the materialized cuboids, and how to transform the OLAP operations onto the selected cuboid(s). These issues are discussed in Section 4.4.3 as well as in Chapter 5.

Finally, during load and refresh, the materialized cuboids should be updated efficiently. Parallelism and incremental update techniques for this operation should be explored.

14. Discuss Indexing in OLAP Data for data accessing in DW.

4.4.2 Indexing OLAP Data: Bitmap Index and Join Index

To facilitate efficient data accessing, most data warehouse systems support index structures and materialized views (using cuboids). General methods to select cuboids for materialization were discussed in Section 4.4.1. In this subsection, we examine how to index OLAP data by *bitmap indexing* and *join indexing*.

The **bitmap indexing** method is popular in OLAP products because it allows quick searching in data cubes. The bitmap index is an alternative representation of the *record_ID (RID)* list. In the bitmap index for a given attribute, there is a distinct bit vector, ***Bv***, for each value v in the attribute's domain. If a given attribute's domain consists of n values, then n bits are needed for each entry in the bitmap index (i.e., there are n bit vectors). If the attribute has the value v for a given row in the data table, then the bit representing that value is set to 1 in the corresponding row of the bitmap index. All other bits for that row are set to 0.

Example 4.7 **Bitmap indexing.** In the *AllElectronics* data warehouse, suppose the dimension *item* at the top level has four values (representing item types): "*home entertainment*," "*computer*," "*phone*," and "*security*." Each value (e.g., "*computer*") is represented by a bit vector in the *item* bitmap index table. Suppose that the cube is stored as a relation table with 100,000 rows. Because the domain of *item* consists of four values, the bitmap index table requires four bit vectors (or lists), each with 100,000 bits. Figure 4.15 shows a base (data) table containing the dimensions *item* and *city*, and its mapping to bitmap index tables for each of the dimensions. ■

Base table

RID	*item*	*city*
R1	H	V
R2	C	V
R3	P	V
R4	S	V
R5	H	T
R6	C	T
R7	P	T
R8	S	T

item bitmap index table

RID	H	C	P	S
R1	1	0	0	0
R2	0	1	0	0
R3	0	0	1	0
R4	0	0	0	1
R5	1	0	0	0
R6	0	1	0	0
R7	0	0	1	0
R8	0	0	0	1

city bitmap index table

RID	V	T
R1	1	0
R2	1	0
R3	1	0
R4	1	0
R5	0	1
R6	0	1
R7	0	1
R8	0	1

Note: H for "home entertainment," C for "computer," P for "phone," S for "security," V for "Vancouver," T for "Toronto."

Figure 4.15 Indexing OLAP data using bitmap indices.

Bitmap indexing is advantageous compared to hash and tree indices. It is especially useful for low-cardinality domains because comparison, join, and aggregation operations are then reduced to bit arithmetic, which substantially reduces the processing time. Bitmap indexing leads to significant reductions in space and input/output (I/O) since a string of characters can be represented by a single bit. For higher-cardinality domains, the method can be adapted using compression techniques.

The **join indexing** method gained popularity from its use in relational database query processing. Traditional indexing maps the value in a given column to a list of rows having that value. In contrast, join indexing registers the joinable rows of two relations from a relational database. For example, if two relations $R(RID, A)$ and $S(B, SID)$ join on the attributes A and B, then the join index record contains the pair (RID, SID), where RID and SID are record identifiers from the R and S relations, respectively. Hence, the join index records can identify joinable tuples without performing costly join operations. Join indexing is especially useful for maintaining the relationship between a foreign key[2] and its matching primary keys, from the joinable relation.

The star schema model of data warehouses makes join indexing attractive for cross-table search, because the linkage between a fact table and its corresponding dimension tables comprises the fact table's foreign key and the dimension table's primary key. Join

[2] A set of attributes in a relation schema that forms a primary key for another relation schema is called a **foreign key**.

indexing maintains relationships between attribute values of a dimension (e.g., within a dimension table) and the corresponding rows in the fact table. Join indices may span multiple dimensions to form **composite join indices**. We can use join indices to identify subcubes that are of interest.

Example 4.8 **Join indexing.** In Example 3.4, we defined a star schema for *AllElectronics* of the form "*sales_star* [*time, item, branch, location*]: *dollars_sold* = sum *(sales_in_dollars)*." An example of a join index relationship between the *sales* fact table and the *location* and *item* dimension tables is shown in Figure 4.16. For example, the "Main Street" value in the *location* dimension table joins with tuples T57, T238, and T884 of the *sales* fact table. Similarly, the "Sony-TV" value in the *item* dimension table joins with tuples T57 and T459 of the *sales* fact table. The corresponding join index tables are shown in Figure 4.17.

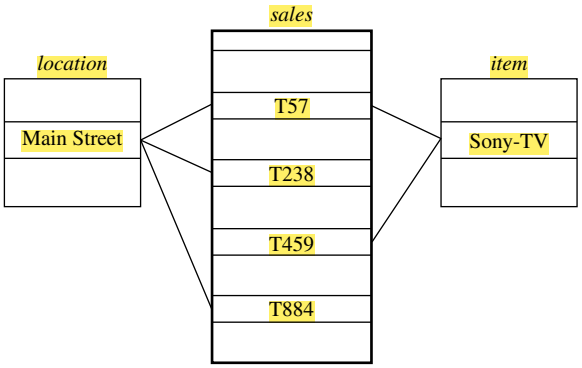


Figure 4.16 Linkages between a *sales* fact table and *location* and *item* dimension tables.

Join index table for *location/sales*

location	*sales_key*
. . .	. . .
Main Street	T57
Main Street	T238
Main Street	T884
. . .	. . .

Join index table for *item/sales*

item	*sales_key*
. . .	. . .
Sony-TV	T57
Sony-TV	T459
. . .	. . .

Join index table linking *location* and *item* to *sales*

location	*item*	*sales_key*
. . .	. . .	. . .
Main Street	Sony-TV	T57
. . .	. . .	. . .

Figure 4.17 Join index tables based on the linkages between the *sales* fact table and the *location* and *item* dimension tables shown in Figure 4.16.

Suppose that there are 360 time values, 100 items, 50 branches, 30 locations, and 10 million sales tuples in the *sales_star* data cube. If the *sales* fact table has recorded sales for only 30 items, the remaining 70 items will obviously not participate in joins. If join indices are not used, additional I/Os have to be performed to bring the joining portions of the fact table and the dimension tables together. ■

To further speed up query processing, the join indexing and the bitmap indexing methods can be integrated to form **bitmapped join indices**.

4.4.3 Efficient Processing of OLAP Queries

The purpose of materializing cuboids and constructing OLAP index structures is to speed up query processing in data cubes. Given materialized views, query processing should proceed as follows:

1. **Determine which operations should be performed on the available cuboids:** This involves transforming any selection, projection, roll-up (group-by), and drill-down operations specified in the query into corresponding SQL and/or OLAP operations. For example, slicing and dicing a data cube may correspond to selection and/or projection operations on a materialized cuboid.
2. **Determine to which materialized cuboid(s) the relevant operations should be applied:** This involves identifying all of the materialized cuboids that may potentially be used to answer the query, pruning the set using knowledge of "dominance" relationships among the cuboids, estimating the costs of using the remaining materialized cuboids, and selecting the cuboid with the least cost.

Example 4.9 **OLAP query processing.** Suppose that we define a data cube for *AllElectronics* of the form "*sales_cube* [*time, item, location*]: sum*(sales_in_dollars)*." The dimension hierarchies used are "*day* < *month* < *quarter* < *year*" for *time*; "*item_name* < *brand* < *type*" for *item*; and "*street* < *city* < *province_or_state* < *country*" for *location*.

Suppose that the query to be processed is on {*brand, province_or_state*}, with the selection constant "*year* = *2010*." Also, suppose that there are four materialized cuboids available, as follows:

- cuboid 1: {*year, item_name, city*}
- cuboid 2: {*year, brand, country*}
- cuboid 3: {*year, brand, province_or_state*}
- cuboid 4: {*item_name, province_or_state*}, where *year* = 2010

"*Which of these four cuboids should be selected to process the query?*" Finer-granularity data cannot be generated from coarser-granularity data. Therefore, cuboid 2 cannot be used because *country* is a more general concept than *province_or_state*. Cuboids 1, 3, and 4 can be used to process the query because (1) they have the same set or a superset of the

dimensions in the query, (2) the selection clause in the query can imply the selection in the cuboid, and (3) the abstraction levels for the *item* and *location* dimensions in these cuboids are at a finer level than *brand* and *province_or_state*, respectively.

"*How would the costs of each cuboid compare if used to process the query?*" It is likely that using cuboid 1 would cost the most because both *item_name* and *city* are at a lower level than the *brand* and *province_or_state* concepts specified in the query. If there are not many *year* values associated with *items* in the cube, but there are several *item_names* for each *brand*, then cuboid 3 will be smaller than cuboid 4, and thus cuboid 3 should be chosen to process the query. However, if efficient indices are available for cuboid 4, then cuboid 4 may be a better choice. Therefore, some cost-based estimation is required to decide which set of cuboids should be selected for query processing. ■

15. Discuss different OLAP Server Architectures.

4.4.4 OLAP Server Architectures: ROLAP versus MOLAP versus HOLAP

Logically, OLAP servers present business users with multidimensional data from data warehouses or data marts, without concerns regarding how or where the data are stored. However, the physical architecture and implementation of OLAP servers must consider data storage issues. Implementations of a warehouse server for OLAP processing include the following:

Relational OLAP (ROLAP) servers: These are the intermediate servers that stand in between a relational back-end server and client front-end tools. They use a *relational* or *extended-relational DBMS* to store and manage warehouse data, and OLAP middleware to support missing pieces. ROLAP servers include optimization for each DBMS back end, implementation of aggregation navigation logic, and additional tools and services. ROLAP technology tends to have greater scalability than MOLAP technology. The DSS server of Microstrategy, for example, adopts the ROLAP approach.

Multidimensional OLAP (MOLAP) servers: These servers support multidimensional data views through *array-based multidimensional storage engines*. They map multidimensional views directly to data cube array structures. The advantage of using a data cube is that it allows fast indexing to precomputed summarized data. Notice that with multidimensional data stores, the storage utilization may be low if the data set is sparse. In such cases, sparse matrix compression techniques should be explored (Chapter 5).

Many MOLAP servers adopt a two-level storage representation to handle dense and sparse data sets: Denser subcubes are identified and stored as array structures, whereas sparse subcubes employ compression technology for efficient storage utilization.

Hybrid OLAP (HOLAP) servers: The hybrid OLAP approach combines ROLAP and MOLAP technology, benefiting from the greater scalability of ROLAP and the faster computation of MOLAP. For example, a HOLAP server may allow large volumes

of detailed data to be stored in a relational database, while aggregations are kept in a separate MOLAP store. The Microsoft SQL Server 2000 supports a hybrid OLAP server.

Specialized SQL servers: To meet the growing demand of OLAP processing in relational databases, some database system vendors implement specialized SQL servers that provide advanced query language and query processing support for SQL queries over star and snowflake schemas in a read-only environment.

"*How are data actually stored in ROLAP and MOLAP architectures?*" Let's first look at ROLAP. As its name implies, ROLAP uses relational tables to store data for online analytical processing. Recall that the fact table associated with a base cuboid is referred to as a *base fact table*. The base fact table stores data at the abstraction level indicated by the join keys in the schema for the given data cube. Aggregated data can also be stored in fact tables, referred to as **summary fact tables**. Some summary fact tables store both base fact table data and aggregated data (see Example 3.10). Alternatively, separate summary fact tables can be used for each abstraction level to store only aggregated data.

Example 4.10 **A ROLAP data store.** Table 4.4 shows a summary fact table that contains both base fact data and aggregated data. The schema is "⟨*record_identifier (RID), item, . . . , day, month, quarter, year, dollars_sold*⟩," where *day, month, quarter*, and *year* define the sales date, and *dollars_sold* is the sales amount. Consider the tuples with an *RID* of 1001 and 1002, respectively. The data of these tuples are at the base fact level, where the sales dates are October 15, 2010, and October 23, 2010, respectively. Consider the tuple with an *RID* of 5001. This tuple is at a more general level of abstraction than the tuples 1001 and 1002. The *day* value has been generalized to all, so that the corresponding *time* value is October 2010. That is, the *dollars_sold* amount shown is an aggregation representing the entire month of October 2010, rather than just October 15 or 23, 2010. The special value all is used to represent subtotals in summarized data. ■

MOLAP uses multidimensional array structures to store data for online analytical processing. This structure is discussed in greater detail in Chapter 5.

Most data warehouse systems adopt a client-server architecture. A relational data store always resides at the data warehouse/data mart server site. A multidimensional data store can reside at either the database server site or the client site.

Table 4.4 Single Table for Base and Summary Facts

RID	*item*	. . .	*day*	*month*	*quarter*	*year*	*dollars_sold*
1001	TV	. . .	15	10	Q4	2010	250.60
1002	TV	. . .	23	10	Q4	2010	175.00
. . .	. . .	. . .	. . .	. . .	. . .	. . .	. . .
5001	TV	. . .	all	10	Q4	2010	45,786.08
. . .	. . .	. . .	. . .	. . .	. . .	. . .	. . .

16. Explain multidimensional Data Generalization by Attribute-Oriented Induction.

4.5 Data Generalization by Attribute-Oriented Induction

Conceptually, the data cube can be viewed as a kind of multidimensional data generalization. In general, *data generalization* summarizes data by replacing relatively low-level values (e.g., numeric values for an attribute *age*) with higher-level concepts (e.g., *young*, *middle-aged*, and *senior*), or by reducing the number of dimensions to summarize data in concept space involving fewer dimensions (e.g., removing *birth_date* and *telephone number* when summarizing the behavior of a group of students). Given the large amount of data stored in databases, it is useful to be able to describe concepts in concise and succinct terms at generalized (rather than low) levels of abstraction. Allowing data sets to be generalized at multiple levels of abstraction facilitates users in examining the general behavior of the data. Given the *AllElectronics* database, for example, instead of examining individual customer transactions, sales managers may prefer to view the data generalized to higher levels, such as summarized by customer groups according to geographic regions, frequency of purchases per group, and customer income.

This leads us to the notion of *concept description*, which is a form of data generalization. A concept typically refers to a data collection such as *frequent_buyers*, *graduate_students*, and so on. As a data mining task, concept description is not a simple enumeration of the data. Instead, **concept description** generates descriptions for data *characterization* and *comparison*. It is sometimes called **class description** when the concept to be described refers to a class of objects. **Characterization** provides a concise and succinct summarization of the given data collection, while concept or class **comparison** (also known as **discrimination**) provides descriptions comparing two or more data collections.

Up to this point, we have studied data cube (or OLAP) approaches to concept description using multidimensional, multilevel data generalization in data warehouses. "*Is data cube technology sufficient to accomplish all kinds of concept description tasks for large data sets?*" Consider the following cases.

- **Complex data types and aggregation**: Data warehouses and OLAP tools are based on a multidimensional data model that views data in the form of a data cube, consisting of dimensions (or attributes) and measures (aggregate functions). However, many current OLAP systems confine dimensions to non-numeric data and measures to numeric data. In reality, the database can include attributes of various data types, including numeric, non-numeric, spatial, text, or image, which ideally should be included in the concept description.

  Furthermore, the aggregation of attributes in a database may include sophisticated data types such as the collection of non-numeric data, the merging of spatial regions, the composition of images, the integration of texts, and the grouping of object pointers. Therefore, OLAP, with its restrictions on the possible dimension and measure types, represents a simplified model for data analysis. Concept description should handle complex data types of the attributes and their aggregations, as necessary.

- **User control versus automation**: Online analytical processing in data warehouses is a user-controlled process. The selection of dimensions and the application of OLAP operations (e.g., drill-down, roll-up, slicing, and dicing) are primarily directed and controlled by users. Although the control in most OLAP systems is quite user-friendly, users do require a good understanding of the role of each dimension. Furthermore, in order to find a satisfactory description of the data, users may need to specify a long sequence of OLAP operations. It is often desirable to have a more automated process that helps users determine which dimensions (or attributes) should be included in the analysis, and the degree to which the given data set should be generalized in order to produce an interesting summarization of the data.

This section presents an alternative method for concept description, called *attribute-oriented induction*, which works for complex data types and relies on a data-driven generalization process.

4.5.1 Attribute-Oriented Induction for Data Characterization

The **attribute-oriented induction** (**AOI**) approach to concept description was first proposed in 1989, a few years before the introduction of the data cube approach. The data cube approach is essentially based on *materialized views* of the data, which typically have been precomputed in a data warehouse. In general, it performs offline aggregation before an OLAP or data mining query is submitted for processing. On the other hand, the attribute-oriented induction approach is basically a *query-oriented*, generalization-based, online data analysis technique. Note that there is no inherent barrier distinguishing the two approaches based on online aggregation versus offline precomputation. Some aggregations in the data cube can be computed online, while offline precomputation of multidimensional space can speed up attribute-oriented induction as well.

The general idea of attribute-oriented induction is to first collect the task-relevant data using a database query and then perform generalization based on the examination of the number of each attribute's distinct values in the relevant data set. The generalization is performed by either *attribute removal* or *attribute generalization*. Aggregation is performed by merging identical generalized tuples and accumulating their respective counts. This reduces the size of the generalized data set. The resulting generalized relation can be mapped into different forms (e.g., charts or rules) for presentation to the user.

The following illustrates the process of attribute-oriented induction. We first discuss its use for characterization. The method is extended for the mining of class comparisons in Section 4.5.3.

Example 4.11 **A data mining query for characterization.** Suppose that a user wants to describe the general characteristics of graduate students in the *Big University* database, given the attributes *name, gender, major, birth_place, birth_date, residence, phone# (telephone*

number), and *gpa (grade_point_average)*. A data mining query for this characterization can be expressed in the data mining query language, DMQL, as follows:

```
use Big_University_DB
mine characteristics as "Science_Students"
in relevance to name, gender, major, birth_place, birth_date, residence,
     phone#, gpa
from student
where status in "graduate"
```

We will see how this example of a typical data mining query can apply attribute-oriented induction to the mining of characteristic descriptions.

First, **data focusing** should be performed *before* attribute-oriented induction. This step corresponds to the specification of the task-relevant data (i.e., data for analysis). The data are collected based on the information provided in the data mining query. Because a data mining query is usually relevant to only a portion of the database, selecting the relevant data set not only makes mining more efficient, but also derives more meaningful results than mining the entire database.

Specifying the set of relevant attributes (i.e., attributes for mining, as indicated in DMQL with the in relevance to clause) may be difficult for the user. A user may select only a few attributes that he or she feels are important, while missing others that could also play a role in the description. For example, suppose that the dimension *birth_place* is defined by the attributes *city, province_or_state*, and *country*. Of these attributes, let's say that the user has only thought to specify *city*. In order to allow generalization on the *birth_place* dimension, the other attributes defining this dimension should also be included. In other words, having the system automatically include *province_or_state* and *country* as relevant attributes allows *city* to be generalized to these higher conceptual levels during the induction process.

At the other extreme, suppose that the user may have introduced too many attributes by specifying all of the possible attributes with the clause in relevance to $*$. In this case, all of the attributes in the relation specified by the from clause would be included in the analysis. Many of these attributes are unlikely to contribute to an interesting description. A correlation-based analysis method (Section 3.3.2) can be used to perform attribute *relevance analysis* and filter out statistically irrelevant or weakly relevant attributes from the descriptive mining process. Other approaches such as attribute subset selection, are also described in Chapter 3.

Table 4.5 Initial Working Relation: A Collection of Task-Relevant Data

name	*gender*	*major*	*birth_place*	*birth_date*	*residence*	*phone#*	*gpa*
Jim Woodman	M	CS	Vancouver, BC, Canada	12-8-76	3511 Main St., Richmond	687-4598	3.67
Scott Lachance	M	CS	Montreal, Que, Canada	7-28-75	345 1st Ave., Richmond	253-9106	3.70
Laura Lee	F	Physics	Seattle, WA, USA	8-25-70	125 Austin Ave., Burnaby	420-5232	3.83
...	...	...	...	...	...	...	...

"*What does the* 'where *status* in "graduate"' *clause mean?*" The where clause implies that a concept hierarchy exists for the attribute *status*. Such a concept hierarchy organizes primitive-level data values for *status* (e.g., "M.Sc.," "M.A.," "M.B.A.," "Ph.D.," "B.Sc.," and "B.A.") into higher conceptual levels (e.g., "graduate" and "undergraduate"). This use of concept hierarchies does not appear in traditional relational query languages, yet is likely to become a common feature in data mining query languages.

The data mining query presented in Example 4.11 is transformed into the following relational query for the collection of the task-relevant data set:

```
use Big_University_DB
select name, gender, major, birth_place, birth_date, residence, phone#, gpa
from student
where status in {"M.Sc.," "M.A.," "M.B.A.," "Ph.D."}
```

The transformed query is executed against the relational database, *Big_University_DB*, and returns the data shown earlier in Table 4.5. This table is called the (task-relevant) **initial working relation**. It is the data on which induction will be performed. Note that each tuple is, in fact, a conjunction of attribute–value pairs. Hence, we can think of a tuple within a relation as a rule of conjuncts, and of induction on the relation as the generalization of these rules. ■

"*Now that the data are ready for attribute-oriented induction, how is attribute-oriented induction performed?*" The essential operation of attribute-oriented induction is *data generalization*, which can be performed in either of two ways on the initial working relation: *attribute removal* and *attribute generalization*.

Attribute removal is based on the following rule: *If there is a large set of distinct values for an attribute of the initial working relation, but either (case 1) there is no generalization operator on the attribute (e.g., there is no concept hierarchy defined for the attribute), or (case 2) its higher-level concepts are expressed in terms of other attributes, then the attribute should be removed from the working relation.*

Let's examine the reasoning behind this rule. An attribute–value pair represents a conjunct in a generalized tuple, or rule. The removal of a conjunct eliminates a constraint and thus generalizes the rule. If, as in case 1, there is a large set of distinct values for an attribute but there is no generalization operator for it, the attribute should be removed because it cannot be generalized. Preserving it would imply keeping a large number of disjuncts, which contradicts the goal of generating concise rules. On the other hand, consider case 2, where the attribute's higher-level concepts are expressed in terms of other attributes. For example, suppose that the attribute in question is *street*, with higher-level concepts that are represented by the attributes ⟨*city, province_or_state, country*⟩. The removal of *street* is equivalent to the application of a generalization operator. This rule corresponds to the generalization rule known as *dropping condition* in the machine learning literature on *learning from examples*.

Attribute generalization is based on the following rule: *If there is a large set of distinct values for an attribute in the initial working relation, and there exists a set of generalization operators on the attribute, then a generalization operator should be selected and applied*

to the attribute. This rule is based on the following reasoning. Use of a generalization operator to generalize an attribute value within a tuple, or rule, in the working relation will make the rule cover more of the original data tuples, thus generalizing the concept it represents. This corresponds to the generalization rule known as *climbing generalization trees* in *learning from examples*, or *concept tree ascension*.

Both rules–*attribute removal* and *attribute generalization*–claim that if there is a *large* set of distinct values for an attribute, further generalization should be applied. This raises the question: How large is "*a large set of distinct values for an attribute*" considered to be?

Depending on the attributes or application involved, a user may prefer some attributes to remain at a rather low abstraction level while others are generalized to higher levels. The control of how high an attribute should be generalized is typically quite subjective. The control of this process is called **attribute generalization control**. If the attribute is generalized "too high," it may lead to overgeneralization, and the resulting rules may not be very informative.

On the other hand, if the attribute is not generalized to a "sufficiently high level," then undergeneralization may result, where the rules obtained may not be informative either. Thus, a balance should be attained in attribute-oriented generalization. There are many possible ways to control a generalization process. We will describe two common approaches and illustrate how they work.

The first technique, called **attribute generalization threshold control**, either sets one generalization threshold for all of the attributes, or sets one threshold for each attribute. If the number of distinct values in an attribute is greater than the attribute threshold, further attribute removal or attribute generalization should be performed. Data mining systems typically have a default attribute threshold value generally ranging from 2 to 8 and should allow experts and users to modify the threshold values as well. If a user feels that the generalization reaches too high a level for a particular attribute, the threshold can be increased. This corresponds to drilling down along the attribute. Also, to further generalize a relation, the user can reduce an attribute's threshold, which corresponds to rolling up along the attribute.

The second technique, called **generalized relation threshold control**, sets a threshold for the generalized relation. If the number of (distinct) tuples in the generalized relation is greater than the threshold, further generalization should be performed. Otherwise, no further generalization should be performed. Such a threshold may also be preset in the data mining system (usually within a range of 10 to 30), or set by an expert or user, and should be adjustable. For example, if a user feels that the generalized relation is too small, he or she can increase the threshold, which implies drilling down. Otherwise, to further generalize a relation, the threshold can be reduced, which implies rolling up.

These two techniques can be applied in sequence: First apply the attribute threshold control technique to generalize each attribute, and then apply relation threshold control to further reduce the size of the generalized relation. No matter which generalization control technique is applied, the user should be allowed to adjust the generalization thresholds in order to obtain interesting concept descriptions.

In many database-oriented induction processes, users are interested in obtaining quantitative or statistical information about the data at different abstraction levels.

Thus, it is important to accumulate count and other aggregate values in the induction process. Conceptually, this is performed as follows. The aggregate function, count(), is associated with each database tuple. Its value for each tuple in the initial working relation is initialized to 1. Through attribute removal and attribute generalization, tuples within the initial working relation may be generalized, resulting in groups of *identical tuples*. In this case, all of the identical tuples forming a group should be merged into one tuple.

The count of this new, generalized tuple is set to the total number of tuples from the initial working relation that are represented by (i.e., merged into) the new generalized tuple. For example, suppose that by attribute-oriented induction, 52 data tuples from the initial working relation are all generalized to the same tuple, T. That is, the generalization of these 52 tuples resulted in 52 identical instances of tuple T. These 52 identical tuples are merged to form one instance of T, with a count that is set to 52. Other popular aggregate functions that could also be associated with each tuple include sum() and avg(). For a given generalized tuple, sum() contains the sum of the values of a given numeric attribute for the initial working relation tuples making up the generalized tuple. Suppose that tuple T contained sum(*units_sold*) as an aggregate function. The sum value for tuple T would then be set to the total number of units sold for each of the 52 tuples. The aggregate avg() (average) is computed according to the formula avg() = sum()/count().

Example 4.12 **Attribute-oriented induction.** Here we show how attribute-oriented induction is performed on the initial working relation of Table 4.5. For each attribute of the relation, the generalization proceeds as follows:

1. *name:* Since there are a large number of distinct values for *name* and there is no generalization operation defined on it, this attribute is removed.
2. *gender:* Since there are only two distinct values for *gender*, this attribute is retained and no generalization is performed on it.
3. *major:* Suppose that a concept hierarchy has been defined that allows the attribute *major* to be generalized to the values {arts&sciences, engineering, business}. Suppose also that the attribute generalization threshold is set to 5, and that there are more than 20 distinct values for *major* in the initial working relation. By attribute generalization and attribute generalization control, *major* is therefore generalized by climbing the given concept hierarchy.
4. *birth_place:* This attribute has a large number of distinct values; therefore, we would like to generalize it. Suppose that a concept hierarchy exists for *birth_place*, defined as "*city* < *province_or_state* < *country*." If the number of distinct values for *country* in the initial working relation is greater than the attribute generalization threshold, then *birth_place* should be removed, because even though a generalization operator exists for it, the generalization threshold would not be satisfied. If, instead, the number of distinct values for *country* is less than the attribute generalization threshold, then *birth_place* should be generalized to *birth_country*.
5. *birth_date:* Suppose that a hierarchy exists that can generalize *birth_date* to *age* and *age* to *age_range*, and that the number of age ranges (or intervals) is small with

Table 4.6 Generalized Relation Obtained by Attribute-Oriented Induction on Table 4.5's Data

gender	*major*	*birth_country*	*age_range*	*residence_city*	*gpa*	count
M	Science	Canada	20 – 25	Richmond	very_good	16
F	Science	Foreign	25 – 30	Burnaby	excellent	22
...	...	...	...	...	...	...

respect to the attribute generalization threshold. Generalization of *birth_date* should therefore take place.

6. *residence:* Suppose that *residence* is defined by the attributes *number, street, residence_city, residence_province_or_state*, and *residence_country*. The number of distinct values for *number* and *street* will likely be very high, since these concepts are quite low level. The attributes *number* and *street* should therefore be removed so that *residence* is then generalized to *residence_city*, which contains fewer distinct values.
7. *phone#:* As with the *name* attribute, *phone#* contains too many distinct values and should therefore be removed in generalization.
8. *gpa:* Suppose that a concept hierarchy exists for *gpa* that groups values for grade point average into numeric intervals like {3.75–4.0, 3.5–3.75, ...}, which in turn are grouped into descriptive values such as {"excellent", "very_good", ...}. The attribute can therefore be generalized.

The generalization process will result in groups of identical tuples. For example, the first two tuples of Table 4.5 both generalize to the same identical tuple (namely, the first tuple shown in Table 4.6). Such identical tuples are then merged into one, with their counts accumulated. This process leads to the generalized relation shown in Table 4.6.

Based on the vocabulary used in OLAP, we may view count() as a *measure*, and the remaining attributes as *dimensions*. Note that aggregate functions, such as sum(), may be applied to numeric attributes (e.g., *salary* and *sales*). These attributes are referred to as *measure attributes*. ■

4.5.2 Efficient Implementation of Attribute-Oriented Induction

"How is attribute-oriented induction actually implemented?" Section 4.5.1 provided an introduction to attribute-oriented induction. The general procedure is summarized in Figure 4.18. The efficiency of this algorithm is analyzed as follows:

- Step 1 of the algorithm is essentially a relational query to collect the task-relevant data into the **working relation**, W. Its processing efficiency depends on the query processing methods used. Given the successful implementation and commercialization of database systems, this step is expected to have good performance.
- Step 2 collects statistics on the working relation. This requires scanning the relation at most once. The cost for computing the minimum desired level and determining the mapping pairs, (v, v'), for each attribute is dependent on the number of distinct

Algorithm: Attribute-oriented induction. Mining generalized characteristics in a relational database given a user's data mining request.

Input:

- *DB*, a relational database;
- *DMQuery*, a data mining query;
- *a_list*, a list of attributes (containing attributes, a_i);
- *Gen*(a_i), a set of concept hierarchies or generalization operators on attributes, a_i;
- *a_gen_thresh*(a_i), attribute generalization thresholds for each a_i.

Output: *P*, a *Prime_generalized_relation*.

Method:

1. $W \leftarrow$ get_task_relevant_data (*DMQuery, DB*); // Let W, the working relation, hold the task-relevant data.
2. prepare_for_generalization (W); // This is implemented as follows.
   - **(a)** Scan W and collect the distinct values for each attribute, a_i. (*Note:* If W is very large, this may be done by examining a sample of W.)
   - **(b)** For each attribute a_i, determine whether a_i should be removed. If not, compute its minimum desired level L_i based on its given or default attribute threshold, and determine the mapping pairs (v, v'), where v is a distinct value of a_i in W, and v' is its corresponding generalized value at level L_i.
3. $P \leftarrow$ generalization (W),

   The *Prime_generalized_relation*, P, is derived by replacing each value v in W by its corresponding v' in the mapping while accumulating `count` and computing any other aggregate values.

   This step can be implemented efficiently using either of the two following variations:

   - **(a)** For each generalized tuple, insert the tuple into a sorted prime relation P by a binary search: if the tuple is already in P, simply increase its `count` and other aggregate values accordingly; otherwise, insert it into P.
   - **(b)** Since in most cases the number of distinct values at the prime relation level is small, the prime relation can be coded as an m-dimensional array, where m is the number of attributes in P, and each dimension contains the corresponding generalized attribute values. Each array element holds the corresponding `count` and other aggregation values, if any. The insertion of a generalized tuple is performed by measure aggregation in the corresponding array element.

Figure 4.18 Basic algorithm for attribute-oriented induction.

values for each attribute and is smaller than $|W|$, the number of tuples in the working relation. Notice that it may not be necessary to scan the working relation once, since if the working relation is large, a sample of such a relation will be sufficient to get statistics and determine which attributes should be generalized to a certain high level and which attributes should be removed. Moreover, such statistics may also be obtained in the process of extracting and generating a working relation in Step 1.

- Step 3 derives the **prime relation**, P. This is performed by scanning each tuple in the working relation and inserting generalized tuples into P. There are a total of $|W|$ tuples in W and p tuples in P. For each tuple, $\boldsymbol{t}$, in W, we substitute its attribute values based on the derived mapping pairs. This results in a generalized tuple, $\boldsymbol{t'}$. If variation (a) in Figure 4.18 is adopted, each $\boldsymbol{t'}$ takes $O(\log p)$ to find the location for the count increment or tuple insertion. Thus, the total time complexity is $O(|W| \times \log p)$ for all of the generalized tuples. If variation (b) is adopted, each $\boldsymbol{t'}$ takes $O(1)$ to find the tuple for the count increment. Thus, the overall time complexity is $O(N)$ for all of the generalized tuples.

Many data analysis tasks need to examine a good number of dimensions or attributes. This may involve *dynamically* introducing and testing additional attributes rather than just those specified in the mining query. Moreover, a user with little knowledge of the *truly* relevant data set may simply specify "in relevance to $*$" in the mining query, which includes all of the attributes in the analysis. Therefore, an advanced–concept description mining process needs to perform attribute relevance analysis on large sets of attributes to select the most relevant ones. This analysis may employ correlation measures or tests of statistical significance, as described in Chapter 3 on data preprocessing.

Example 4.13 **Presentation of generalization results.** Suppose that attribute-oriented induction was performed on a *sales* relation of the *AllElectronics* database, resulting in the generalized description of Table 4.7 for sales last year. The description is shown in the form of a generalized relation. Table 4.6 is another generalized relation example.

Such generalized relations can also be presented in the form of cross-tabulation forms, various kinds of graphic presentation (e.g., pie charts and bar charts), and quantitative characteristics rules (i.e., showing how different value combinations are distributed in the generalized relation). ■

Table 4.7 Generalized Relation for Last Year's Sales

location	*item*	*sales (in million dollars)*	count *(in thousands)*
Asia	TV	15	300
Europe	TV	12	250
North America	TV	28	450
Asia	computer	120	1000
Europe	computer	150	1200
North America	computer	200	1800

4.5.3 Attribute-Oriented Induction for Class Comparisons

In many applications, users may not be interested in having a single class (or concept) described or characterized, but prefer to mine a description that compares or distinguishes one class (or concept) from other comparable classes (or concepts). Class discrimination or comparison (hereafter referred to as **class comparison**) mines descriptions that distinguish a target class from its contrasting classes. Notice that the target and contrasting classes must be *comparable* in the sense that they share similar dimensions and attributes. For example, the three classes *person, address*, and *item* are not comparable. However, sales in the last three years are comparable classes, and so are, for example, computer science students versus physics students.

Our discussions on class characterization in the previous sections handle multilevel data summarization and characterization in a single class. The techniques developed can be extended to handle class comparison across several comparable classes. For example, the attribute generalization process described for class characterization can be modified so that the generalization is performed *synchronously* among all the classes compared. This allows the attributes in all of the classes to be generalized to the *same* abstraction levels.

Suppose, for instance, that we are given the *AllElectronics* data for sales in 2009 and in 2010 and want to compare these two classes. Consider the dimension *location* with abstractions at the *city*, *province_or_state*, and *country* levels. Data in each class should be generalized to the same *location* level. That is, they are all synchronously generalized to either the *city* level, the *province_or_state* level, or the *country* level. Ideally, this is more useful than comparing, say, the sales in Vancouver in 2009 with the sales in the United States in 2010 (i.e., where each set of sales data is generalized to a different level). The users, however, should have the option to overwrite such an automated, synchronous comparison with their own choices, when preferred.

"*How is class comparison performed?*" In general, the procedure is as follows:

1. **Data collection**: The set of relevant data in the database is collected by query processing and is partitioned respectively into a *target class* and one or a set of *contrasting classes*.
2. **Dimension relevance analysis**: If there are many dimensions, then dimension relevance analysis should be performed on these classes to select only the highly relevant dimensions for further analysis. Correlation or entropy-based measures can be used for this step (Chapter 3).
3. **Synchronous generalization**: Generalization is performed on the target class to the level controlled by a user- or expert-specified dimension threshold, which results in a **prime target class relation**. The concepts in the contrasting class(es) are generalized to the same level as those in the prime target class relation, forming the **prime contrasting class(es) relation**.
4. **Presentation of the derived comparison**: The resulting class comparison description can be visualized in the form of tables, graphs, and rules. This presentation usually includes a "contrasting" measure such as count% (percentage count) that reflects the

comparison between the target and contrasting classes. The user can adjust the comparison description by applying drill-down, roll-up, and other OLAP operations to the target and contrasting classes, as desired.

The preceding discussion outlines a general algorithm for mining comparisons in databases. In comparison with characterization, the previous algorithm involves synchronous generalization of the target class with the contrasting classes, so that classes are simultaneously compared at the same abstraction levels.

Example 4.14 mines a class comparison describing the graduate and undergraduate students at *Big University*.

Example 4.14 Mining a class comparison. Suppose that you would like to compare the general properties of the graduate and undergraduate students at *Big_University*, given the attributes *name, gender, major, birth_place, birth_date, residence, phone#*, and *gpa*.

This data mining task can be expressed in DMQL as follows:

```
use Big_University_DB
mine comparison as "grad_vs_undergrad_students"
in relevance to name, gender, major, birth_place, birth_date, residence,
    phone#, gpa
for "graduate_students"
where status in "graduate"
versus "undergraduate_students"
where status in "undergraduate"
analyze count%
from student
```

Let's see how this typical example of a data mining query for mining comparison descriptions can be processed.

First, the query is transformed into two relational queries that collect two sets of task-relevant data: one for the *initial target-class working relation* and the other for the *initial contrasting-class working relation*, as shown in Tables 4.8 and 4.9. This can also be viewed as the construction of a data cube, where the status {graduate, undergraduate} serves as one dimension, and the other attributes form the remaining dimensions.

Second, dimension relevance analysis can be performed, when necessary, on the two classes of data. After this analysis, irrelevant or weakly relevant dimensions (e.g., *name, gender, birth_place, residence*, and *phone#*) are removed from the resulting classes. Only the highly relevant attributes are included in the subsequent analysis.

Third, synchronous generalization is performed on the target class to the levels controlled by user- or expert-specified dimension thresholds, forming the *prime target class relation*. The contrasting class is generalized to the same levels as those in the prime target class relation, forming the *prime contrasting class(es) relation*, as presented in Tables 4.10 and 4.11. In comparison with undergraduate students, graduate students tend to be older and have a higher GPA in general.

Table 4.8 Initial Working Relations: The Target Class (Graduate Students)

name	*gender*	*major*	*birth_place*	*birth_date*	*residence*	*phone#*	*gpa*
Jim Woodman	M	CS	Vancouver, BC, Canada	12-8-76	3511 Main St., Richmond	687-4598	3.67
Scott Lachance	M	CS	Montreal, Que, Canada	7-28-75	345 1st Ave., Vancouver	253-9106	3.70
Laura Lee	F	Physics	Seattle, WA, USA	8-25-70	125 Austin Ave., Burnaby	420-5232	3.83
...	...	...	...	...	...	...	...

Table 4.9 Initial Working Relations: The Contrasting Class (Undergraduate Students)

name	*gender*	*major*	*birth_place*	*birth_date*	*residence*	*phone#*	*gpa*
Bob Schumann	M	Chemistry	Calgary, Alt, Canada	1-10-78	2642 Halifax St., Burnaby	294-4291	2.96
Amy Eau	F	Biology	Golden, BC, Canada	3-30-76	463 Sunset Cres., Vancouver	681-5417	3.52
...	...	...	...	...	...	...	...

Table 4.10 Prime Generalized Relation for the Target Class (Graduate Students)

major	*age_range*	*gpa*	count%
Science	21...25	good	5.53
Science	26...30	good	5.02
Science	over_30	very good	5.86
...	...	...	...
Business	over_30	excellent	4.68

Table 4.11 Prime Generalized Relation for the Contrasting Class (Undergraduate Students)

major	*age_range*	*gpa*	count%
Science	16...20	fair	5.53
Science	16...20	good	4.53
...	...	...	...
Science	26...30	good	2.32
...	...	...	...
Business	over_30	excellent	0.68

Finally, the resulting class comparison is presented in the form of tables, graphs, and/or rules. This visualization includes a contrasting measure (e.g., count%) that compares the target class and the contrasting class. For example, 5.02% of the graduate students majoring in science are between 26 and 30 years old and have a "good" GPA, while only 2.32% of undergraduates have these same characteristics. Drilling and other

OLAP operations may be performed on the target and contrasting classes as deemed necessary by the user in order to adjust the abstraction levels of the final description. ■

In summary, attribute-oriented induction for data characterization and generalization provides an alternative data generalization method in comparison to the data cube approach. It is not confined to relational data because such an induction can be performed on spatial, multimedia, sequence, and other kinds of data sets. In addition, there is no need to precompute a data cube because generalization can be performed online upon receiving a user's query.

Moreover, automated analysis can be added to such an induction process to automatically filter out irrelevant or unimportant attributes. However, because attribute-oriented induction automatically generalizes data to a higher level, it cannot efficiently support the process of drilling down to levels deeper than those provided in the generalized relation. The integration of data cube technology with attribute-oriented induction may provide a balance between precomputation and online computation. This would also support fast online computation when it is necessary to drill down to a level deeper than that provided in the generalized relation.

4.6 Summary

- A **data warehouse** is a *subject-oriented, integrated, time-variant*, and *nonvolatile* data collection organized in support of management decision making. Several factors distinguish data warehouses from operational databases. Because the two systems provide quite different functionalities and require different kinds of data, it is necessary to maintain data warehouses separately from operational databases.
- Data warehouses often adopt a **three-tier architecture**. The bottom tier is a *warehouse database server*, which is typically a relational database system. The middle tier is an *OLAP server*, and the top tier is a *client* that contains query and reporting tools.
- A data warehouse contains **back-end tools and utilities** for populating and refreshing the warehouse. These cover data extraction, data cleaning, data transformation, loading, refreshing, and warehouse management.
- Data warehouse **metadata** are data defining the warehouse objects. A metadata repository provides details regarding the warehouse structure, data history, the algorithms used for summarization, mappings from the source data to the warehouse form, system performance, and business terms and issues.
- A **multidimensional data model** is typically used for the design of corporate *data warehouses* and *departmental data marts*. Such a model can adopt a *star schema*, *snowflake schema*, or *fact constellation schema*. The core of the *multidimensional model* is the **data cube**, which consists of a large set of *facts* (or *measures*) and a number of *dimensions*. Dimensions are the entities or perspectives with respect to which an organization wants to keep records and are hierarchical in nature.

- A data cube consists of a **lattice of cuboids**, each corresponding to a different degree of summarization of the given multidimensional data.
- **Concept hierarchies** organize the values of attributes or dimensions into gradual abstraction levels. They are useful in mining at multiple abstraction levels.
- **Online analytical processing** can be performed in data warehouses/marts using the multidimensional data model. Typical OLAP operations include *roll-up*, and *drill-*(*down, across, through*), *slice-and-dice*, and *pivot* (*rotate*), as well as statistical operations such as ranking and computing moving averages and growth rates. OLAP operations can be implemented efficiently using the data cube structure.
- Data warehouses are used for *information processing* (querying and reporting), *analytical processing* (which allows users to navigate through summarized and detailed data by OLAP operations), and *data mining* (which supports knowledge discovery). OLAP-based data mining is referred to as **multidimensional data mining** (also known as exploratory multidimensional data mining, online analytical mining, or OLAM). It emphasizes the interactive and exploratory nature of data mining.
- OLAP servers may adopt a **relational OLAP (ROLAP)**, a **multidimensional OLAP (MOLAP)**, or a **hybrid OLAP (HOLAP)** implementation. A ROLAP server uses an extended relational DBMS that maps OLAP operations on multidimensional data to standard relational operations. A MOLAP server maps multidimensional data views directly to array structures. A HOLAP server combines ROLAP and MOLAP. For example, it may use ROLAP for historic data while maintaining frequently accessed data in a separate MOLAP store.
- **Full materialization** refers to the computation of all of the cuboids in the lattice defining a data cube. It typically requires an excessive amount of storage space, particularly as the number of dimensions and size of associated concept hierarchies grow. This problem is known as the **curse of dimensionality**. Alternatively, **partial materialization** is the selective computation of a subset of the cuboids or subcubes in the lattice. For example, an **iceberg cube** is a data cube that stores only those cube cells that have an aggregate value (e.g., count) above some minimum support threshold.
- OLAP query processing can be made more efficient with the use of indexing techniques. In **bitmap indexing**, each attribute has its own bitmap index table. Bitmap indexing reduces join, aggregation, and comparison operations to bit arithmetic. **Join indexing** registers the joinable rows of two or more relations from a relational database, reducing the overall cost of OLAP join operations. **Bitmapped join indexing**, which combines the bitmap and join index methods, can be used to further speed up OLAP query processing.
- **Data generalization** is a process that abstracts a large set of task-relevant data in a database from a relatively low conceptual level to higher conceptual levels. Data generalization approaches include data cube-based data aggregation and

attribute-oriented induction. **Concept description** is the most basic form of descriptive data mining. It describes a given set of task-relevant data in a concise and summarative manner, presenting interesting general properties of the data. Concept (or class) description consists of **characterization** and **comparison** (or **discrimination**). The former summarizes and describes a data collection, called the **target class**, whereas the latter summarizes and distinguishes one data collection, called the **target class**, from other data collection(s), collectively called the **contrasting class(es)**.

- **Concept characterization** can be implemented using **data cube (OLAP-based) approaches** and the **attribute-oriented induction approach**. These are attribute- or dimension-based generalization approaches. The **attribute-oriented induction approach** consists of the following techniques: *data focusing*, *data generalization by attribute removal or attribute generalization*, *count and aggregate value accumulation*, *attribute generalization control*, and *generalization data visualization*.
- **Concept comparison** can be performed using the attribute-oriented induction or data cube approaches in a manner similar to concept characterization. Generalized tuples from the target and contrasting classes can be quantitatively compared and contrasted.

4.7 Exercises

4.1 State why, for the integration of multiple heterogeneous information sources, many companies in industry prefer the *update-driven approach* (which constructs and uses data warehouses), rather than the *query-driven approach* (which applies wrappers and integrators). Describe situations where the query-driven approach is preferable to the update-driven approach.

4.2 Briefly compare the following concepts. You may use an example to explain your point(s).

(a) Snowflake schema, fact constellation, starnet query model

(b) Data cleaning, data transformation, refresh

(c) Discovery-driven cube, multifeature cube, virtual warehouse

4.3 Suppose that a data warehouse consists of the three dimensions *time, doctor*, and *patient*, and the two measures *count* and *charge*, where *charge* is the fee that a doctor charges a patient for a visit.

(a) Enumerate three classes of schemas that are popularly used for modeling data warehouses.

(b) Draw a schema diagram for the above data warehouse using one of the schema classes listed in (a).

(c) Starting with the base cuboid [*day*, *doctor*, *patient*], what specific *OLAP operations* should be performed in order to list the total fee collected by each doctor in 2010?

(d) To obtain the same list, write an SQL query assuming the data are stored in a relational database with the schema *fee* (*day, month, year, doctor, hospital, patient, count, charge*).

4.4 Suppose that a data warehouse for *Big_University* consists of the four dimensions *student, course, semester*, and *instructor*, and two measures *count* and *avg_grade*. At the lowest conceptual level (e.g., for a given student, course, semester, and instructor combination), the *avg_grade* measure stores the actual course grade of the student. At higher conceptual levels, *avg_grade* stores the average grade for the given combination.

(a) Draw a *snowflake schema* diagram for the data warehouse.

(b) Starting with the base cuboid [*student*, *course*, *semester*, *instructor*], what specific *OLAP operations* (e.g., roll-up from *semester* to *year*) should you perform in order to list the average grade of *CS* courses for each *Big_University* student.

(c) If each dimension has five levels (including all), such as "*student* < *major* < *status* < *university* < all", how many cuboids will this cube contain (including the base and apex cuboids)?

4.5 Suppose that a data warehouse consists of the four dimensions *date, spectator, location*, and *game*, and the two measures *count* and *charge*, where *charge* is the fare that a spectator pays when watching a game on a given date. Spectators may be students, adults, or seniors, with each category having its own charge rate.

(a) Draw a *star schema* diagram for the data warehouse.

(b) Starting with the base cuboid [*date*, *spectator*, *location*, *game*], what specific *OLAP operations* should you perform in order to list the total charge paid by student spectators at *GM_Place* in 2010?

(c) *Bitmap indexing* is useful in data warehousing. Taking this cube as an example, briefly discuss advantages and problems of using a bitmap index structure.

4.6 A data warehouse can be modeled by either a *star schema* or a *snowflake schema*. Briefly describe the similarities and the differences of the two models, and then analyze their advantages and disadvantages with regard to one another. Give your opinion of which might be more empirically useful and state the reasons behind your answer.

4.7 Design a data warehouse for a regional weather bureau. The weather bureau has about 1000 probes, which are scattered throughout various land and ocean locations in the region to collect basic weather data, including air pressure, temperature, and precipitation at each hour. All data are sent to the central station, which has collected such data for more than 10 years. Your design should facilitate efficient querying and online analytical processing, and derive general weather patterns in multidimensional space.

4.8 A popular data warehouse implementation is to construct a multidimensional database, known as a data cube. Unfortunately, this may often generate a huge, yet very sparse, multidimensional matrix.

(a) Present an example illustrating such a huge and sparse data cube.

(b) Design an implementation method that can elegantly overcome this sparse matrix problem. Note that you need to explain your data structures in detail and discuss the space needed, as well as how to retrieve data from your structures.

(c) Modify your design in (b) to handle *incremental data updates*. Give the reasoning behind your new design.

4.9 Regarding the *computation of measures* in a data cube:

(a) Enumerate three categories of measures, based on the kind of aggregate functions used in computing a data cube.

(b) For a data cube with the three dimensions *time, location*, and *item*, which category does the function *variance* belong to? Describe how to compute it if the cube is partitioned into many chunks.
Hint: The formula for computing *variance* is $\frac{1}{N}\sum_{i=1}^{N}(x_i - \bar{x_i})^2$, where $\bar{x_i}$ is the average of x_is.

(c) Suppose the function is "*top 10 sales*." Discuss how to efficiently compute this measure in a data cube.

4.10 Suppose a company wants to design a data warehouse to facilitate the analysis of moving vehicles in an online analytical processing manner. The company registers huge amounts of auto movement data in the format of (*Auto_ID, location, speed, time*). Each *Auto_ID* represents a vehicle associated with information (e.g., *vehicle_category, driver_category*), and each location may be associated with a street in a city. Assume that a street map is available for the city.

(a) Design such a data warehouse to facilitate effective online analytical processing in multidimensional space.

(b) The movement data may contain noise. Discuss how you would develop a method to automatically discover data records that were likely erroneously registered in the data repository.

(c) The movement data may be sparse. Discuss how you would develop a method that constructs a reliable data warehouse despite the sparsity of data.

(d) If you want to drive from A to B starting at a particular time, discuss how a system may use the data in this warehouse to work out a fast route.

4.11 Radio-frequency identification is commonly used to trace object movement and perform inventory control. An RFID reader can successfully read an RFID tag from a limited distance at any scheduled time. Suppose a company wants to design a data warehouse to facilitate the analysis of objects with RFID tags in an online analytical processing manner. The company registers huge amounts of RFID data in the format of *(RFID, at_location, time)*, and also has some information about the objects carrying the RFID tag, for example, *(RFID, product_name, product_category, producer, date_produced, price)*.

(a) Design a data warehouse to facilitate effective registration and online analytical processing of such data.

(b) The RFID data may contain lots of redundant information. Discuss a method that maximally reduces redundancy during data registration in the RFID data warehouse.

(c) The RFID data may contain lots of noise such as missing registration and misread IDs. Discuss a method that effectively cleans up the noisy data in the RFID data warehouse.

(d) You may want to perform online analytical processing to determine how many TV sets were shipped from the LA seaport to BestBuy in Champaign, IL, by *month*, *brand*, and *price_range*. Outline how this could be done efficiently if you were to store such RFID data in the warehouse.

(e) If a customer returns a jug of milk and complains that is has spoiled before its expiration date, discuss how you can investigate such a case in the warehouse to find out what the problem is, either in shipping or in storage.

4.12 In many applications, new data sets are incrementally added to the existing large data sets. Thus, an important consideration is whether a measure can be computed efficiently in an incremental manner. Use *count, standard deviation*, and *median* as examples to show that a distributive or algebraic measure facilitates efficient incremental computation, whereas a holistic measure does not.

4.13 Suppose that we need to record three measures in a data cube: min(), average(), and median(). Design an efficient computation and storage method for each measure given that the cube allows data to be *deleted incrementally* (i.e., in small portions at a time) from the cube.

4.14 In data warehouse technology, a multiple dimensional view can be implemented by a relational database technique (*ROLAP*), by a multidimensional database technique (*MOLAP*), or by a hybrid database technique (*HOLAP*).

(a) Briefly describe each implementation technique.

(b) For each technique, explain how each of the following functions may be implemented:

i. The generation of a data warehouse (including aggregation)

ii. Roll-up

iii. Drill-down

iv. Incremental updating

(c) Which implementation techniques do you prefer, and why?

4.15 Suppose that a data warehouse contains 20 dimensions, each with about five levels of granularity.

(a) Users are mainly interested in four particular dimensions, each having three frequently accessed levels for rolling up and drilling down. How would you design a data cube structure to support this preference efficiently?

(b) At times, a user may want to *drill through* the cube to the raw data for one or two particular dimensions. How would you support this feature?

4.16 A data cube, *C*, has *n* dimensions, and each dimension has exactly *p* distinct values in the base cuboid. Assume that there are no concept hierarchies associated with the dimensions.

(a) What is the *maximum number of cells* possible in the base cuboid?

(b) What is the *minimum number of cells* possible in the base cuboid?

(c) What is the *maximum number of cells* possible (including both base cells and aggregate cells) in the *C* data cube?

(d) What is the *minimum number of cells* possible in *C*?

4.17 What are the differences between the three main types of data warehouse usage: *information processing*, *analytical processing*, and *data mining*? Discuss the motivation behind *OLAP mining (OLAM)*.

4.8 Bibliographic Notes

There are a good number of introductory-level textbooks on data warehousing and OLAP technology—for example, Kimball, Ross, Thornthwaite, et al. [KRTM08]; Imhoff, Galemmo, and Geiger [IGG03]; and Inmon [Inm96]. Chaudhuri and Dayal [CD97] provide an early overview of data warehousing and OLAP technology. A set of research papers on materialized views and data warehouse implementations were collected in *Materialized Views: Techniques, Implementations, and Applications* by Gupta and Mumick [GM99].

The history of decision support systems can be traced back to the 1960s. However, the proposal to construct large data warehouses for multidimensional data analysis is credited to Codd [CCS93] who coined the term *OLAP* for *online analytical processing*. The OLAP Council was established in 1995. Widom [Wid95] identified several research problems in data warehousing. Kimball and Ross [KR02] provide an overview of the deficiencies of SQL regarding the ability to support comparisons that are common in the business world, and present a good set of application cases that require data warehousing and OLAP technology. For an overview of OLAP systems versus statistical databases, see Shoshani [Sho97].

Gray et al. [GCB+97] proposed the data cube as a relational aggregation operator generalizing group-by, crosstabs, and subtotals. Harinarayan, Rajaraman, and Ullman [HRU96] proposed a greedy algorithm for the partial materialization of cuboids in the computation of a data cube. Data cube computation methods have been investigated by numerous studies such as Sarawagi and Stonebraker [SS94]; Agarwal et al. [AAD+96]; Zhao, Deshpande, and Naughton [ZDN97]; Ross and Srivastava [RS97]; Beyer and Ramakrishnan [BR99]; Han, Pei, Dong, and Wang [HPDW01]; and Xin, Han, Li, and Wah [XHLW03]. These methods are discussed in depth in Chapter 5.

The concept of iceberg queries was first introduced in Fang, Shivakumar, Garcia-Molina et al. [FSGM+98]. The use of join indices to speed up relational query processing was proposed by Valduriez [Val87]. O'Neil and Graefe [OG95] proposed a bitmapped

join index method to speed up OLAP-based query processing. A discussion of the performance of bitmapping and other nontraditional index techniques is given in O'Neil and Quass [OQ97].

For work regarding the selection of materialized cuboids for efficient OLAP query processing, see, for example, Chaudhuri and Dayal [CD97]; Harinarayan, Rajaraman, and Ullman [HRU96]; and Sristava et al. [SDJL96]. Methods for cube size estimation can be found in Deshpande et al. [DNR+97], Ross and Srivastava [RS97], and Beyer and Ramakrishnan [BR99]. Agrawal, Gupta, and Sarawagi [AGS97] proposed operations for modeling multidimensional databases. Methods for answering queries quickly by online aggregation are described in Hellerstein, Haas, and Wang [HHW97] and Hellerstein et al. [HAC+99]. Techniques for estimating the top N queries are proposed in Carey and Kossman [CK98] and Donjerkovic and Ramakrishnan [DR99]. Further studies on intelligent OLAP and discovery-driven exploration of data cubes are presented in the bibliographic notes in Chapter 5.

This page intentionally left blank

5 Data Cube Technology

Data warehouse systems provide online analytical processing (OLAP) tools for interactive analysis of multidimensional data at varied granularity levels. OLAP tools typically use the *data cube* and a multidimensional data model to provide flexible access to summarized data. For example, a data cube can store precomputed measures (like count() and total_sales()) for multiple combinations of data dimensions (like *item*, *region*, and *customer*). Users can pose OLAP queries on the data. They can also interactively explore the data in a multidimensional way through OLAP operations like *drill-down* (to see more specialized data such as total sales per city) or *roll-up* (to see the data at a more generalized level such as total sales per country).

Although the data cube concept was originally intended for OLAP, it is also useful for data mining. **Multidimensional data mining** is an approach to data mining that integrates OLAP-based data analysis with knowledge discovery techniques. It is also known as *exploratory multidimensional data mining* and *online analytical mining* (*OLAM*). It searches for interesting patterns by exploring the data in multidimensional space. This gives users the freedom to dynamically focus on any subset of interesting dimensions. Users can interactively drill down or roll up to varying abstraction levels to find classification models, clusters, predictive rules, and outliers.

This chapter focuses on data cube technology. In particular, we study methods for data cube computation and methods for multidimensional data analysis. Precomputing a data cube (or parts of a data cube) allows for fast accessing of summarized data. Given the high dimensionality of most data, multidimensional analysis can run into performance bottlenecks. Therefore, it is important to study data cube computation techniques. Luckily, data cube technology provides many effective and scalable methods for cube computation. Studying these methods will also help in our understanding and further development of scalable methods for other data mining tasks such as the discovery of frequent patterns (Chapters 6 and 7).

We begin in Section 5.1 with preliminary concepts for cube computation. These summarize the data cube notion as a lattice of cuboids, and describe basic forms of cube materialization. General strategies for cube computation are given. Section 5.2 follows with an in-depth look at specific methods for data cube computation. We study both *full materialization* (i.e., where all the cuboids representing a data cube are precomputed

© 2012 Elsevier Inc. All rights reserved.

and thereby ready for use) and *partial cuboid materialization* (where, say, only the more "useful" parts of the data cube are precomputed). The *multiway array aggregation* method is detailed for full cube computation. Methods for partial cube computation, including *BUC*, *Star-Cubing*, and the use of *cube shell fragments*, are discussed.

In Section 5.3, we study cube-based query processing. The techniques described build on the standard methods of cube computation presented in Section 5.2. You will learn about *sampling cubes* for OLAP query answering on sampling data (e.g., survey data, which represent a sample or subset of a target data population of interest). In addition, you will learn how to compute *ranking cubes* for efficient top-k (ranking) query processing in large relational data sets.

In Section 5.4, we describe various ways to perform multidimensional data analysis using data cubes. *Prediction cubes* are introduced, which facilitate predictive modeling in multidimensional space. We discuss *multifeature cubes*, which compute complex queries involving multiple dependent aggregates at multiple granularities. You will also learn about the *exception-based discovery-driven exploration* of cube space, where visual cues are displayed to indicate discovered data exceptions at all aggregation levels, thereby guiding the user in the data analysis process.

5.1 Data Cube Computation: Preliminary Concepts

Data cubes facilitate the online analytical processing of multidimensional data. *"But how can we compute data cubes in advance, so that they are handy and readily available for query processing?"* This section contrasts full cube materialization (i.e., precomputation) versus various strategies for partial cube materialization. For completeness, we begin with a review of the basic terminology involving data cubes. We also introduce a cube cell notation that is useful for describing data cube computation methods.

5.1.1 Cube Materialization: Full Cube, Iceberg Cube, Closed Cube, and Cube Shell

Figure 5.1 shows a 3-D data cube for the dimensions *A*, *B*, and *C*, and an aggregate measure, *M*. Commonly used measures include count(), sum(), min(), max(), and total_sales(). A data cube is a lattice of cuboids. Each cuboid represents a group-by. *ABC* is the base cuboid, containing all three of the dimensions. Here, the aggregate measure, *M*, is computed for each possible combination of the three dimensions. The base cuboid is the least generalized of all the cuboids in the data cube. The most generalized cuboid is the apex cuboid, commonly represented as all. It contains one value—it aggregates measure *M* for all the tuples stored in the base cuboid. To drill down in the data cube, we move from the apex cuboid downward in the lattice. To roll up, we move from the base cuboid upward. For the purposes of our discussion in this chapter, we will always use the term *data cube* to refer to a lattice of cuboids rather than an individual cuboid.

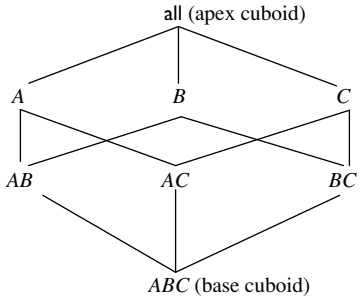


Figure 5.1 Lattice of cuboids making up a 3-D data cube with the dimensions *A*, *B*, and *C* for some aggregate measure, *M*.

A cell in the base cuboid is a **base cell**. A cell from a nonbase cuboid is an **aggregate cell**. An aggregate cell aggregates over one or more dimensions, where each aggregated dimension is indicated by a $*$ in the cell notation. Suppose we have an n-dimensional data cube. Let $a = (a_1, a_2, \ldots, a_n, measures)$ be a cell from one of the cuboids making up the data cube. We say that a is an ***m*-dimensional cell** (i.e., from an m-dimensional cuboid) if exactly m ($m \leq n$) values among $\{a_1, a_2, \ldots, a_n\}$ are *not* $*$. If $m = n$, then a is a base cell; otherwise, it is an aggregate cell (i.e., where $m < n$).

Example 5.1 **Base and aggregate cells.** Consider a data cube with the dimensions *month, city*, and *customer_group*, and the measure *sales*. (*Jan*, $*$, $*$, 2800) and ($*$, *Chicago*, $*$, 1200) are 1-D cells; (*Jan*, $*$, *Business*, 150) is a 2-D cell; and (*Jan*, *Chicago*, *Business*, 45) is a 3-D cell. Here, all base cells are 3-D, whereas 1-D and 2-D cells are aggregate cells. ■

An ancestor–descendant relationship may exist between cells. In an n-dimensional data cube, an i-D cell $a = (a_1, a_2, \ldots, a_n, measures_a)$ is an **ancestor** of a j-D cell $b = (b_1, b_2, \ldots, b_n, measures_b)$, and b is a **descendant** of a, if and only if (1) $i < j$, and (2) for $1 \leq k \leq n$, $a_k = b_k$ whenever $a_k \neq *$. In particular, cell a is called a **parent** of cell b, and b is a **child** of a, if and only if $j = i + 1$.

Example 5.2 **Ancestor and descendant cells.** Referring to Example 5.1, 1-D cell $a = (Jan, *, *, 2800)$ and 2-D cell $b = (Jan, *, Business, 150)$ are *ancestors* of 3-D cell $c = (Jan, Chicago, Business, 45)$; c is a *descendant* of both a and b; b is a *parent* of c; and c is a *child* of b. ■

To ensure fast OLAP, it is sometimes desirable to precompute the **full cube** (i.e., all the cells of all the cuboids for a given data cube). A method of full cube computation is given in Section 5.2.1. Full cube computation, however, is exponential to the number of dimensions. That is, a data cube of n dimensions contains 2^n cuboids. There are even

more cuboids if we consider concept hierarchies for each dimension.[1] In addition, the size of each cuboid depends on the cardinality of its dimensions. Thus, precomputation of the full cube can require huge and often excessive amounts of memory.

Nonetheless, full cube computation algorithms are important. **Individual** cuboids may be stored on secondary storage and accessed when necessary. Alternatively, we can use such algorithms to compute smaller cubes, consisting of a subset of the given set of dimensions, or a smaller range of possible values for some of the dimensions. In these cases, the smaller cube is a full cube for the given subset of dimensions and/or dimension values. A thorough understanding of full cube computation methods will help us develop efficient methods for computing partial cubes. Hence, it is important to explore scalable methods for computing all the cuboids making up a data cube, that is, for full materialization. These methods must take into consideration the limited amount of main memory available for cuboid computation, the total size of the computed data cube, as well as the time required for such computation.

Partial materialization of data cubes offers an interesting trade-off between storage space and response time for OLAP. Instead of computing the full cube, we can compute only a subset of the data cube's cuboids, or subcubes consisting of subsets of cells from the various cuboids.

Many cells in a cuboid may actually be of little or no interest to the data analyst. Recall that each cell in a full cube records an aggregate value such as count or sum. For many cells in a cuboid, the measure value will be zero. When the product of the cardinalities for the dimensions in a cuboid is large relative to the number of nonzero-valued tuples that are stored in the cuboid, then we say that the cuboid is **sparse**. If a cube contains many sparse cuboids, we say that the cube is **sparse**.

In many cases, a substantial amount of the cube's space could be taken up by a large number of cells with very low measure values. This is because the cube cells are often quite sparsely distributed within a multidimensional space. For example, a customer may only buy a few items in a store at a time. Such an event will generate only a few nonempty cells, leaving most other cube cells empty. In such situations, it is useful to materialize only those cells in a cuboid (group-by) with a measure value above some minimum threshold. In a data cube for sales, say, we may wish to materialize only those cells for which *count* $\geq$ *10* (i.e., where at least 10 tuples exist for the cell's given combination of dimensions), or only those cells representing *sales* $\geq$ $*100*. This not only saves processing time and disk space, but also leads to a more focused analysis. The cells that cannot pass the threshold are likely to be too trivial to warrant further analysis.

Such partially materialized cubes are known as **iceberg cubes**. The minimum threshold is called the **minimum support threshold**, or *minimum support* (*min_sup*), for short. By materializing only a fraction of the cells in a data cube, the result is seen as the "tip of the iceberg," where the "iceberg" is the potential full cube including all cells. An iceberg cube can be specified with an SQL query, as shown in Example 5.3.

[1] Eq. (4.1) of Section 4.4.1 gives the total number of cuboids in a data cube where each dimension has an associated concept hierarchy.

Example 5.3 Iceberg cube.

```
compute cube sales_iceberg as
select month, city, customer_group, count(*)
from salesInfo
cube by month, city, customer_group
having count(*) >= min_sup
```

The **compute cube** statement specifies the precomputation of the iceberg cube, *sales_iceberg*, with the dimensions *month*, *city*, and *customer_group*, and the aggregate measure **count()**. The input tuples are in the *salesInfo* relation. The **cube by** clause specifies that aggregates (group-by's) are to be formed for each of the possible subsets of the given dimensions. If we were computing the full cube, each group-by would correspond to a cuboid in the data cube lattice. The constraint specified in the **having** clause is known as the **iceberg condition**. Here, the iceberg measure is **count()**. Note that the iceberg cube computed here could be used to answer group-by queries on any combination of the specified dimensions of the form **having count(*) >=** v, where $v \geq min_sup$. Instead of **count()**, the iceberg condition could specify more complex measures such as **average()**.

If we were to omit the **having** clause, we would end up with the full cube. Let's call this cube *sales_cube*. The iceberg cube, *sales_iceberg*, excludes all the cells of *sales_cube* with a count that is less than *min_sup*. Obviously, if we were to set the minimum support to 1 in *sales_iceberg*, the resulting cube would be the full cube, *sales_cube*. ■

A naïve approach to computing an iceberg cube would be to first compute the full cube and then prune the cells that do not satisfy the iceberg condition. However, this is still prohibitively expensive. An efficient approach is to compute only the iceberg cube directly without computing the full cube. Sections 5.2.2 and 5.2.3 discuss methods for efficient iceberg cube computation.

Introducing iceberg cubes will lessen the burden of computing trivial aggregate cells in a data cube. However, we could still end up with a large number of uninteresting cells to compute. For example, suppose that there are 2 base cells for a database of 100 dimensions, denoted as $\{(a_1, a_2, a_3, \ldots, a_{100}) : 10, (a_1, a_2, b_3, \ldots, b_{100}) : 10\}$, where each has a cell count of 10. If the minimum support is set to 10, there will still be an impermissible number of cells to compute and store, although most of them are not interesting. For example, there are $2^{101} - 6$ distinct aggregate cells,[2] like $\{(a_1, a_2, a_3, a_4, \ldots, a_{99}, *) : 10, \ldots, (a_1, a_2, *, a_4, \ldots, a_{99}, a_{100}) : 10, \ldots, (a_1, a_2, a_3, *, \ldots, *, *) : 10\}$, but most of them do not contain much new information. If we ignore all the aggregate cells that can be obtained by replacing some constants by $*$'s while keeping the same measure value, there are only three distinct cells left: $\{(a_1, a_2, a_3, \ldots, a_{100}) : 10, (a_1, a_2, b_3, \ldots, b_{100}) : 10, (a_1, a_2, *, \ldots, *) : 20\}$. That is, out of $2^{101} - 4$ distinct base and aggregate cells, only three really offer valuable information.

[2]The proof is left as an exercise for the reader.

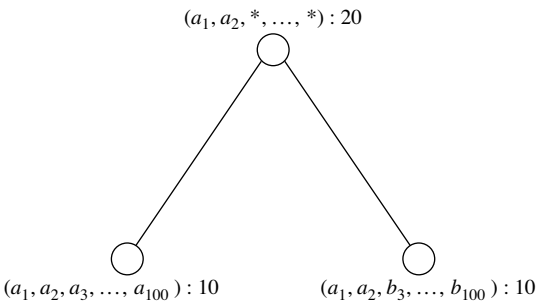


Figure 5.2 Three closed cells forming the lattice of a closed cube.

To systematically compress a data cube, we need to introduce the concept of *closed coverage*. A cell, c, is a *closed cell* if there exists no cell, d, such that d is a specialization (descendant) of cell c (i.e., where d is obtained by replacing $*$ in c with a non-$*$ value), and d has the same measure value as c. A **closed cube** is a data cube consisting of only closed cells. For example, the three cells derived in the preceding paragraph are the three closed cells of the data cube for the data set $\{(a_1, a_2, a_3, \ldots, a_{100}) : 10, (a_1, a_2, b_3, \ldots, b_{100}) : 10\}$. They form the lattice of a closed cube as shown in Figure 5.2. Other nonclosed cells can be derived from their corresponding closed cells in this lattice. For example, "$(a_1, *, *, \ldots, *) : 20$" can be derived from "$(a_1, a_2, *, \ldots, *) : 20$" because the former is a generalized nonclosed cell of the latter. Similarly, we have "$(a_1, a_2, b_3, *, \ldots, *) : 10$."

Another strategy for partial materialization is to precompute only the cuboids involving a small number of dimensions such as three to five. These cuboids form a **cube shell** for the corresponding data cube. Queries on additional combinations of the dimensions will have to be computed on-the-fly. For example, we could compute all cuboids with three dimensions or less in an n-dimensional data cube, resulting in a cube shell of size 3. This, however, can still result in a large number of cuboids to compute, particularly when n is large. Alternatively, we can choose to precompute only portions or *fragments* of the cube shell based on cuboids of interest. Section 5.2.4 discusses a method for computing **shell fragments** and explores how they can be used for efficient OLAP query processing.

5.1.2 General Strategies for Data Cube Computation

There are several methods for efficient data cube computation, based on the various kinds of cubes described in Section 5.1.1. In general, there are two basic data structures used for storing cuboids. The implementation of relational OLAP (ROLAP) uses relational tables, whereas multidimensional arrays are used in multidimensional OLAP (MOLAP). Although ROLAP and MOLAP may each explore different cube computation techniques, some optimization "tricks" can be shared among the different

data representations. The following are general optimization techniques for efficient computation of data cubes.

Optimization Technique 1: Sorting, hashing, and grouping. Sorting, hashing, and grouping operations should be applied to the dimension attributes to reorder and cluster related tuples.

In cube computation, aggregation is performed on the tuples (or cells) that share the same set of dimension values. Thus, it is important to explore sorting, hashing, and grouping operations to access and group such data together to facilitate computation of such aggregates.

To compute total sales by *branch*, *day*, and *item*, for example, it can be more efficient to sort tuples or cells by *branch*, and then by *day*, and then group them according to the *item* name. Efficient implementations of such operations in large data sets have been extensively studied in the database research community. Such implementations can be extended to data cube computation.

This technique can also be further extended to perform **shared-sorts** (i.e., sharing sorting costs across multiple cuboids when sort-based methods are used), or to perform **shared-partitions** (i.e., sharing the partitioning cost across multiple cuboids when hash-based algorithms are used).

Optimization Technique 2: Simultaneous aggregation and caching of intermediate results. In cube computation, it is efficient to compute higher-level aggregates from previously computed lower-level aggregates, rather than from the base fact table. Moreover, simultaneous aggregation from cached intermediate computation results may lead to the reduction of expensive disk input/output (I/O) operations.

To compute sales by *branch*, for example, we can use the intermediate results derived from the computation of a lower-level cuboid such as sales by *branch* and *day*. This technique can be further extended to perform **amortized scans** (i.e., computing as many cuboids as possible at the same time to amortize disk reads).

Optimization Technique 3: Aggregation from the smallest child when there exist multiple child cuboids. When there exist multiple child cuboids, it is usually more efficient to compute the desired parent (i.e., more generalized) cuboid from the smallest, previously computed child cuboid.

To compute a sales cuboid, C_{branch}, when there exist two previously computed cuboids, $C_{\{branch,year\}}$ and $C_{\{branch,item\}}$, for example, it is obviously more efficient to compute C_{branch} from the former than from the latter if there are many more distinct items than distinct years.

Many other optimization techniques may further improve computational efficiency. For example, string dimension attributes can be mapped to integers with values ranging from zero to the cardinality of the attribute.

In iceberg cube computation the following optimization technique plays a particularly important role.

Optimization Technique 4: The Apriori pruning method can be explored to compute iceberg cubes efficiently. The **Apriori property**,[3] in the context of data cubes, states as follows: *If a given cell does not satisfy minimum support, then no descendant of the cell (i.e., more specialized cell) will satisfy minimum support either.* This property can be used to substantially reduce the computation of iceberg cubes.

Recall that the specification of iceberg cubes contains an iceberg condition, which is a constraint on the cells to be materialized. A common iceberg condition is that the cells must satisfy a *minimum support* threshold such as a minimum count or sum. In this situation, the Apriori property can be used to prune away the exploration of the cell's descendants. For example, if the count of a cell, c, in a cuboid is less than a minimum support threshold, v, then the count of any of c's descendant cells in the lower-level cuboids can never be greater than or equal to v, and thus can be pruned.

In other words, if a condition (e.g., the iceberg condition specified in the `having` clause) is violated for some cell c, then every descendant of c will also violate that condition. Measures that obey this property are known as **antimonotonic**.[4] This form of pruning was made popular in frequent pattern mining, yet also aids in data cube computation by cutting processing time and disk space requirements. It can lead to a more focused analysis because cells that cannot pass the threshold are unlikely to be of interest.

In the following sections, we introduce several popular methods for efficient cube computation that explore these optimization strategies.

5.2 Data Cube Computation Methods

Data cube computation is an essential task in data warehouse implementation. The precomputation of all or part of a data cube can greatly reduce the response time and enhance the performance of online analytical processing. However, such computation is challenging because it may require substantial computational time and storage space. This section explores efficient methods for data cube computation. Section 5.2.1 describes the *multiway array aggregation* (MultiWay) method for computing full cubes. Section 5.2.2 describes a method known as BUC, which computes iceberg cubes from the apex cuboid downward. Section 5.2.3 describes the Star-Cubing method, which integrates top-down and bottom-up computation.

Finally, Section 5.2.4 describes a shell-fragment cubing approach that computes shell fragments for efficient high-dimensional OLAP. To simplify our discussion, we exclude

[3]The Apriori property was proposed in the Apriori algorithm for association rule mining by Agrawal and Srikant [AS94b]. Many algorithms in association rule mining have adopted this property (see Chapter 6).

[4]**Antimonotone** is based on *condition violation*. This differs from **monotone**, which is based on *condition satisfaction*.

the cuboids that would be generated by climbing up any existing hierarchies for the dimensions. Those cube types can be computed by extension of the discussed methods. Methods for the efficient computation of closed cubes are left as an exercise for interested readers.

5.2.1 Multiway Array Aggregation for Full Cube Computation

The **multiway array aggregation** (or simply **MultiWay**) method computes a full data cube by using a multidimensional array as its basic data structure. It is a typical MOLAP approach that uses direct array addressing, where dimension values are accessed via the position or index of their corresponding array locations. Hence, MultiWay cannot perform any value-based reordering as an optimization technique. A different approach is developed for the array-based cube construction, as follows:

1. Partition the array into chunks. A **chunk** is a subcube that is small enough to fit into the memory available for cube computation. **Chunking** is a method for dividing an n-dimensional array into small n-dimensional chunks, where each chunk is stored as an object on disk. The chunks are compressed so as to remove wasted space resulting from *empty array cells*. A cell is *empty* if it does not contain any valid data (i.e., its cell count is 0). For instance, "*chunkID* + *offset*" can be used as a cell-addressing mechanism to **compress a sparse array structure** and when searching for cells within a chunk. Such a compression technique is powerful at handling sparse cubes, both on disk and in memory.
2. Compute aggregates by visiting (i.e., accessing the values at) cube cells. The order in which cells are visited can be optimized so as to *minimize the number of times that each cell must be revisited*, thereby reducing memory access and storage costs. The trick is to exploit this ordering so that portions of the aggregate cells in multiple cuboids can be computed simultaneously, and any unnecessary revisiting of cells is avoided.

This chunking technique involves "overlapping" some of the aggregation computations; therefore, it is referred to as multiway array aggregation. It performs **simultaneous aggregation**, that is, it computes aggregations simultaneously on multiple dimensions.

We explain this approach to array-based cube construction by looking at a concrete example.

Example 5.4 **Multiway array cube computation.** Consider a 3-D data array containing the three dimensions A, B, and C. The 3-D array is partitioned into small, memory-based chunks. In this example, the array is partitioned into 64 chunks as shown in Figure 5.3. Dimension A is organized into four equal-sized partitions: a_0, a_1, a_2, and a_3. Dimensions B and C are similarly organized into four partitions each. Chunks 1, 2, ..., 64 correspond to the subcubes $a_0b_0c_0$, $a_1b_0c_0$, ..., $a_3b_3c_3$, respectively. Suppose that the cardinality of

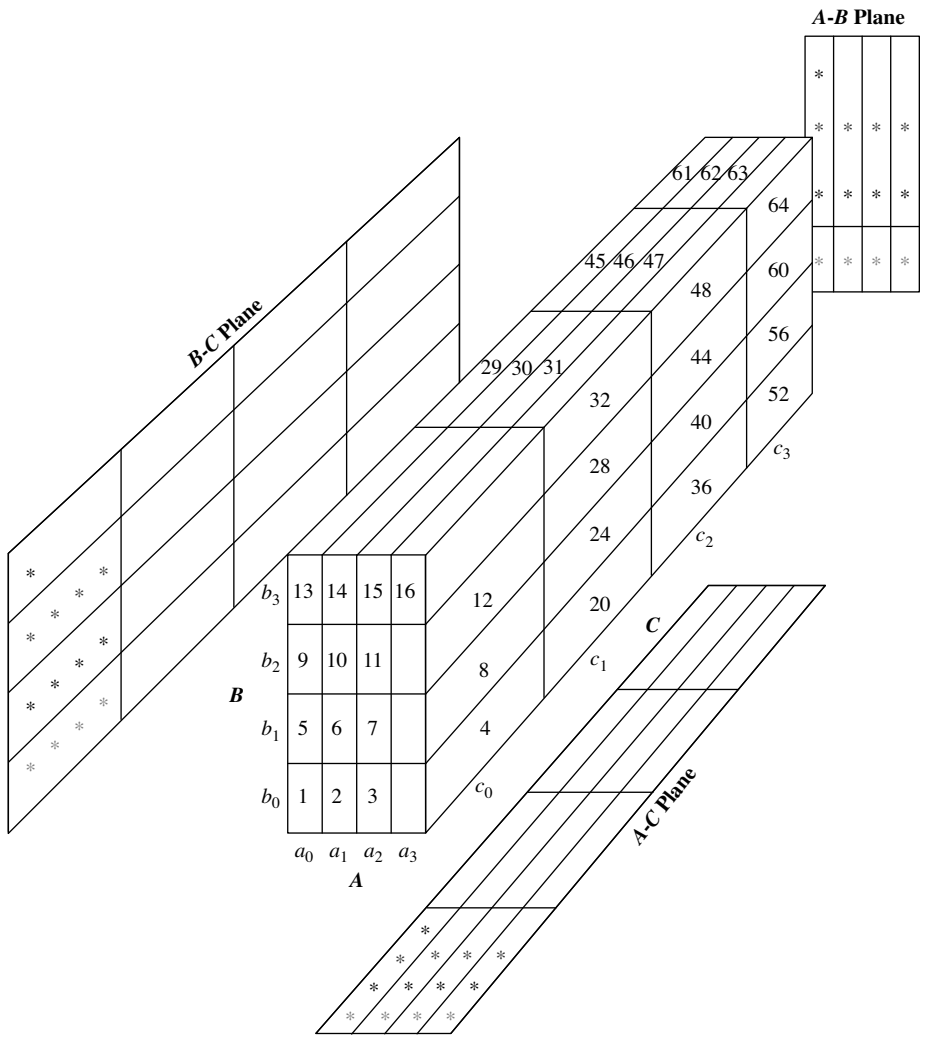


Figure 5.3 A 3-D array for the dimensions *A*, *B*, and *C*, organized into 64 *chunks*. Each chunk is small enough to fit into the memory available for cube computation. The ∗'s indicate the chunks from 1 to 13 that have been aggregated so far in the process.

the dimensions *A*, *B*, and *C* is 40, 400, and 4000, respectively. Thus, the size of the array for each dimension, *A*, *B*, and *C*, is also 40, 400, and 4000, respectively. The size of each partition in *A*, *B*, and *C* is therefore 10, 100, and 1000, respectively. Full materialization of the corresponding data cube involves the computation of all the cuboids defining this cube. The resulting full cube consists of the following cuboids:

- The base cuboid, denoted by *ABC* (from which all the other cuboids are directly or indirectly computed). This cube is already computed and corresponds to the given 3-D array.
- The 2-D cuboids, *AB*, *AC*, and *BC*, which respectively correspond to the group-by's *AB*, *AC*, and *BC*. These cuboids must be computed.
- The 1-D cuboids, *A*, *B*, and *C*, which respectively correspond to the group-by's *A*, *B*, and *C*. These cuboids must be computed.
- The 0-D (apex) cuboid, denoted by **all**, which corresponds to the group-by (); that is, there is no group-by here. This cuboid must be computed. It consists of only one value. If, say, the data cube measure is **count**, then the value to be computed is simply the total count of all the tuples in *ABC*.

Let's look at how the multiway array aggregation technique is used in this computation. There are many possible orderings with which chunks can be read into memory for use in cube computation. Consider the ordering labeled from 1 to 64, shown in Figure 5.3. Suppose we want to compute the b_0c_0 chunk of the *BC* cuboid. We allocate space for this chunk in *chunk memory*. By scanning *ABC* chunks 1 through 4, the b_0c_0 chunk is computed. That is, the cells for b_0c_0 are aggregated over a_0 to a_3. The chunk memory can then be assigned to the next chunk, b_1c_0, which completes its aggregation after the scanning of the next four *ABC* chunks: 5 through 8. Continuing in this way, the entire *BC* cuboid can be computed. Therefore, only *one* *BC* chunk needs to be in memory at a time, for the computation of all the *BC* chunks.

In computing the *BC* cuboid, we will have scanned each of the 64 chunks. "*Is there a way to avoid having to rescan all of these chunks for the computation of other cuboids such as AC and AB?*" The answer is, most definitely, *yes*. This is where the "multiway computation" or "simultaneous aggregation" idea comes in. For example, when chunk 1 (i.e., $a_0b_0c_0$) is being scanned (say, for the computation of the 2-D chunk b_0c_0 of *BC*, as described previously), all of the other 2-D chunks relating to $a_0b_0c_0$ can be simultaneously computed. That is, when $a_0b_0c_0$ is being scanned, each of the three chunks (b_0c_0, a_0c_0, and a_0b_0) on the three 2-D aggregation planes (*BC*, *AC*, and *AB*) should be computed then as well. In other words, multiway computation simultaneously aggregates to each of the 2-D planes while a 3-D chunk is in memory.

Now let's look at how different orderings of chunk scanning and of cuboid computation can affect the overall data cube computation efficiency. Recall that the size of the dimensions *A*, *B*, and *C* is 40, 400, and 4000, respectively. Therefore, the largest 2-D plane is *BC* (of size $400 \times 4000 = 1{,}600{,}000$). The second largest 2-D plane is *AC* (of size $40 \times 4000 = 160{,}000$). *AB* is the smallest 2-D plane (of size $40 \times 400 = 16{,}000$).

Suppose that the chunks are scanned in the order shown, from chunks 1 to 64. As previously mentioned, b_0c_0 is fully aggregated after scanning the row containing chunks 1 through 4; b_1c_0 is fully aggregated after scanning chunks 5 through 8, and so on. Thus, we need to scan four chunks of the 3-D array to *fully* compute one chunk of the *BC* cuboid (where *BC* is the largest of the 2-D planes). In other words, by scanning in this

order, one *BC* chunk is fully computed for each row scanned. In comparison, the complete computation of one chunk of the second largest 2-D plane, *AC*, requires scanning 13 chunks, given the ordering from 1 to 64. That is, a_0c_0 is fully aggregated only after the scanning of chunks 1, 5, 9, and 13.

Finally, the complete computation of one chunk of the smallest 2-D plane, *AB*, requires scanning 49 chunks. For example, a_0b_0 is fully aggregated after scanning chunks 1, 17, 33, and 49. Hence, *AB* requires the longest scan of chunks to complete its computation. To avoid bringing a 3-D chunk into memory more than once, the minimum memory requirement for holding all relevant 2-D planes in chunk memory, according to the chunk ordering of 1 to 64, is as follows: 40×400 (for the whole *AB* plane) + 40×1000 (for one column of the *AC* plane) + 100×1000 (for one *BC* plane chunk) = 16,000 + 40,000 + 100,000 = 156,000 memory units.

Suppose, instead, that the chunks are scanned in the order 1, 17, 33, 49, 5, 21, 37, 53, and so on. That is, suppose the scan is in the order of first aggregating toward the *AB* plane, and then toward the *AC* plane, and lastly toward the *BC* plane. The minimum memory requirement for holding 2-D planes in chunk memory would be as follows: 400×4000 (for the whole *BC* plane) + 40×1000 (for one *AC* plane row) + 10×100 (for one *AB* plane chunk) = 1,600,000 + 40,000 + 1000 = 1,641,000 memory units. Notice that this is *more than 10 times* the memory requirement of the scan ordering of 1 to 64.

Similarly, we can work out the minimum memory requirements for the multiway computation of the 1-D and 0-D cuboids. Figure 5.4 shows the most efficient way to compute 1-D cuboids. Chunks for 1-D cuboids *A* and *B* are computed during the computation of the smallest 2-D cuboid, *AB*. The smallest 1-D cuboid, *A*, will have all of its chunks allocated in memory, whereas the larger 1-D cuboid, *B*, will have only one chunk allocated in memory at a time. Similarly, chunk *C* is computed during the computation of the second smallest 2-D cuboid, *AC*, requiring only one chunk in memory at a time. Based on this analysis, we see that the most efficient ordering in this array cube computation is the chunk ordering of 1 to 64, with the stated memory allocation strategy. ■

Example 5.4 assumes that there is enough memory space for *one-pass* cube computation (i.e., to compute all of the cuboids from one scan of all the chunks). If there is insufficient memory space, the computation will require more than one pass through the 3-D array. In such cases, however, the basic principle of ordered chunk computation remains the same. MultiWay is most effective when the product of the cardinalities of dimensions is moderate and the data are not too sparse. When the dimensionality is high or the data are very sparse, the in-memory arrays become too large to fit in memory, and this method becomes infeasible.

With the use of appropriate sparse array compression techniques and careful ordering of the computation of cuboids, it has been shown by experiments that MultiWay array cube computation is significantly faster than traditional ROLAP (relational record-based) computation. Unlike ROLAP, the array structure of MultiWay does not require saving space to store search keys. Furthermore, MultiWay uses direct array addressing,

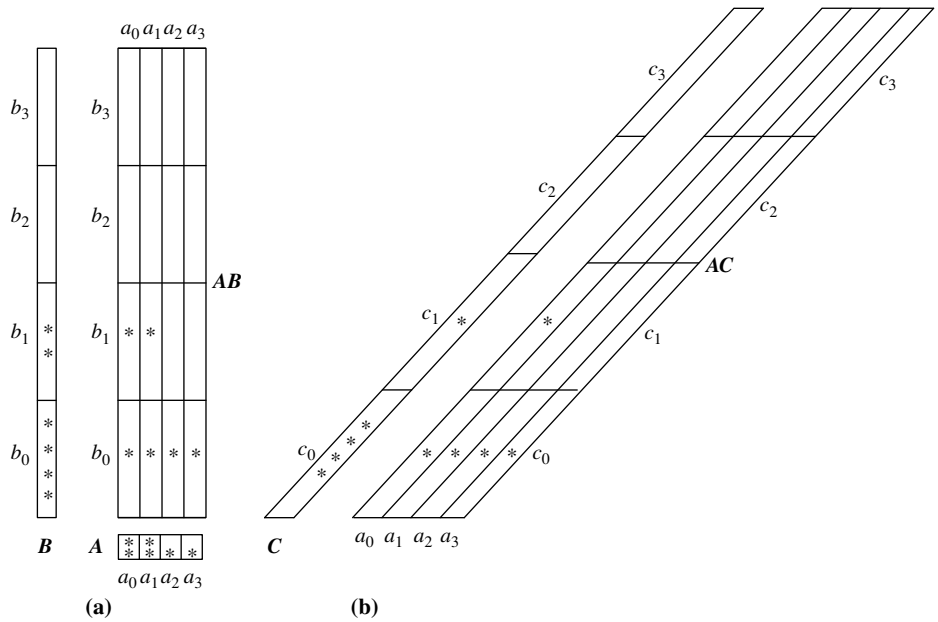


Figure 5.4 Memory allocation and computation order for computing Example 5.4's 1-D cuboids. (a) The 1-D cuboids, *A* and *B*, are aggregated during the computation of the smallest 2-D cuboid, *AB*. (b) The 1-D cuboid, *C*, is aggregated during the computation of the second smallest 2-D cuboid, *AC*. The ∗'s represent chunks that, so far, have been aggregated to.

which is faster than ROLAP's key-based addressing search strategy. For ROLAP cube computation, instead of cubing a table directly, it can be faster to convert the table to an array, cube the array, and then convert the result back to a table. However, this observation works only for cubes with a relatively small number of dimensions, because the number of cuboids to be computed is exponential to the number of dimensions.

"*What would happen if we tried to use MultiWay to compute iceberg cubes?*" Remember that the Apriori property states that if a given cell does not satisfy minimum support, then neither will any of its descendants. Unfortunately, MultiWay's computation starts from the base cuboid and progresses upward toward more generalized, ancestor cuboids. It cannot take advantage of Apriori pruning, which requires a parent node to be computed before its child (i.e., more specific) nodes. For example, if the count of a cell *c* in, say, *AB*, does not satisfy the minimum support specified in the iceberg condition, we cannot prune away cell *c*, because the count of *c*'s ancestors in the *A* or *B* cuboids may be greater than the minimum support, and their computation will need aggregation involving the count of *c*.

5.2.2 BUC: Computing Iceberg Cubes from the Apex Cuboid Downward

BUC is an algorithm for the computation of sparse and iceberg cubes. Unlike MultiWay, BUC constructs the cube from the apex cuboid toward the base cuboid. This allows BUC to share data partitioning costs. This processing order also allows BUC to prune during construction, using the Apriori property.

Figure 5.5 shows a lattice of cuboids, making up a 3-D data cube with the dimensions *A*, *B*, and *C*. The apex (0-D) cuboid, representing the concept **all** (i.e., $(*, *, *)$), is at the top of the lattice. This is the most aggregated or generalized level. The 3-D base cuboid, *ABC*, is at the bottom of the lattice. It is the least aggregated (most detailed or specialized) level. This representation of a lattice of cuboids, with the apex at the top and the base at the bottom, is commonly accepted in data warehousing. It consolidates the notions of *drill-down* (where we can move from a highly aggregated cell to lower, more detailed cells) and *roll-up* (where we can move from detailed, low-level cells to higher-level, more aggregated cells).

BUC stands for "Bottom-Up Construction." However, according to the lattice convention described before and used throughout this book, the BUC processing order is actually top-down! The BUC authors view a lattice of cuboids in the reverse order,

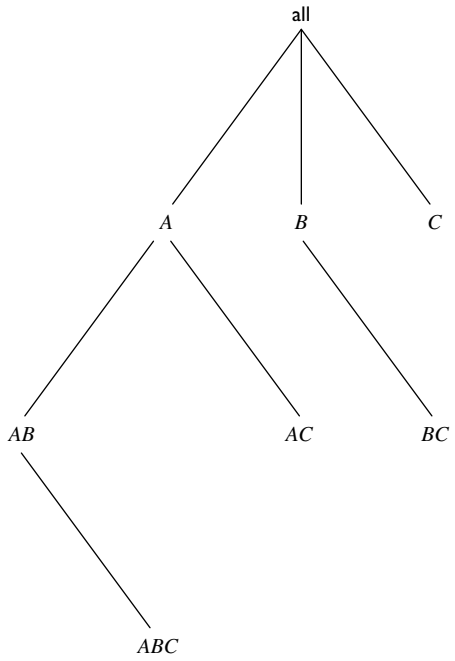


Figure 5.5 BUC's exploration for a 3-D data cube computation. Note that the computation starts from the apex cuboid.

with the apex cuboid at the bottom and the base cuboid at the top. In that view, BUC does bottom-up construction. However, because we adopt the application worldview where *drill-down* refers to drilling from the apex cuboid down toward the base cuboid, the exploration process of BUC is regarded as top-down. BUC's exploration for the computation of a 3-D data cube is shown in Figure 5.5.

The BUC algorithm is shown on the next page in Figure 5.6. We first give an explanation of the algorithm and then follow up with an example. Initially, the algorithm is called with the input relation (set of tuples). BUC aggregates the entire input (line 1) and writes the resulting total (line 3). (Line 2 is an optimization feature that is discussed later in our example.) For each dimension d (line 4), the input is partitioned on d (line 6). On return from Partition(), *dataCount* contains the total number of tuples for each distinct value of dimension d. Each distinct value of d *forms its own partition.* Line 8 iterates through each partition. Line 10 tests the partition for minimum support. That is, if the number of tuples in the partition satisfies (i.e., is $\geq$) the minimum support, then the partition becomes the input relation for a recursive call made to BUC, which computes the iceberg cube on the partitions for dimensions $d + 1$ to *numDims* (line 12).

Note that for a full cube (i.e., where minimum support in the having clause is 1), the minimum support condition is always satisfied. Thus, the recursive call descends one level deeper into the lattice. On return from the recursive call, we continue with the next partition for d. After all the partitions have been processed, the entire process is repeated for each of the remaining dimensions.

Example 5.5 BUC construction of an iceberg cube. Consider the iceberg cube expressed in SQL as follows:

```
compute cube iceberg_cube as
select A, B, C, D, count(*)
from R
cube by A, B, C, D
having count(*) >= 3
```

Let's see how BUC constructs the iceberg cube for the dimensions A, B, C, and D, where 3 is the minimum support count. Suppose that dimension A has four distinct values, a_1, a_2, a_3, a_4; B has four distinct values, b_1, b_2, b_3, b_4; C has two distinct values, c_1, c_2; and D has two distinct values, d_1, d_2. If we consider each group-by to be a *partition*, then we must compute every combination of the grouping attributes that satisfy the minimum support (i.e., that have three tuples).

Figure 5.7 illustrates how the input is partitioned first according to the different attribute values of dimension A, and then B, C, and D. To do so, BUC scans the input, aggregating the tuples to obtain a count for all, corresponding to the cell $(*, *, *, *)$. Dimension A is used to split the input into four partitions, one for each distinct value of A. The number of tuples (counts) for each distinct value of A is recorded in *dataCount*.

BUC uses the Apriori property to save time while searching for tuples that satisfy the iceberg condition. Starting with A dimension value, a_1, the a_1 partition is aggregated, creating one tuple for the A group-by, corresponding to the cell $(a_1, *, *, *)$.

Algorithm: BUC. Algorithm for the computation of sparse and iceberg cubes.

Input:

- *input*: the relation to aggregate;
- *dim*: the starting dimension for this iteration.

Globals:

- constant *numDims*: the total number of dimensions;
- constant *cardinality[numDims]*: the cardinality of each dimension;
- constant *min_sup*: the minimum number of tuples in a partition for it to be output;
- *outputRec*: the current output record;
- *dataCount[numDims]*: stores the size of each partition. *dataCount[i]* is a list of integers of size *cardinality[i]*.

Output: Recursively output the iceberg cube cells satisfying the minimum support.

Method:

```
(1)  Aggregate(input); // Scan input to compute measure, e.g., count. Place result in outputRec.
(2)  if input.count() == 1 then // Optimization
         WriteDescendants(input[0], dim); return;
     endif
(3)  write outputRec;
(4)  for (d = dim; d < numDims; d++) do //Partition each dimension
(5)      C = cardinality[d];
(6)      Partition(input, d, C, dataCount[d]); //create C partitions of data for dimension d
(7)      k = 0;
(8)      for (i = 0; i < C; i++) do // for each partition (each value of dimension d)
(9)              c = dataCount[d][i];
(10)             if c >= min_sup then // test the iceberg condition
(11)                     outputRec.dim[d] = input[k].dim[d];
(12)                     BUC(input[k..k + c − 1], d + 1); // aggregate on next dimension
(13)             endif
(14)             k +=c;
(15)     endfor
(16)     outputRec.dim[d] = all;
(17) endfor
```

Figure 5.6 BUC algorithm for sparse or iceberg cube computation. *Source:* Beyer and Ramakrishnan [BR99].

Suppose $(a_1, *, *, *)$ satisfies the minimum support, in which case a recursive call is made on the partition for a_1. BUC partitions a_1 on the dimension B. It checks the count of $(a_1, b_1, *, *)$ to see if it satisfies the minimum support. If it does, it outputs the aggregated tuple to the AB group-by and recurses on $(a_1, b_1, *, *)$ to partition on C, starting

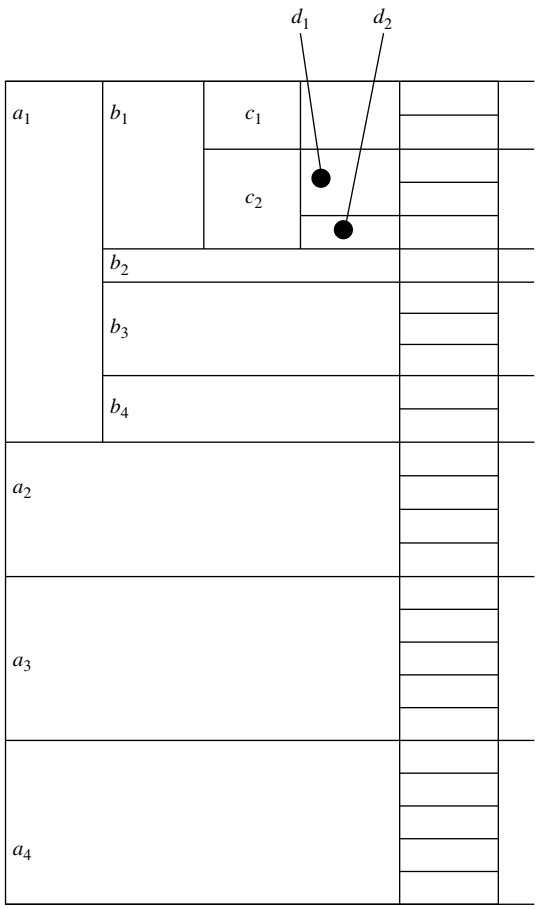


Figure 5.7 BUC partitioning snapshot given an example 4-D data set.

with c_1. Suppose the cell count for $(a_1, b_1, c_1, *)$ is 2, which does not satisfy the minimum support. According to the Apriori property, if a cell does not satisfy the minimum support, then neither can any of its descendants. Therefore, BUC prunes any further exploration of $(a_1, b_1, c_1, *)$. That is, it avoids partitioning this cell on dimension D. It backtracks to the a_1, b_1 partition and recurses on $(a_1, b_1, c_2, *)$, and so on. By checking the iceberg condition each time before performing a recursive call, BUC saves a great deal of processing time whenever a cell's count does not satisfy the minimum support.

The partition process is facilitated by a linear sorting method, CountingSort. CountingSort is fast because it does not perform any key comparisons to find partition boundaries. In addition, the counts computed during the sort can be reused to compute the group-by's in BUC. Line 2 is an optimization for partitions having a count of 1 such as $(a_1, b_2, *, *)$ in our example. To save on partitioning costs, the count is written

to each of the tuple's descendant group-by's. This is particularly useful since, in practice, many partitions have a single tuple. ■

The BUC performance is sensitive to the order of the dimensions and to skew in the data. Ideally, the most discriminating dimensions should be processed first. Dimensions should be processed in the order of decreasing cardinality. The higher the cardinality, the smaller the partitions, and thus the more partitions there will be, thereby providing BUC with a greater opportunity for pruning. Similarly, the more uniform a dimension (i.e., having less skew), the better it is for pruning.

BUC's major contribution is the idea of sharing partitioning costs. However, unlike MultiWay, it does not share the computation of aggregates between parent and child group-by's. For example, the computation of cuboid *AB* does not help that of *ABC*. The latter needs to be computed essentially from scratch.

5.2.3 Star-Cubing: Computing Iceberg Cubes Using a Dynamic Star-Tree Structure

In this section, we describe the **Star-Cubing** algorithm for computing iceberg cubes. Star-Cubing combines the strengths of the other methods we have studied up to this point. It integrates top-down and bottom-up cube computation and explores both multidimensional aggregation (similar to MultiWay) and Apriori-like pruning (similar to BUC). It operates from a data structure called a star-tree, which performs lossless data compression, thereby reducing the computation time and memory requirements.

The Star-Cubing algorithm explores both the bottom-up and top-down computation models as follows: On the global computation order, it uses the bottom-up model. However, it has a sublayer underneath based on the top-down model, which explores the notion of *shared dimensions*, as we shall see in the following. This integration allows the algorithm to aggregate on multiple dimensions while still partitioning parent group-by's and pruning child group-by's that do not satisfy the iceberg condition.

Star-Cubing's approach is illustrated in Figure 5.8 for a 4-D data cube computation. If we were to follow only the bottom-up model (similar to MultiWay), then the cuboids marked as pruned by Star-Cubing would still be explored. Star-Cubing is able to prune the indicated cuboids because it considers shared dimensions. *ACD*/*A* means cuboid *ACD* has shared dimension *A*, *ABD*/*AB* means cuboid *ABD* has shared dimension *AB*, *ABC*/*ABC* means cuboid *ABC* has shared dimension *ABC*, and so on. This comes from the generalization that all the cuboids in the subtree rooted at *ACD* include dimension *A*, all those rooted at *ABD* include dimensions *AB*, and all those rooted at *ABC* include dimensions *ABC* (even though there is only one such cuboid). We call these common dimensions the **shared dimensions** of those particular subtrees.

The introduction of shared dimensions facilitates shared computation. Because the shared dimensions are identified early on in the tree expansion, we can avoid recomputing them later. For example, cuboid *AB* extending from *ABD* in Figure 5.8 would actually be pruned because *AB* was already computed in *ABD*/*AB*. Similarly, cuboid

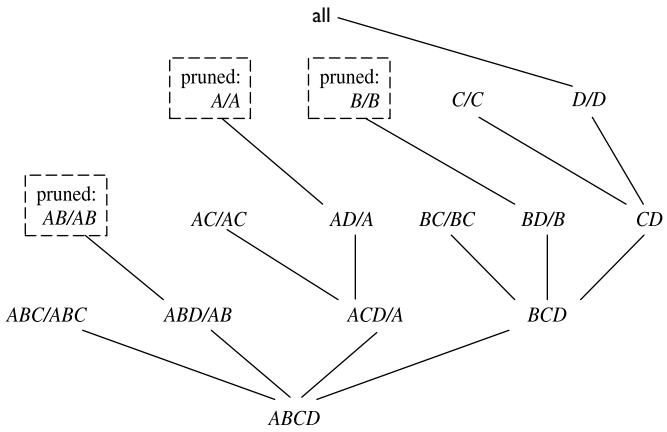


Figure 5.8 Star-Cubing: bottom-up computation with top-down expansion of shared dimensions.

A extending from *AD* would also be pruned because it was already computed in *ACD*/*A*.

Shared dimensions allow us to do Apriori-like pruning if the measure of an iceberg cube, such as *count*, is antimonotonic. That is, if the aggregate value on a shared dimension does not satisfy the iceberg condition, then *all the cells descending from this shared dimension cannot satisfy the iceberg condition either*. These cells and their descendants can be pruned because these descendant cells are, by definition, more specialized (i.e., contain more dimensions) than those in the shared dimension(s). The number of tuples covered by the descendant cells will be less than or equal to the number of tuples covered by the shared dimensions. Therefore, if the aggregate value on a shared dimension fails the iceberg condition, the descendant cells cannot satisfy it either.

Example 5.6 **Pruning shared dimensions.** If the value in the shared dimension *A* is a_1 and it fails to satisfy the iceberg condition, then the whole subtree rooted at a_1CD/a_1 (including a_1C/a_1C, a_1D/a_1, a_1/a_1) can be pruned because they are all more specialized versions of a_1. ■

To explain how the Star-Cubing algorithm works, we need to explain a few more concepts, namely, *cuboid trees*, *star-nodes*, and *star-trees*.

We use trees to represent individual cuboids. Figure 5.9 shows a fragment of the **cuboid tree** of the base cuboid, *ABCD*. Each level in the tree represents a dimension, and each node represents an attribute value. Each node has four fields: the attribute value, aggregate value, pointer to possible first child, and pointer to possible first sibling. Tuples in the cuboid are inserted one by one into the tree. A path from the root to a leaf node represents a tuple. For example, node c_2 in the tree has an aggregate (count) value of 5,

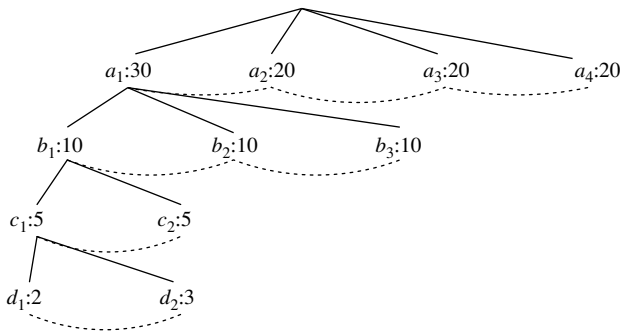


Figure 5.9 Base cuboid tree fragment.

which indicates that there are five cells of value $(a_1, b_1, c_2, *)$. This representation collapses the common prefixes to save memory usage and allows us to aggregate the values at internal nodes. With aggregate values at internal nodes, we can prune based on shared dimensions. For example, the *AB* cuboid tree can be used to prune possible cells in *ABD*.

If the single-dimensional aggregate on an attribute value *p* does not satisfy the iceberg condition, it is useless to distinguish such nodes in the iceberg cube computation. Thus, the node *p* can be replaced by $*$ so that the cuboid tree can be further compressed. We say that the node *p* in an attribute *A* is a **star-node** if the single-dimensional aggregate on *p* does not satisfy the iceberg condition; otherwise, *p* is a *non-star-node*. A cuboid tree that is compressed using star-nodes is called a **star-tree**.

Example 5.7 **Star-tree construction.** A base cuboid table is shown in Table 5.1. There are five tuples and four dimensions. The cardinalities for dimensions *A*, *B*, *C*, *D* are 2, 4, 4, 4, respectively. The one-dimensional aggregates for all attributes are shown in Table 5.2. Suppose *min_sup* = 2 in the iceberg condition. Clearly, only attribute values a_1, a_2, b_1, c_3, d_4 satisfy the condition. All other values are below the threshold and thus become star-nodes. By collapsing star-nodes, the reduced base table is Table 5.3. Notice that the table contains two fewer rows and also fewer distinct values than Table 5.1.

Table 5.1 Base (Cuboid) Table: Before Star Reduction

A	*B*	*C*	*D*	count
a_1	b_1	c_1	d_1	1
a_1	b_1	c_4	d_3	1
a_1	b_2	c_2	d_2	1
a_2	b_3	c_3	d_4	1
a_2	b_4	c_3	d_4	1

Table 5.2 One-Dimensional Aggregates

Dimension	count = 1	count ≥ 2
A	—	$a_1(3), a_2(2)$
B	b_2, b_3, b_4	$b_1(2)$
C	c_1, c_2, c_4	$c_3(2)$
D	d_1, d_2, d_3	$d_4(2)$

Table 5.3 Compressed Base Table: After Star Reduction

A	*B*	*C*	*D*	count
a_1	b_1	*	*	2
a_1	*	*	*	1
a_2	*	c_3	d_4	2

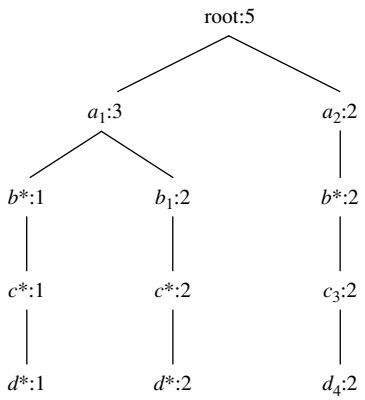


Figure 5.10 Compressed base table star-tree.

We use the reduced base table to construct the cuboid tree because it is smaller. The resultant star-tree is shown in Figure 5.10. ■

Now, let's see how the Star-Cubing algorithm uses star-trees to compute an iceberg cube. The algorithm is given later in Figure 5.13.

Example 5.8 **Star-Cubing.** Using the star-tree generated in Example 5.7 (Figure 5.10), we start the aggregation process by traversing in a bottom-up fashion. Traversal is depth-first. The first stage (i.e., the processing of the first branch of the tree) is shown in Figure 5.11. The leftmost tree in the figure is the base star-tree. Each attribute value is shown with its corresponding aggregate value. In addition, subscripts by the nodes in the tree show the

traversal order. The remaining four trees are *BCD*, *ACD*/*A*, *ABD*/*AB*, and *ABC*/*ABC*. They are the child trees of the base star-tree, and correspond to the level of 3-D cuboids above the base cuboid in Figure 5.8. The subscripts in them correspond to the same subscripts in the base tree—they denote the step or order in which they are created during the tree traversal. For example, when the algorithm is at step 1, the *BCD* child tree root is created. At step 2, the *ACD*/*A* child tree root is created. At step 3, the *ABD*/*AB* tree root and the $b*$ node in *BCD* are created.

When the algorithm has reached step 5, the trees in memory are exactly as shown in Figure 5.11. Because depth-first traversal has reached a leaf at this point, it starts backtracking. Before traversing back, the algorithm notices that all possible nodes in the base dimension (*ABC*) have been visited. This means the *ABC*/*ABC* tree is complete, so the count is output and the tree is destroyed. Similarly, upon moving back from $d*$ to $c*$ and seeing that $c*$ has no siblings, the count in *ABD*/*AB* is also output and the tree is destroyed.

When the algorithm is at $b*$ during the backtraversal, it notices that there exists a sibling in b_1. Therefore, it will keep *ACD*/*A* in memory and perform a depth-first search

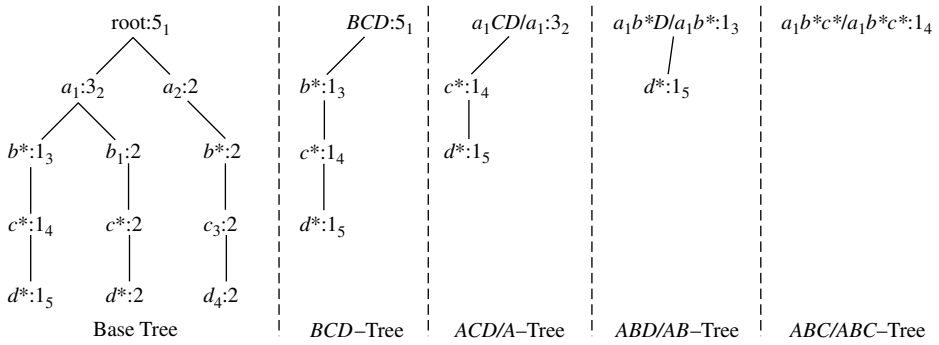


Figure 5.11 Aggregation stage one: processing the leftmost branch of the base tree.

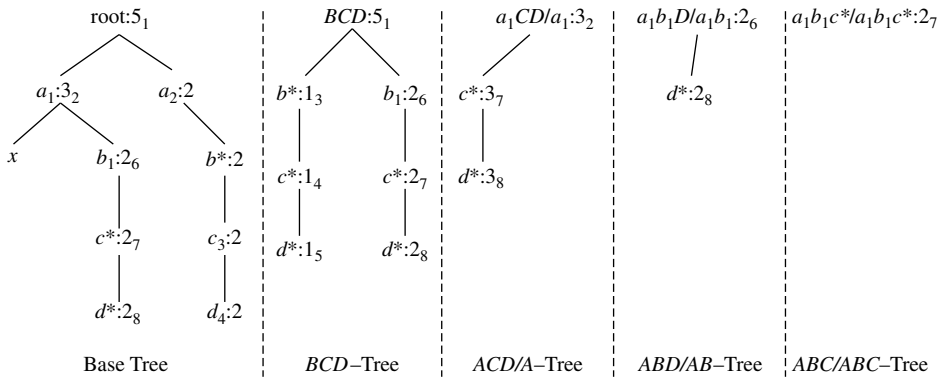


Figure 5.12 Aggregation stage two: processing the second branch of the base tree.

Algorithm: Star-Cubing. Compute iceberg cubes by Star-Cubing.

Input:

- *R*: a relational table
- *min_support*: minimum support threshold for the iceberg condition (taking count as the measure).

Output: The computed iceberg cube.

Method: Each star-tree corresponds to one cuboid tree node, and vice versa.

```
BEGIN
    scan R twice, create star-table S and star-tree T;
    output count of T.root;
  call starcubing(T, T.root);
END

procedure starcubing(T, cnode)// cnode: current node
{
(1)   for each non-null child C of T's cuboid tree
(2)       insert or aggregate cnode to the corresponding
              position or node in C's star-tree;
(3)   if (cnode.count ≥ min_support) then {
(4)       if (cnode ≠ root) then
(5)           output cnode.count;
(6)       if (cnode is a leaf) then
(7)           output cnode.count;
(8)       else { // initiate a new cuboid tree
(9)           create C_C as a child of T's cuboid tree;
(10)          let T_C be C_C's star-tree;
(11)          T_C.root's count = cnode.count;
(12)      }
(13) }
(14)  if (cnode is not a leaf) then
(15)      starcubing(T, cnode.first_child);
(16)  if (C_C is not null) then {
(17)      starcubing(T_C, T_C.root);
(18)      remove C_C from T's cuboid tree; }
(19)  if (cnode has sibling) then
(20)      starcubing(T, cnode.sibling);
(21) remove T;
}
```

Figure 5.13 Star-Cubing algorithm.

on b_1 just as it did on $b*$. This traversal and the resultant trees are shown in Figure 5.12. The child trees ACD/A and ABD/AB are created again but now with the new values from the b_1 subtree. For example, notice that the aggregate count of $c*$ in the ACD/A tree has increased from 1 to 3. The trees that remained intact during the last traversal are reused and the new aggregate values are added on. For instance, another branch is added to the *BCD* tree.

Just like before, the algorithm will reach a leaf node at $d*$ and traverse back. This time, it will reach a_1 and notice that there exists a sibling in a_2. In this case, all child trees except *BCD* in Figure 5.12 are destroyed. Afterward, the algorithm will perform the same traversal on a_2. *BCD* continues to grow while the other subtrees start fresh with a_2 instead of a_1. ■

A node must satisfy two conditions in order to generate child trees: (1) the measure of the node must satisfy the iceberg condition; and (2) the tree to be generated must include at least one non-star-node (i.e., nontrivial). This is because if all the nodes were star-nodes, then none of them would satisfy *min_sup*. Therefore, it would be a complete waste to compute them. This pruning is observed in Figures 5.11 and 5.12. For example, the left subtree extending from node a_1 in the base tree in Figure 5.11 does not include any nonstar-nodes. Therefore, the a_1CD/a_1 subtree should not have been generated. It is shown, however, for illustration of the child tree generation process.

Star-Cubing is sensitive to the ordering of dimensions, as with other iceberg cube construction algorithms. For best performance, the dimensions are processed in order of decreasing cardinality. This leads to a better chance of early pruning, because the higher the cardinality, the smaller the partitions, and therefore the higher possibility that the partition will be pruned.

Star-Cubing can also be used for full cube computation. When computing the full cube for a dense data set, Star-Cubing's performance is comparable with MultiWay and is much faster than BUC. If the data set is sparse, Star-Cubing is significantly faster than MultiWay and faster than BUC, in most cases. For iceberg cube computation, Star-Cubing is faster than BUC, where the data are skewed and the speed-up factor increases as *min_sup* decreases.

5.2.4 Precomputing Shell Fragments for Fast High-Dimensional OLAP

Recall the reason that we are interested in precomputing data cubes: Data cubes facilitate fast OLAP in a multidimensional data space. However, a full data cube of high dimensionality needs massive storage space and unrealistic computation time. Iceberg cubes provide a more feasible alternative, as we have seen, wherein the iceberg condition is used to specify the computation of only a subset of the full cube's cells. However, although an iceberg cube is smaller and requires less computation time than its corresponding full cube, it is not an ultimate solution.

For one, the computation and storage of the iceberg cube can still be costly. For example, if the base cuboid cell, $(a_1, a_2, \ldots, a_{60})$, passes minimum support (or the iceberg

threshold), it will generate 2^{60} iceberg cube cells. Second, it is difficult to determine an appropriate iceberg threshold. Setting the threshold too low will result in a huge cube, whereas setting the threshold too high may invalidate many useful applications. Third, an iceberg cube cannot be incrementally updated. Once an aggregate cell falls below the iceberg threshold and is pruned, its measure value is lost. Any incremental update would require recomputing the cells from scratch. This is extremely undesirable for large real-life applications where incremental appending of new data is the norm.

One possible solution, which has been implemented in some commercial data warehouse systems, is to compute a thin **cube shell**. For example, we could compute all cuboids with three dimensions or less in a 60-dimensional data cube, resulting in a cube shell of size 3. The resulting cuboids set would require much less computation and storage than the full 60-dimensional data cube. However, there are two disadvantages to this approach. First, we would still need to compute $\binom{60}{3} + \binom{60}{2} + 60 = 36{,}050$ cuboids, each with many cells. Second, such a cube shell does not support high-dimensional OLAP because (1) it does not support OLAP on four or more dimensions, and (2) it cannot even support drilling along three dimensions, such as, say, (A_4, A_5, A_6), *on a subset of data* selected based on the constants provided in three *other* dimensions, such as (A_1, A_2, A_3), because this essentially requires the computation of the corresponding 6-D cuboid. (Notice that there is no cell in cuboid (A_4, A_5, A_6) computed for any particular constant set, such as (a_1, a_2, a_3), associated with dimensions (A_1, A_2, A_3).)

Instead of computing a cube shell, we can compute only portions or fragments of it. This section discusses the *shell fragment* approach for OLAP query processing. It is based on the following key observation about OLAP in high-dimensional space. Although a data cube may contain many dimensions, *most OLAP operations are performed on only a small number of dimensions at a time.* In other words, an OLAP query is likely to ignore many dimensions (i.e., treating them as irrelevant), fix some dimensions (e.g., using query constants as instantiations), and leave only a few to be manipulated (for drilling, pivoting, etc.). This is because it is neither realistic nor fruitful for anyone to comprehend the changes of thousands of cells involving tens of dimensions simultaneously in a high-dimensional space at the same time.

Instead, it is more natural to first locate some cuboids of interest and then drill along one or two dimensions to examine the changes of a few related dimensions. Most analysts will only need to examine, at any one moment, the combinations of a small number of dimensions. This implies that if multidimensional aggregates can be computed quickly on a *small number of dimensions inside a high-dimensional space*, we may still achieve fast OLAP without materializing the original high-dimensional data cube. Computing the full cube (or, often, even an iceberg cube or cube shell) can be excessive. Instead, a *semi-online computation model with certain preprocessing* may offer a more feasible solution. Given a base cuboid, some quick preparation computation can be done first (i.e., offline). After that, a query can then be computed online using the preprocessed data.

The shell fragment approach follows such a semi-online computation strategy. It involves two algorithms: one for computing cube shell fragments and the other for query processing with the cube fragments. The shell fragment approach can handle databases

Table 5.4 Original Database

TID	*A*	*B*	*C*	*D*	*E*
1	a_1	b_1	c_1	d_1	e_1
2	a_1	b_2	c_1	d_2	e_1
3	a_1	b_2	c_1	d_1	e_2
4	a_2	b_1	c_1	d_1	e_2
5	a_2	b_1	c_1	d_1	e_3

of high dimensionality and can quickly compute small local cubes online. It explores the *inverted index* data structure, which is popular in information retrieval and Web-based information systems.

The basic idea is as follows. Given a high-dimensional data set, we partition the dimensions into a set of disjoint dimension *fragments*, convert each fragment into its corresponding inverted index representation, and then construct *cube shell fragments* while keeping the inverted indices associated with the cube cells. Using the precomputed cubes' shell fragments, we can dynamically assemble and compute cuboid cells of the required data cube online. This is made efficient by set intersection operations on the inverted indices.

To illustrate the shell fragment approach, we use the tiny database of Table 5.4 as a running example. Let the cube measure be count(). Other measures will be discussed later. We first look at how to construct the inverted index for the given database.

Example 5.9 **Construct the inverted index.** For each attribute value in each dimension, list the tuple identifiers (*TIDs*) of all the tuples that have that value. For example, attribute value a_2 appears in tuples 4 and 5. The TID list for a_2 then contains exactly two items, namely 4 and 5. The resulting inverted index table is shown in Table 5.5. It retains all the original database's information. If each table entry takes one unit of memory, Tables 5.4 and 5.5 each takes 25 units, that is, the inverted index table uses the same amount of memory as the original database. ■

"*How do we compute shell fragments of a data cube?*" The shell fragment computation algorithm, **Frag-Shells**, is summarized in Figure 5.14. We first partition all the dimensions of the given data set into independent groups of dimensions, called *fragments* (line 1). We scan the base cuboid and construct an inverted index for each attribute (lines 2 to 6). Line 3 is for when the measure is other than the tuple count(), which will be described later. For each fragment, we compute the full *local* (i.e., fragment-based) data cube while retaining the inverted indices (lines 7 to 8). Consider a database of 60 dimensions, namely, $A_1, A_2, \ldots, A_{60}$. We can first partition the 60 dimensions into 20 fragments of size 3: (A_1, A_2, A_3), (A_4, A_5, A_6), …, (A_{58}, A_{59}, A_{60}). For each fragment, we compute its full data cube while recording the inverted indices. For example, in fragment (A_1, A_2, A_3), we would compute seven cuboids: A_1, A_2, A_3, A_1A_2, A_2A_3, A_1A_3, $A_1A_2A_3$. Furthermore, an inverted index

Table 5.5 Inverted Index

Attribute Value	*TID List*	*List Size*
a_1	{1, 2, 3}	3
a_2	{4, 5}	2
b_1	{1, 4, 5}	3
b_2	{2, 3}	2
c_1	{1, 2, 3, 4, 5}	5
d_1	{1, 3, 4, 5}	4
d_2	{2}	1
e_1	{1, 2}	2
e_2	{3, 4}	2
e_3	{5}	1

Algorithm: Frag-Shells. Compute shell fragments on a given high-dimensional base table (i.e., base cuboid).

Input: A base cuboid, B, of n dimensions, namely, $(A_1, \ldots, A_n)$.

Output:

- a set of fragment partitions, $\{P_1, \ldots, P_k\}$, and their corresponding (local) fragment cubes, $\{S_1, \ldots, S_k\}$, where P_i represents some set of dimension(s) and $P_1 \cup \ldots \cup P_k$ make up all the n dimensions
- an *ID_measure* array if the measure is not the tuple count, count()

Method:

(1) partition the set of dimensions $(A_1, \ldots, A_n)$ into
    a set of k fragments $P_1, \ldots, P_k$ (based on data & query distribution)
(2) scan base cuboid, B, once and do the following {
(3) insert each ⟨*TID, measure*⟩ into *ID_measure* array
(4) for each attribute value a_j of each dimension A_i
(5) build an inverted index entry: ⟨a_j, *TIDlist*⟩
(6) }
(7) for each fragment partition P_i
(8) build a local fragment cube, S_i, by intersecting their
    corresponding TIDlists and computing their measures

Figure 5.14 Shell fragment computation algorithm.

is retained for each cell in the cuboids. That is, for each cell, its associated TID list is recorded.

The benefit of computing local cubes of each shell fragment instead of computing the complete cube shell can be seen by a simple calculation. For a base cuboid of

60 dimensions, there are only $7 \times 20 = 140$ cuboids to be computed according to the preceding shell fragment partitioning. This is in contrast to the 36,050 cuboids computed for the cube shell of size 3 described earlier! Notice that the above fragment partitioning is based simply on the grouping of consecutive dimensions. A more desirable approach would be to partition based on popular dimension groupings. This information can be obtained from domain experts or the past history of OLAP queries.

Let's return to our running example to see how shell fragments are computed.

Example 5.10 **Compute shell fragments.** Suppose we are to compute the shell fragments of size 3. We first divide the five dimensions into two fragments, namely (A, B, C) and (D, E). For each fragment, we compute the full local data cube by intersecting the TID lists in Table 5.5 in a top-down depth-first order in the cuboid lattice. For example, to compute the cell $(a_1, b_2, *)$, we intersect the TID lists of a_1 and b_2 to obtain a new list of $\{2, 3\}$. Cuboid AB is shown in Table 5.6.

After computing cuboid AB, we can then compute cuboid ABC by intersecting all pairwise combinations between Table 5.6 and the row c_1 in Table 5.5. Notice that because cell (a_2, b_2) is empty, it can be effectively discarded in subsequent computations, based on the Apriori property. The same process can be applied to compute fragment (D, E), which is completely independent from computing (A, B, C). Cuboid DE is shown in Table 5.7. ■

If the measure in the iceberg condition is count() (as in tuple counting), there is no need to reference the original database for this because the *length* of the TID list is equivalent to the tuple count. "*Do we need to reference the original database if computing other measures such as average()?*" Actually, we can build and reference an *ID_measure*

Table 5.6 Cuboid AB

Cell	*Intersection*	*TID List*	*List Size*
(a_1, b_1)	$\{1, 2, 3\} \cap \{1, 4, 5\}$	$\{1\}$	1
(a_1, b_2)	$\{1, 2, 3\} \cap \{2, 3\}$	$\{2, 3\}$	2
(a_2, b_1)	$\{4, 5\} \cap \{1, 4, 5\}$	$\{4, 5\}$	2
(a_2, b_2)	$\{4, 5\} \cap \{2, 3\}$	$\{\}$	0

Table 5.7 Cuboid DE

Cell	*Intersection*	*TID List*	*List Size*
(d_1, e_1)	$\{1, 3, 4, 5\} \cap \{1, 2\}$	$\{1\}$	1
(d_1, e_2)	$\{1, 3, 4, 5\} \cap \{3, 4\}$	$\{3, 4\}$	2
(d_1, e_3)	$\{1, 3, 4, 5\} \cap \{5\}$	$\{5\}$	1
(d_2, e_1)	$\{2\} \cap \{1, 2\}$	$\{2\}$	1

array instead, which stores what we need to compute other measures. For example, to compute **average()**, we let the *ID_measure* array hold three elements, namely, (*TID*, **item_count**, **sum**), for each cell (line 3 of the shell fragment computation algorithm in Figure 5.14). The **average()** measure for each aggregate cell can then be computed by accessing only this *ID_measure* array, using **sum()**/**item_count()**. Considering a database with 10^6 tuples, each taking 4 bytes each for *TID*, **item_count**, and **sum**, the *ID_measure* array requires 12 MB, whereas the corresponding database of 60 dimensions will require $(60 + 3) \times 4 \times 10^6 = 252$ MB (assuming each attribute value takes 4 bytes). Obviously, *ID_measure* array is a more compact data structure and is more likely to fit in memory than the corresponding high-dimensional database.

To illustrate the design of the *ID_measure* array, let's look at Example 5.11.

Example 5.11 Computing cubes with the average() measure. Table 5.8 shows an example sales database where each tuple has two associated values, such as **item_count** and **sum**, where **item_count** is the count of items sold.

To compute a data cube for this database with the measure **average()**, we need to have a TID list for each cell: $\{TID_1, \ldots, TID_n\}$. Because each TID is uniquely associated with a particular set of measure values, all future computation just needs to fetch the measure values associated with the tuples in the list. In other words, by keeping an *ID_measure* array in memory for online processing, we can handle complex algebraic measures, such as average, variance, and standard deviation. Table 5.9 shows what exactly should be kept for our example, which is substantially smaller than the database itself. ■

Table 5.8 Database with Two Measure Values

TID	*A*	*B*	*C*	*D*	*E*	*item_count*	*sum*
1	a_1	b_1	c_1	d_1	e_1	5	70
2	a_1	b_2	c_1	d_2	e_1	3	10
3	a_1	b_2	c_1	d_1	e_2	8	20
4	a_2	b_1	c_1	d_1	e_2	5	40
5	a_2	b_1	c_1	d_1	e_3	2	30

Table 5.9 Table 5.8 *ID_measure* Array

TID	*item_count*	*sum*
1	5	70
2	3	10
3	8	20
4	5	40
5	2	30

The shell fragments are negligible in both storage space and computation time in comparison with the full data cube. Note that we can also use the Frag-Shells algorithm to compute the full data cube by including all the dimensions as a single fragment. Because the order of computation with respect to the cuboid lattice is top-down and depth-first (similar to that of BUC), the algorithm can perform Apriori pruning if applied to the construction of iceberg cubes.

"Once we have computed the shell fragments, how can they be used to answer OLAP queries?" Given the precomputed shell fragments, we can view the cube space as a virtual cube and perform OLAP queries related to the cube online. In general, two types of queries are possible: (1) *point query* and (2) *subcube query*.

In a **point query**, all of the *relevant* dimensions in the cube have been instantiated (i.e., there are no *inquired* dimensions in the relevant dimensions set). For example, in an n-dimensional data cube, $A_1A_2 \ldots A_n$, a point query could be in the form of $\langle A_1, A_5, A_9 : M? \rangle$, where $A_1 = \{a_{11}, a_{18}\}$, $A_5 = \{a_{52}, a_{55}, a_{59}\}$, $A_9 = a_{94}$, and M is the inquired measure for each corresponding cube cell. For a cube with a small number of dimensions, we can use $*$ to represent a "don't care" position where the corresponding dimension is *irrelevant*, that is, neither inquired nor instantiated. For example, in the query $\langle a_2, b_1, c_1, d_1, * : \text{count}() ? \rangle$ for the database in Table 5.4, the first four dimension values are instantiated to a_2, b_1, c_1, and d_1, respectively, while the last dimension is irrelevant, and count() (which is the tuple count by context) is the inquired measure.

In a **subcube query**, at least one of the *relevant* dimensions in the cube is *inquired*. For example, in an n-dimensional data cube $A_1A_2 \ldots A_n$, a subcube query could be in the form $\langle A_1, A_5?, A_9, A_{21}? : M? \rangle$, where $A_1 = \{a_{11}, a_{18}\}$ and $A_9 = a_{94}$, A_5 and A_{21} are the inquired dimensions, and M is the inquired measure. For a cube with a small number of dimensions, we can use $*$ for an irrelevant dimension and ? for an inquired one. For example, in the query $\langle a_2, ?, c_1, *, ? : \text{count}() ? \rangle$ we see that the first and third dimension values are instantiated to a_2 and c_1, respectively, while the fourth is irrelevant, and the second and the fifth are inquired. *A subcube query computes all possible value combinations of the inquired dimensions.* It essentially returns a local data cube consisting of the inquired dimensions.

"How can we use shell fragments to answer a point query?" Because a point query explicitly provides the instantiated variables set on the relevant dimensions set, we can make maximal use of the precomputed shell fragments by finding the *best fitting* (i.e., *dimension-wise completely matching*) fragments to fetch and intersect the associated TID lists.

Let the point query be of the form $\langle \alpha_i, \alpha_j, \alpha_k, \alpha_p : M? \rangle$, where α_i represents a set of instantiated values of dimension A_i, and so on for α_j, α_k, and α_p. First, we check the shell fragment schema to determine which dimensions among A_i, A_j, A_k, and A_p are in the same fragment(s). Suppose A_i and A_j are in the same fragment, while A_k and A_p are in two other fragments. We fetch the corresponding TID lists on the precomputed 2-D fragment for dimensions A_i and A_j using the instantiations α_i and α_j, and fetch the TID lists on the 1-D fragments for dimensions A_k and A_p using the instantiations α_k and α_p, respectively. The obtained TID lists are intersected to derive the TID list table. This table is then used to derive the specified measure (e.g., by taking the length of the TID lists

for tuple count(), or by fetching item_count() and sum() from the *ID_measure* array to compute average()) for the final set of cells.

Example 5.12 **Point query.** Suppose a user wants to compute the point query $\langle a_2, b_1, c_1, d_1, *$: count()?$\rangle$ for our database in Table 5.4 and that the shell fragments for the partitions (A, B, C) and (D, E) are precomputed as described in Example 5.10. The query is broken down into two subqueries based on the precomputed fragments: $\langle a_2, b_1, c_1, *, *\rangle$ and $\langle *, *, *, d_1, *\rangle$. The best-fit precomputed shell fragments for the two subqueries are *ABC* and *D*. The fetch of the TID lists for the two subqueries returns two lists: {4, 5} and {1, 3, 4, 5}. Their intersection is the list {4, 5}, which is of size 2. Thus, the final answer is count() = 2. ■

"How can we use shell fragments to answer a subcube query?" A subcube query returns a local data cube based on the instantiated and inquired dimensions. Such a data cube needs to be aggregated in a multidimensional way so that online analytical processing (drilling, dicing, pivoting, etc.) can be made available to users for flexible manipulation and analysis. Because instantiated dimensions usually provide highly selective constants that dramatically reduce the size of the valid TID lists, we should make maximal use of the precomputed shell fragments by finding the fragments that best fit the set of instantiated dimensions, and fetching and intersecting the associated TID lists to derive the reduced TID list. This list can then be used to intersect the best-fitting shell fragments consisting of the inquired dimensions. This will generate the relevant and inquired base cuboid, which can then be used to compute the relevant subcube on-the-fly using an efficient online cubing algorithm.

Let the subcube query be of the form $\langle \alpha_i, \alpha_j, A_k?, \alpha_p, A_q? : M?\rangle$, where α_i, α_j, and α_p represent a set of instantiated values of dimension A_i, A_j, and A_p, respectively, and A_k and A_q represent two inquired dimensions. First, we check the shell fragment schema to determine which dimensions among (1) A_i, A_j, and A_p, and (2) A_k and A_q are in the same fragment partition. Suppose A_i and A_j belong to the same fragment, as do A_k and A_q, but that A_p is in a different fragment. We fetch the corresponding TID lists in the precomputed 2-D fragment for A_i and A_j using the instantiations α_i and α_j, then fetch the TID list on the precomputed 1-D fragment for A_p using instantiation α_p, and then fetch the TID lists on the precomputed 2-D fragments for A_k and A_q, respectively, using no instantiations (i.e., all possible values). The obtained TID lists are intersected to derive the final TID lists, which are used to fetch the corresponding measures from the *ID_measure* array to derive the "base cuboid" of a 2-D subcube for two dimensions (A_k, A_q). A fast cube computation algorithm can be applied to compute this 2-D cube based on the derived base cuboid. The computed 2-D cube is then ready for OLAP operations.

Example 5.13 **Subcube query.** Suppose that a user wants to compute the subcube query, $\langle a_2, b_1, ?, *, ? :$ count()?$\rangle$, for our database shown earlier in Table 5.4, and that the shell fragments have been precomputed as described in Example 5.10. The query can be broken into three best-fit fragments according to the instantiated and inquired dimensions: *AB*, *C*,

and E, where AB has the instantiation (a_2, b_1). The fetch of the TID lists for these partitions returns $(a_2, b_1) : \{4, 5\}$, $(c_1) : \{1, 2, 3, 4, 5\}$ and $\{(e_1 : \{1, 2\}), (e_2 : \{3, 4\}), (e_3 : \{5\})\}$, respectively. The intersection of these corresponding TID lists contains a cuboid with two tuples: $\{(c_1, e_2) : \{4\},$[5] $(c_1, e_3) : \{5\}\}$. This base cuboid can be used to compute the 2-D data cube, which is trivial. ■

For large data sets, a fragment size of 2 or 3 typically results in reasonable storage requirements for the shell fragments and for fast query response time. Querying with shell fragments is substantially faster than answering queries using precomputed data cubes that are stored on disk. In comparison to full cube computation, Frag-Shells is recommended if there are less than four inquired dimensions. Otherwise, more efficient algorithms, such as Star-Cubing, can be used for fast online cube computation. Frag-Shells can be easily extended to allow incremental updates, the details of which are left as an exercise.

5.3 Processing Advanced Kinds of Queries by Exploring Cube Technology

Data cubes are not confined to the simple multidimensional structure illustrated in the last section for typical business data warehouse applications. The methods described in this section further develop data cube technology for effective processing of advanced kinds of queries. Section 5.3.1 explores *sampling cubes*. This extension of data cube technology can be used to answer queries on *sample data*, such as survey data, which represent a sample or subset of a target data population of interest. Section 5.3.2 explains how *ranking cubes* can be computed to answer top-k queries, such as "find the top 5 cars," according to some user-specified criteria.

The basic data cube structure has been further extended for various sophisticated data types and new applications. Here we list some examples, such as *spatial data cubes* for the design and implementation of *geospatial data warehouses*, and *multimedia data cubes* for the multidimensional analysis of *multimedia data* (those containing images and videos). *RFID data cubes* handle the compression and multidimensional analysis of *RFID* (i.e., radio-frequency identification) data. *Text cubes* and *topic cubes* were developed for the application of vector-space models and generative language models, respectively, in the analysis of *multidimensional text databases* (which contain both structure attributes and narrative text attributes).

5.3.1 Sampling Cubes: OLAP-Based Mining on Sampling Data

When collecting data, we often collect only a *subset* of the data we would ideally like to gather. In statistics, this is known as collecting a **sample** of the data population.

[5] That is, the intersection of the TID lists for (a_2, b_1), (c_1), and (e_2) is $\{4\}$.

The resulting data are called **sample data**. Data are often sampled to save on costs, manpower, time, and materials. In many applications, the collection of the entire data population of interest is unrealistic. In the study of TV ratings or pre-election polls, for example, it is impossible to gather the opinion of *everyone* in the population. Most published ratings or polls rely on a data sample for analysis. The results are extrapolated for the entire population, and associated with certain statistical measures such as a *confidence interval*. The confidence interval tells us how reliable a result is. Statistical surveys based on sampling are a common tool in many fields like politics, healthcare, market research, and social and natural sciences.

"*How effective is OLAP on sample data*?" OLAP traditionally has the full data population on hand, yet with sample data, we have only a small subset. If we try to apply traditional OLAP tools to sample data, we encounter three challenges. First, sample data are often sparse in the multidimensional sense. When a user drills down on the data, it is easy to reach a point with very few or no samples even when the overall sample size is large. Traditional OLAP simply uses whatever data are available to compute a query answer. To extrapolate such an answer for a population based on a small sample could be misleading: A single outlier or a slight bias in the sampling can distort the answer significantly. Second, with sample data, statistical methods are used to provide a measure of reliability (e.g., a confidence interval) to indicate the quality of the query answer as it pertains to the population. Traditional OLAP is not equipped with such tools.

A *sampling cube* framework was introduced to tackle each of the preceding challenges.

Sampling Cube Framework

The **sampling cube** is a data cube structure that stores the sample data and their multidimensional aggregates. It supports OLAP on sample data. It calculates confidence intervals as a quality measure for any multidimensional query. Given a sample data relation (i.e., base cuboid) R, the sampling cube C_R typically computes the sample mean, sample standard deviation, and other task-specific measures.

In statistics, a *confidence interval* is used to indicate the reliability of an estimate. Suppose we want to estimate the mean age of all viewers of a given TV show. We have sample data (a subset) of this data population. Let's say our sample mean is 35 years. This becomes our estimate for the entire population of viewers as well, but how confident can we be that 35 is also the mean of the true population? It is unlikely that the sample mean will be exactly equal to the true population mean because of sampling error. Therefore, we need to qualify our estimate in some way to indicate the general magnitude of this error. This is typically done by computing a **confidence interval**, which is an *estimated value range with a given high probability of covering the true population value*. A confidence interval for our example could be "*the actual mean will not vary by +/− two standard deviations 95% of the time*." (Recall that the standard deviation is just a number, which can be computed as shown in Section 2.2.2.) A confidence interval is always qualified by a particular *confidence level*. In our example, it is 95%.

The confidence interval is calculated as follows. Let x be a set of samples. The mean of the samples is denoted by $\bar{x}$, and the number of samples in x is denoted by l. Assuming

that the standard deviation of the population is unknown, the *sample* standard deviation of x is denoted by s. Given a desired confidence level, the **confidence interval** for $\bar{x}$ is

$$\bar{x} \pm t_c \hat{\sigma}_{\bar{x}}, \tag{5.1}$$

where t_c is the *critical t-value* associated with the confidence level and $\hat{\sigma}_{\bar{x}} = \frac{s}{\sqrt{l}}$ is the *estimated standard error of the mean*. To find the appropriate t_c, specify the desired confidence level (e.g., 95%) and also the *degree of freedom*, which is just $l - 1$.

The important thing to note is that the computation involved in computing a confidence interval is *algebraic*. Let's look at the three terms involved in Eq. (5.1). The first is the mean of the sample set, $\bar{x}$, which is algebraic; the second is the critical t-value, which is calculated by a lookup, and with respect to x, it depends on l, a distributive measure; and the third is $\hat{\sigma}_{\bar{x}} = \frac{s}{\sqrt{l}}$, which also turns out to be algebraic if one records the linear sum ($\sum_{i=1}^{l} x_i$) and squared sum ($\sum_{i=1}^{l} x_i^2$). Because the terms involved are either algebraic or distributive, the confidence interval computation is algebraic. Actually, since both the mean and confidence interval are algebraic, at every cell, exactly three values are sufficient to calculate them—all of which are either distributive or algebraic:

1. l
2. $sum = \sum_{i=1}^{l} x_i$
3. $squared\ sum = \sum_{i=1}^{l} x_i^2$

There are many efficient techniques for computing algebraic and distributive measures (Section 4.2.4). Therefore, any of the previously developed cubing algorithms can be used to efficiently construct a sampling cube.

Now that we have established that sampling cubes can be computed efficiently, our next step is to find a way of boosting the confidence of results obtained for queries on sample data.

Query Processing: Boosting Confidences for Small Samples

A query posed against a data cube can be either a *point query* or a *range query*. Without loss of generality, consider the case of a point query. Here, it corresponds to a cell in sampling cube C_R. The goal is to provide an accurate point estimate for the samples in that cell. Because the cube also reports the confidence interval associated with the sample mean, there is some measure of "reliability" to the returned answer. If the confidence interval is small, the reliability is deemed good; however, if the interval is large, the reliability is questionable.

"*What can we do to boost the reliability of query answers?*" Consider what affects the confidence interval size. There are two main factors: the variance of the sample data and the sample size. First, a rather large variance in the cell may indicate that the chosen cube

cell is poor for prediction. A better solution is probably to drill down on the query cell to a more specific one (i.e., asking more specific queries). Second, a small sample size can cause a large confidence interval. When there are very few samples, the corresponding t_c is large because of the small degree of freedom. This in turn could cause a large confidence interval. Intuitively, this makes sense. Suppose one is trying to figure out the average income of people in the United States. Just asking two or three people does not give much confidence to the returned response.

The best way to solve this small sample size problem is to get more data. Fortunately, there is usually an abundance of additional data available in the cube. The data do not match the query cell exactly; however, we can consider data from cells that are "close by." There are two ways to incorporate such data to enhance the reliability of the query answer: (1) *intracuboid query expansion*, where we consider nearby cells *within* the same cuboid, and (2) *intercuboid query expansion*, where we consider more general versions (from parent cuboids) of the query cell. Let's see how this works, starting with intracuboid query expansion.

Method 1. Intracuboid query expansion. Here, we expand the sample size by including nearby cells in the *same* cuboid as the queried cell, as shown in Figure 5.15(a). We just have to be careful that the new samples serve to increase the confidence in the answer without changing the query's semantics.

So, the first question is "*Which dimensions should be expanded?*" The best candidates should be the dimensions that are *uncorrelated* or *weakly correlated* with the measure

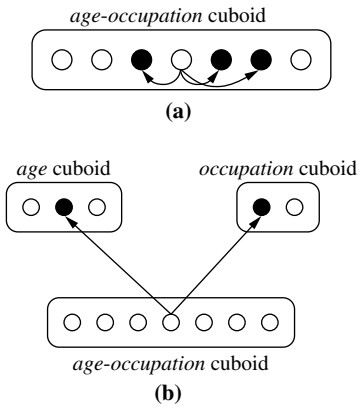


Figure 5.15 Query expansion within sampling cube: Given small data samples, both methods use strategies to boost the reliability of query answers by considering additional data cell values. (a) Intracuboid expansion considers nearby cells in the same cuboid as the queried cell. (b) Intercuboid expansion considers more general cells from parent cuboids.

value (i.e., the value to be predicted). Expanding within these dimensions will likely increase the sample size and not shift the query's answer. Consider an example of a 2-D query specifying *education* = "college" and *birth_month* = "July." Let the cube measure be *average income*. Intuitively, education has a high correlation to income while birth month does not. It would be harmful to expand the *education* dimension to include values such as "graduate" or "high school." They are likely to alter the final result. However, expansion in the *birth_month* dimension to include other month values could be helpful, because it is unlikely to change the result but will increase sampling size.

To mathematically measure the correlation of a dimension to the cube value, the correlation between the dimension's values and their aggregated cube measures is computed. *Pearson's correlation coefficient* for numeric data and the χ^2 correlation test for nominal data are popularly used correlation measures, although many other measures, such as *covariance*, can be used. (These measures were presented in Section 3.3.2.) A dimension that is strongly correlated with the value to be predicted should *not* be a candidate for expansion. Notice that since the correlation of a dimension with the cube measure is independent of a particular query, it should be precomputed and stored with the cube measure to facilitate efficient online analysis.

After selecting dimensions for expansion, the next question is "*Which values within these dimensions should the expansion use?*" This relies on the semantic knowledge of the dimensions in question. The goal should be to select semantically similar values to minimize the risk of altering the final result. Consider the *age* dimension—similarity of values in this dimension is clear. There is a definite (numeric) order to the values. Dimensions with numeric or ordinal (ranked) data (like *education*) have a definite ordering among data values. Therefore, we can select values that are close to the instantiated query value. For nominal data of a dimension that is organized in a multilevel hierarchy in a data cube (e.g., *location*), we should select those values located in the same branch of the tree (e.g., the same district or city).

By considering additional data during query expansion, we are aiming for a more accurate and reliable answer. As mentioned before, strongly correlated dimensions are precluded from expansion for this purpose. An additional strategy is to ensure that new samples share the "same" cube measure value (e.g., mean income) as the existing samples in the query cell. The two-sample ***t*-test** is a relatively simple statistical method that can be used to determine whether two samples have the same mean (or any other point estimate), where "same" means that they do not differ significantly. (It is described in greater detail in Section 8.5.5 on model selection using statistical tests of significance.)

The test determines whether two samples have the same mean (the null hypothesis) with the only assumption being that they are both normally distributed. The test fails if there is evidence that the two samples do not share the same mean. Furthermore, the test can be performed with a confidence level as an input. This allows the user to control how strict or loose the query expansion will be.

Example 5.14 shows how the intracuboid expansion strategies just described can be used to answer a query on sample data.

Table 5.10 Sample Customer Survey Data

gender	*age*	*education*	*occupation*	*income*
female	23	college	teacher	$85,000
female	40	college	programmer	$50,000
female	31	college	programmer	$52,000
female	50	graduate	teacher	$90,000
female	62	graduate	CEO	$500,000
male	25	high school	programmer	$50,000
male	28	high school	CEO	$250,000
male	40	college	teacher	$80,000
male	50	college	programmer	$45,000
male	57	graduate	programmer	$80,000

Example 5.14 **Intracuboid query expansion to answer a query on sample data.** Consider a book retailer trying to learn more about its customers' annual income levels. In Table 5.10, a sample of the survey data collected is shown.[6] In the survey, customers are segmented by four attributes, namely *gender*, *age*, *education*, and *occupation*.

Let a query on customer income be "$age = 25$," where the user specifies a 95% confidence level. Suppose this returns an *income* value of $50,000 with a rather large confidence interval.[7] Suppose also, that this confidence interval is larger than a preset threshold and that the *age* dimension was found to have little correlation with *income* in this data set. Therefore, intracuboid expansion starts within the *age* dimension. The nearest cell is "$age = 23$," which returns an *income* of $85,000. The two-sample t-test at the 95% confidence level passes so the query expands; it is now "$age = \{23, 25\}$" with a smaller confidence interval than initially. However, it is still larger than the threshold, so expansion continues to the next nearest cell: "$age = 28$," which returns an *income* of $250,000. The two sample t-test between this cell and the original query cell fails; as a result, it is ignored. Next, "$age = 31$" is checked and it passes the test.

The confidence interval of the three cells combined is now below the threshold and the expansion finishes at "$age = \{23, 25, 31\}$." The mean of the *income* values at these three cells is $\frac{85{,}000+50{,}000+52{,}000}{3} = \$62,333$, which is returned as the query answer. It has a smaller confidence interval, and thus is more reliable than the response of $50,000, which would have been returned if intracuboid expansion had not been considered. ■

Method 2. Intercuboid query expansion. In this case, the expansion occurs by looking to a *more general cell*, as shown in Figure 5.15(b). For example, the cell in the 2-D cuboid

[6] For the sake of illustration, ignore the fact that the sample size is too small to be statistically significant.

[7] For the sake of the example, suppose this is true even though there is only one sample. In practice, more points are needed to calculate a legitimate value.

age-occupation can use its parent in either of the 1-D cuboids, *age* or *occupation*. Think of intercuboid expansion as just an extreme case of intracuboid expansion, where *all* the cells within a dimension are used in the expansion. This essentially sets the dimension to $*$ and thus generalizes to a higher-level cuboid.

A k-dimensional cell has k direct parents in the cuboid lattice, where each parent is $(k-1)$-dimensional. There are *many* more ancestor cells in the data cube (e.g., if multiple dimensions are rolled up simultaneously). However, we choose only one parent here to make the search space tractable and to limit the change in the query's semantics. As with intracuboid query expansion, correlated dimensions are not allowed in intercuboid expansions. Within the uncorrelated dimensions, the two-sample t-test can be performed to confirm that the parent and the query cell share the same sample mean. If multiple parent cells pass the test, the test's confidence level can be adjusted progressively higher until only one passes. Alternatively, multiple parent cells can be used to boost the confidence simultaneously. The choice is application dependent.

Example 5.15 **Intercuboid expansion to answer a query on sample data.** Given the input relation in Table 5.10, let the query on *income* be "*occupation* = teacher $\wedge$ *gender* = male." There is only one sample in Table 5.10 that matches the query, and it has an *income* of \$80,000. Suppose the corresponding confidence interval is larger than a preset threshold. We use intercuboid expansion to find a more reliable answer. There are two parent cells in the data cube: "*gender* = male" and "*occupation* = teacher." By moving up to "*gender* = male" (and thus setting *occupation* to $*$), the mean *income* is \$101,000. A two sample t-test reveals that this parent's sample mean differs significantly from that of the original query cell, so it is ignored. Next, "*occupation* = teacher" is considered. It has a mean *income* of \$85,000 and passes the two-sample t-test. As a result, the query is expanded to "*occupation* = teacher" and an *income* value of \$85,000 is returned with acceptable reliability. ■

"*How can we determine which method to choose—intracuboid expansion or intercuboid expansion?*" This is difficult to answer without knowing the data and the application. A strategy for choosing between the two is to consider what the tolerance is for change in the query's semantics. This depends on the specific dimensions chosen in the query. For instance, the user might tolerate a bigger change in semantics for the *age* dimension than *education*. The difference in tolerance could be so large that the user is willing to set *age* to $*$ (i.e., intercuboid expansion) rather than letting *education* change at all. Domain knowledge is helpful here.

So far, our discussion has only focused on full materialization of the sampling cube. In many real-world problems, this is often impossible, especially for high-dimensional cases. Real-world survey data, for example, can easily contain over 50 variables (i.e., dimensions). The sampling cube size would grow exponentially with the number of dimensions. To handle high-dimensional data, a sampling cube method called *Sampling Cube Shell* was developed. It integrates the Frag-Shell method of Section 5.2.4 with the query expansion approach. The shell computes only a subset of the full sampling cube.

The subset should consist of relatively low-dimensional cuboids (that are commonly queried) and cuboids that offer the most benefit to the user. The details are left to interested readers as an exercise. The method was tested on both real and synthetic data and found to be efficient and effective in answering queries.

5.3.2 Ranking Cubes: Efficient Computation of Top-*k* Queries

The data cube helps not only online analytical processing of multidimensional queries but also search and data mining. In this section, we introduce a new cube structure called *Ranking Cube* and examine how it contributes to the efficient processing of *top-k queries*. Instead of returning a large set of indiscriminative answers to a query, a **top-*k* query** (or **ranking query**) returns only the best k results according to a user-specified preference.

The results are returned in ranked order so that the best is at the top. The user-specified preference generally consists of two components: a *selection condition* and a *ranking function*. Top-k queries are common in many applications like searching web databases, k-nearest-neighbor searches with approximate matches, and similarity queries in multimedia databases.

Example 5.16 A top-*k* query. Consider an online used-car database, R, that maintains the following information for each car: producer (e.g., Ford, Honda), model (e.g., Taurus, Accord), type (e.g., sedan, convertible), color (e.g., red, silver), transmission (e.g., auto, manual), price, mileage, and so on. A typical top-k query over this database is

Q_1: select *top* 5 * from R
where *producer* = "Ford" and *type* = "sedan"
order by $(price - 10K)^2 + (mileage - 30K)^2$ asc

Within the dimensions (or attributes) for R, *producer* and *type* are used here as **selection dimensions**. The **ranking function** is given in the order-by clause. It specifies the **ranking dimensions**, *price* and *mileage*. Q_1 searches for the top-5 sedans made by Ford. The entries found are ranked or sorted in ascending (asc) order, according to the ranking function. The ranking function is formulated so that entries that have price and mileage closest to the user's specified values of \$10K and 30K, respectively, appear toward the top of the list. ■

The database may have many dimensions that could be used for selection, describing, for example, whether a car has power windows, air conditioning, or a sunroof. Users may pick any subset of dimensions and issue a top-k query using their preferred ranking function. There are many other similar application scenarios. For example, when searching for hotels, ranking functions are often constructed based on price and distance to an area of interest. Selection conditions can be imposed on, say, the hotel location

district, the star rating, and whether the hotel offers complimentary treats or Internet access. The ranking functions may be linear, quadratic, or any other form.

As shown in the preceding examples, individual users may not only propose ad hoc ranking functions, but also have different data subsets of interest. Users often want to thoroughly study the data via *multidimensional analysis* of the top-k query results. For example, if unsatisfied by the top-5 results returned by Q_1, the user may roll up on the producer dimension to check the top-5 results on all sedans. The dynamic nature of the problem imposes a great challenge to researchers. OLAP requires offline precomputation so that multidimensional analysis can be performed on-the-fly, yet the ad hoc ranking functions prohibit full materialization. A natural compromise is to adopt a *semi-offline materialization* and *semi-online computation* model.

Suppose a relation R has selection dimensions $(A_1, A_2, \ldots, A_S)$ and ranking dimensions $(N_1, N_2, \ldots, N_R)$. Values in each ranking dimension can be partitioned into multiple intervals according to the data and expected query distributions. Regarding the price of used cars, for example, we may have, say, these four partitions (or value ranges): $\leq 5K$, $[5-10K)$, $[10-15K)$, and $\geq 15K$. A ranking cube can be constructed by performing multidimensional aggregations on selection dimensions. We can store the count for each partition of each ranking dimension, thereby making the cube "rank-aware." The top-k queries can be answered by first accessing the cells in the more preferred value ranges before consulting the cells in the less preferred value ranges.

Example 5.17 **Using a ranking cube to answer a top-k query.** Suppose Table 5.11 shows C_{MT}, a materialized (i.e., precomputed) cuboid of a ranking cube for used-car sales. The cuboid, C_{MT}, is for the selection dimensions *producer* and *type*. It shows the count and corresponding tuple IDs (TIDs) for various partitions of the ranking dimensions, *price* and *mileage*.

Query Q_1 can be answered by using a selection condition to select the appropriate selection dimension values (i.e., *producer* = "Ford" and *type* = "sedan") in cuboid C_{MT}. In addition, the ranking function "$(price - 10K)^2 + (mileage - 30K)^2$" is used to find the tuples that most closely match the user's criteria. If there are not enough matching tuples found in the closest matching cells, the next closest matching cells will need to be accessed. We may even drill down to the corresponding lower-level cells to see the count distributions of cells that match the ranking function and additional criteria regarding, say, model, maintenance situation, or other loaded features. Only users who really want to see more detailed information, such as interior photos, will need to access the physical records stored in the database. ■

Table 5.11 Cuboid of a Ranking Cube for Used-Car Sales

producer	*type*	*price*	*mileage*	*count*	*TIDs*
Ford	sedan	<5K	30–40K	7	$t_6, \ldots, t_{68}$
Ford	sedan	5–10K	30–40K	50	$t_{15}, \ldots, t_{152}$
Honda	sedan	10–15K	30–40K	20	$t_8, \ldots, t_{32}$
…	…	…	…	…	…

Most real-life top-k queries are likely to involve only a small subset of selection attributes. To support high-dimensional ranking cubes, we can carefully select the cuboids that need to be materialized. For example, we could choose to materialize only the 1-D cuboids that contain single-selection dimensions. This will achieve low space overhead and still have high performance when the number of selection dimensions is large. In some cases, there may exist many ranking dimensions to support multiple users with rather different preferences. For example, buyers may search for houses by considering various factors like price, distance to school or shopping, number of years old, floor space, and tax. In this case, a possible solution is to create multiple data partitions, each of which consists of a subset of the ranking dimensions. The query processing may need to search over a joint space involving multiple data partitions.

In summary, the general philosophy of ranking cubes is to materialize such cubes on the set of selection dimensions. Use of the interval-based partitioning in ranking dimensions makes the ranking cube efficient and flexible at supporting ad hoc user queries. Various implementation techniques and query optimization methods have been developed for efficient computation and query processing based on this framework.

5.4 Multidimensional Data Analysis in Cube Space

Data cubes create a flexible and powerful means to group and aggregate data subsets. They allow data to be explored in multiple dimensional combinations and at varying aggregate granularities. This capability greatly increases the analysis bandwidth and helps effective discovery of interesting patterns and knowledge from data. The use of cube space makes the data space both meaningful and tractable.

This section presents methods of multidimensional data analysis that make use of data cubes to organize data into intuitive regions of interest at varying granularities. Section 5.4.1 presents *prediction cubes*, a technique for multidimensional data mining that facilitates predictive modeling in multidimensional space. Section 5.4.2 describes how to construct *multifeature cubes*. These support complex analytical queries involving multiple dependent aggregates at multiple granularities. Finally, Section 5.4.3 describes an interactive method for users to systematically explore cube space. In such *exception-based, discovery-driven exploration*, interesting exceptions or anomalies in the data are automatically detected and marked for users with visual cues.

5.4.1 Prediction Cubes: Prediction Mining in Cube Space

Recently, researchers have turned their attention toward **multidimensional data mining** to uncover knowledge at varying dimensional combinations and granularities. Such mining is also known as *exploratory multidimensional data mining* and *online analytical data mining (OLAM)*. Multidimensional data space is huge. In preparing the data, how can we identify the interesting subspaces for exploration? To what granularities should we aggregate the data? Multidimensional data mining in cube space organizes data of

interest into intuitive regions at various granularities. It analyzes and mines the data by applying various data mining techniques systematically over these regions.

There are at least four ways in which OLAP-style analysis can be fused with data mining techniques:

1. *Use cube space to define the data space for mining.* Each region in cube space represents a subset of data over which we wish to find interesting patterns. Cube space is defined by a set of expert-designed, informative dimension hierarchies, not just arbitrary subsets of data. Therefore, the use of cube space makes the data space both meaningful and tractable.
2. *Use OLAP queries to generate features and targets for mining.* The features and even the targets (that we wish to learn to predict) can sometimes be naturally defined as OLAP aggregate queries over regions in cube space.
3. *Use data mining models as building blocks in a multistep mining process.* Multidimensional data mining in cube space may consist of multiple steps, where data mining models can be viewed as building blocks that are used to describe the behavior of interesting data sets, rather than the end results.
4. *Use data cube computation techniques to speed up repeated model construction.* Multidimensional data mining in cube space may require building a model for each candidate data space, which is usually too expensive to be feasible. However, by carefully sharing computation across model construction for different candidates based on data cube computation techniques, efficient mining is achievable.

In this subsection we study *prediction cubes*, an example of multidimensional data mining where the cube space is explored for prediction tasks. A **prediction cube** is a cube structure that stores prediction models in multidimensional data space and supports prediction in an OLAP manner. Recall that in a data cube, each cell value is an aggregate number (e.g., count) computed over the data subset in that cell. However, each cell value in a prediction cube is computed by evaluating a predictive model built on the data subset in that cell, thereby representing that subset's predictive behavior.

Instead of seeing prediction models as the end result, prediction cubes use prediction models as building blocks to define the interestingness of data subsets, that is, they identify data subsets that indicate more accurate prediction. This is best explained with an example.

Example 5.18 **Prediction cube for identification of interesting cube subspaces.** Suppose a company has a customer table with the attributes *time* (with two granularity levels: *month* and *year*), *location* (with two granularity levels: *state* and *country*), *gender*, *salary*, and one class-label attribute: *valued_customer*. A manager wants to analyze the decision process of whether a customer is highly valued with respect to *time* and *location*. In particular, he is interested in the question "*Are there times at and locations in which the value of a*

customer depended greatly on the customer's gender?" Notice that he believes *time* and *location* play a role in predicting valued customers, but at what granularity levels do they depend on *gender* for this task? For example, is performing analysis using {*month, country*} better than {*year, state*}?

Consider a data table **D** (e.g., the customer table). Let **X** be the attributes set for which no concept hierarchy has been defined (e.g., *gender, salary*). Let Y be the class-label attribute (e.g., *valued_customer*), and **Z** be the set of multilevel attributes, that is, attributes for which concept hierarchies have been defined (e.g., *time, location*). Let **V** be the set of attributes for which we would like to define their predictiveness. In our example, this set is {*gender*}. The predictiveness of **V** on a data subset can be quantified by the difference in accuracy between the model built on that subset using **X** to predict Y and the model built on that subset using $\mathbf{X} - \mathbf{V}$ (e.g., {*salary*}) to predict Y. The intuition is that, if the difference is large, **V** must play an important role in the prediction of class label Y.

Given a set of attributes, **V**, and a learning algorithm, the prediction cube at granularity $\langle l_1, \ldots, l_d \rangle$ (e.g., $\langle year, state \rangle$) is a d-dimensional array, in which the value in each cell (e.g., [2010, Illinois]) is the predictiveness of **V** evaluated on the subset defined by the cell (e.g., the records in the customer table with *time* in 2010 and *location* in Illinois). ■

Supporting OLAP roll-up and drill-down operations on a prediction cube is a computational challenge requiring the materialization of cell values at many different granularities. For simplicity, we can consider only full materialization. A naïve way to fully materialize a prediction cube is to exhaustively build models and evaluate them for each cell and granularity. This method is very expensive if the base data set is large. An ensemble method called **Probability-Based Ensemble (PBE)** was developed as a more feasible alternative. It requires model construction for only the finest-grained cells. OLAP-style bottom-up aggregation is then used to generate the values of the coarser-grained cells.

The prediction of a predictive model can be seen as finding a class label that maximizes a scoring function. The PBE method was developed to approximately make the scoring function of any predictive model distributively decomposable. In our discussion of data cube measures in Section 4.2.4, we showed that distributive and algebraic measures can be computed efficiently. Therefore, if the scoring function used is distributively or algebraically decomposable, prediction cubes can also be computed with efficiency. In this way, the PBE method reduces prediction cube computation to data cube computation.

For example, previous studies have shown that the naïve Bayes classifier has an algebraically decomposable scoring function, and the kernel density–based classifier has a distributively decomposable scoring function.[8] Therefore, either of these could be used

[8] Naïve Bayes classifiers are detailed in Chapter 8. Kernel density–based classifiers, such as support vector machines, are described in Chapter 9.

to implement prediction cubes efficiently. The PBE method presents a novel approach to multidimensional data mining in cube space.

5.4.2 Multifeature Cubes: Complex Aggregation at Multiple Granularities

Data cubes facilitate the answering of analytical or mining-oriented queries as they allow the computation of aggregate data at multiple granularity levels. Traditional data cubes are typically constructed on commonly used dimensions (e.g., *time*, *location*, and *product*) using simple measures (e.g., count( ), average( ), and sum()). In this section, you will learn a newer way to define data cubes called **multifeature cubes**. Multifeature cubes enable more in-depth analysis. They can compute more complex queries of which the measures depend on groupings of multiple aggregates at varying granularity levels. The queries posed can be much more elaborate and task-specific than traditional queries, as we shall illustrate in the next examples. Many complex data mining queries can be answered by multifeature cubes without significant increase in computational cost, in comparison to cube computation for simple queries with traditional data cubes.

To illustrate the idea of multifeature cubes, let's first look at an example of a query on a simple data cube.

Example 5.19 **A simple data cube query.** Let the query be "*Find the total sales in 2010, broken down by item, region, and month, with subtotals for each dimension.*" To answer this query, a traditional data cube is constructed that aggregates the total sales at the following eight different granularity levels: {(*item, region, month*), (*item, region*), (*item, month*), (*month, region*), (*item*), (*month*), (*region*), ()}, where () represents all. This data cube is simple in that it does not involve any dependent aggregates. ■

To illustrate what is meant by "dependent aggregates," let's examine a more complex query, which can be computed with a multifeature cube.

Example 5.20 **A complex query involving dependent aggregates.** Suppose the query is "*Grouping by all subsets of* {*item, region, month*}, *find the maximum price in 2010 for each group and the total sales among all maximum price tuples.*"

The specification of such a query using standard SQL can be long, repetitive, and difficult to optimize and maintain. Alternatively, it can be specified concisely using an extended SQL syntax as follows:

```
select      item, region, month, max(price), sum(R.sales)
from        Purchases
where       year = 2010
cube by     item, region, month: R
such that   R.price = max(price)
```

The tuples representing purchases in 2010 are first selected. The cube by clause computes aggregates (or group-by's) for all possible combinations of the attributes *item,*

region, and *month*. It is an *n*-dimensional generalization of the group-by clause. The attributes specified in the cube by clause are the **grouping attributes**. Tuples with the same value on all grouping attributes form one group. Let the groups be $g_1, \ldots, g_r$. For each group of tuples g_i, the maximum price max_{g_i} among the tuples forming the group is computed. The variable R is a **grouping variable**, ranging over all tuples in group g_i that have a price equal to max_{g_i} (as specified in the such that clause). The sum of sales of the tuples in g_i that R ranges over is computed and returned with the values of the grouping attributes of g_i.

The resulting cube is a multifeature cube in that it supports complex data mining queries for which multiple dependent aggregates are computed at a variety of granularities. For example, the sum of sales returned in this query is dependent on the set of maximum price tuples for each group. In general, multifeature cubes give users the flexibility to define sophisticated, task-specific cubes on which multidimensional aggregation and OLAP-based mining can be performed. ■

"How can multifeature cubes be computed efficiently?" The computation of a multifeature cube depends on the types of aggregate functions used in the cube. In Chapter 4, we saw that aggregate functions can be categorized as either distributive, algebraic, or holistic. Multifeature cubes can be organized into the same categories and computed efficiently by minor extension of the cube computation methods in Section 5.2.

5.4.3 Exception-Based, Discovery-Driven Cube Space Exploration

As studied in previous sections, a data cube may have a large number of cuboids, and each cuboid may contain a large number of (aggregate) cells. With such an overwhelmingly large space, it becomes a burden for users to even just browse a cube, let alone think of exploring it thoroughly. Tools need to be developed to assist users in intelligently exploring the huge aggregated space of a data cube.

In this section, we describe a **discovery-driven approach** to exploring cube space. Precomputed measures indicating data exceptions are used to guide the user in the data analysis process, at all aggregation levels. We hereafter refer to these measures as *exception indicators*. Intuitively, an **exception** is a data cube cell value that is significantly different from the value anticipated, based on a statistical model. The model considers variations and patterns in the measure value across *all the dimensions* to which a cell belongs. For example, if the analysis of *item-sales* data reveals an increase in sales in December in comparison to all other months, this may seem like an exception in the time dimension. However, it is not an exception if the *item* dimension is considered, since there is a similar increase in sales for other items during December.

The model considers exceptions hidden at all aggregated group-by's of a data cube. Visual cues, such as background color, are used to reflect each cell's degree of exception, based on the precomputed exception indicators. Efficient algorithms have been proposed for cube construction, as discussed in Section 5.2. The computation of exception indicators can be overlapped with cube construction, so that the overall construction of data cubes for discovery-driven exploration is efficient.

Three measures are used as exception indicators to help identify data anomalies. These measures indicate the degree of surprise that the quantity in a cell holds, with respect to its expected value. The measures are computed and associated with every cell, for all aggregation levels. They are as follows:

- **SelfExp**: This indicates the degree of surprise of the cell value, relative to other cells at the same aggregation level.
- **InExp**: This indicates the degree of surprise somewhere beneath the cell, if we were to drill down from it.
- **PathExp**: This indicates the degree of surprise for each drill-down path from the cell.

The use of these measures for discovery-driven exploration of data cubes is illustrated in Example 5.21.

Example 5.21 **Discovery-driven exploration of a data cube.** Suppose that you want to analyze the monthly sales at *AllElectronics* as a percentage difference from the previous month. The dimensions involved are *item*, *time*, and *region*. You begin by studying the data aggregated over all items and sales regions for each month, as shown in Figure 5.16.

To view the exception indicators, you click on a button marked highlight exceptions on the screen. This translates the SelfExp and InExp values into visual cues, displayed with each cell. Each cell's background color is based on its SelfExp value. In addition, a box is drawn around each cell, where the thickness and color of the box are functions of its InExp value. Thick boxes indicate high InExp values. In both cases, the darker the color, the greater the degree of exception. For example, the dark, thick boxes for sales during July, August, and September signal the user to explore the lower-level aggregations of these cells by drilling down.

Drill-downs can be executed along the aggregated *item* or *region* dimensions. "*Which path has more exceptions?*" you wonder. To find this out, you select a cell of interest and trigger a path exception module that colors each dimension based on the PathExp value of the cell. This value reflects that path's degree of surprise. Suppose that the path along *item* contains more exceptions.

A drill-down along *item* results in the cube slice of Figure 5.17, showing the sales over time for each item. At this point, you are presented with many different sales values to analyze. By clicking on the highlight exceptions button, the visual cues are displayed, bringing focus to the exceptions. Consider the sales difference of 41% for "*Sony*

Sum of sales	*Month*											
	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
Total		1%	–1%	0%	1%	3%	–1%	–9%	–1%	2%	–4%	3%

Figure 5.16 Change in sales over time.

Avg. sales	*Month*											
Item	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
Sony b/w printer		9%	−8%	2%	−5%	14%	−4%	0%	41%	−13%	−15%	−11%
Sony color printer		0%	0%	3%	2%	4%	−10%	−13%	0%	4%	−6%	4%
HP b/w printer		−2%	1%	2%	3%	8%	0%	−12%	−9%	3%	−3%	6%
HP color printer		0%	0%	−2%	1%	0%	−1%	−7%	−2%	1%	−4%	1%
IBM desktop computer		1%	−2%	−1%	−1%	3%	3%	−10%	4%	1%	−4%	−1%
IBM laptop computer		0%	0%	−1%	3%	4%	2%	−10%	−2%	0%	−9%	3%
Toshiba desktop computer		−2%	−5%	1%	1%	−1%	1%	5%	−3%	−5%	−1%	−1%
Toshiba laptop computer		1%	0%	3%	0%	−2%	−2%	−5%	3%	2%	−1%	0%
Logitech mouse		3%	−2%	−1%	0%	4%	6%	−11%	2%	1%	−4%	0%
Ergo-way mouse		0%	0%	2%	3%	1%	−2%	−2%	−5%	0%	−5%	8%

Figure 5.17 Change in sales for each *item-time* combination.

Avg. sales	*Month*											
Region	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
North		−1%	−3%	−1%	0%	3%	4%	−7%	1%	0%	−3%	−3%
South		−1%	1%	−9%	6%	−1%	−39%	9%	−34%	4%	1%	7%
East		−1%	−2%	2%	−3%	1%	18%	−2%	11%	−3%	−2%	−1%
West		4%	0%	−1%	−3%	5%	1%	−18%	8%	5%	−8%	1%

Figure 5.18 Change in sales for the item *IBM desktop computer* per region.

b/w printers" in September. This cell has a dark background, indicating a high SelfExp value, meaning that the cell is an exception. Consider now the sales difference of −15% for "*Sony b/w printers*" in November and of −11% in December. The −11% value for December is marked as an exception, while the −15% value is not, even though −15% is a bigger deviation than −11%. This is because the exception indicators consider all the dimensions that a cell is in. Notice that the December sales of most of the other items have a large positive value, while the November sales do not. Therefore, by considering the cell's position in the cube, the sales difference for "*Sony b/w printers*" in December is exceptional, while the November sales difference of this item is not.

The InExp values can be used to indicate exceptions at lower levels that are not visible at the current level. Consider the cells for "*IBM desktop computers*" in July and September. These both have a dark, thick box around them, indicating high InExp values. You may decide to further explore the sales of "*IBM desktop computers*" by drilling down along *region*. The resulting sales difference by *region* is shown in Figure 5.18, where the highlight exceptions option has been invoked. The visual cues displayed make it easy to instantly notice an exception for the sales of "*IBM desktop computers*" in the southern region, where such sales have decreased by −39% and −34% in July and September,

respectively. These detailed exceptions were far from obvious when we were viewing the data as an *item-time* group-by, aggregated over *region* in Figure 5.17. Thus, the InExp value is useful for searching for exceptions at lower-level cells of the cube. ■

"*How are the exception values computed?*" The SelfExp, InExp, and PathExp measures are based on a statistical method for table analysis. They take into account all of the group-by's (aggregations) in which a given cell value participates. A cell value is considered an exception based on how much it differs from its expected value, where its expected value is determined with a statistical model. The difference between a given cell value and its expected value is called a **residual**. Intuitively, the larger the residual, the more the given cell value is an exception. The comparison of residual values requires us to scale the values based on the expected standard deviation associated with the residuals. A cell value is therefore considered an exception if its scaled residual value exceeds a prespecified threshold. The SelfExp, InExp, and PathExp measures are based on this scaled residual.

The expected value of a given cell is a function of the higher-level group-by's of the given cell. For example, given a cube with the three dimensions *A*, *B*, and *C*, the expected value for a cell at the *i*th position in *A*, the *j*th position in *B*, and the *k*th position in *C* is a function of γ, γ_i^A, γ_j^B, γ_k^C, γ_{ij}^{AB}, γ_{ik}^{AC}, and γ_{jk}^{BC}, which are coefficients of the statistical model used. The coefficients reflect how different the values at more detailed levels are, based on generalized impressions formed by looking at higher-level aggregations. In this way, the exception quality of a cell value is based on the exceptions of the values below it. Thus, when seeing an exception, it is natural for the user to further explore the exception by drilling down.

"*How can the data cube be efficiently constructed for discovery-driven exploration?*" This computation consists of three phases. The first step involves the computation of the aggregate values defining the cube, such as `sum` or `count`, over which exceptions will be found. The second phase consists of model fitting, in which the coefficients mentioned before are determined and used to compute the standardized residuals. This phase can be overlapped with the first phase because the computations involved are similar. The third phase computes the SelfExp, InExp, and PathExp values, based on the standardized residuals. This phase is computationally similar to phase 1. Therefore, the computation of data cubes for discovery-driven exploration can be done efficiently.

5.5 Summary

- **Data cube computation and exploration** play an essential role in data warehousing and are important for flexible data mining in multidimensional space.
- A data cube consists of a **lattice of cuboids**. Each cuboid corresponds to a different degree of summarization of the given multidimensional data. **Full materialization** refers to the computation of all the cuboids in a data cube lattice. **Partial materialization** refers to the selective computation of a subset of the cuboid cells in the

lattice. Iceberg cubes and shell fragments are examples of partial materialization. An **iceberg cube** is a data cube that stores only those cube cells that have an aggregate value (e.g., count) above some minimum support threshold. For **shell fragments** of a data cube, only some cuboids involving a small number of dimensions are computed, and queries on additional combinations of the dimensions can be computed on-the-fly.

- There are several efficient **data cube computation methods**. In this chapter, we discussed four cube computation methods in detail: (1) **MultiWay** array aggregation for materializing full data cubes in sparse-array-based, bottom-up, shared computation; (2) **BUC** for computing iceberg cubes by exploring ordering and sorting for efficient top-down computation; (3) **Star-Cubing** for computing iceberg cubes by integrating top-down and bottom-up computation using a star-tree structure; and (4) **shell-fragment cubing**, which supports high-dimensional OLAP by precomputing only the partitioned cube shell fragments.
- **Multidimensional data mining in cube space** is the integration of knowledge discovery with multidimensional data cubes. It facilitates systematic and focused knowledge discovery in large structured and semi-structured data sets. It will continue to endow analysts with tremendous flexibility and power at multidimensional and multigranularity exploratory analysis. This is a vast open area for researchers to build powerful and sophisticated data mining mechanisms.
- Techniques for processing advanced queries have been proposed that take advantage of cube technology. These include **sampling cubes** for multidimensional analysis on sampling data, and **ranking cubes** for efficient top-k (ranking) query processing in large relational data sets.
- This chapter highlighted three approaches to multidimensional data analysis with data cubes. **Prediction cubes** compute prediction models in multidimensional cube space. They help users identify interesting data subsets at varying degrees of granularity for effective prediction. **Multifeature cubes** compute complex queries involving multiple dependent aggregates at multiple granularities. **Exception-based, discovery-driven exploration** of cube space displays visual cues to indicate discovered data exceptions at all aggregation levels, thereby guiding the user in the data analysis process.

5.6 Exercises

5.1 Assume that a 10-D base cuboid contains only three base cells: (1) $(a_1, d_2, d_3, d_4, \ldots, d_9, d_{10})$, (2) $(d_1, b_2, d_3, d_4, \ldots, d_9, d_{10})$, and (3) $(d_1, d_2, c_3, d_4, \ldots, d_9, d_{10})$, where $a_1 \neq d_1$, $b_2 \neq d_2$, and $c_3 \neq d_3$. The measure of the cube is count().

(a) How many *nonempty* cuboids will a full data cube contain?

(b) How many *nonempty* aggregate (i.e., nonbase) cells will a full cube contain?

(c) How many *nonempty* aggregate cells will an iceberg cube contain if the condition of the iceberg cube is "count ≥ 2"?

(d) A cell, c, is a *closed cell* if there exists no cell, d, such that d is a specialization of cell c (i.e., d is obtained by replacing a $*$ in c by a non-$*$ value) and d has the same measure value as c. A *closed cube* is a data cube consisting of only closed cells. How many closed cells are in the full cube?

5.2 There are several typical cube computation methods, such as *MultiWay* [ZDN97], *BUC* [BR99], and *Star-Cubing* [XHLW03]. Briefly describe these three methods (i.e., use one or two lines to outline the key points), and compare their feasibility and performance under the following conditions:

(a) Computing a dense full cube of low dimensionality (e.g., less than eight dimensions).

(b) Computing an iceberg cube of around 10 dimensions with a highly skewed data distribution.

(c) Computing a sparse iceberg cube of high dimensionality (e.g., over 100 dimensions).

5.3 Suppose a data cube, C, has D dimensions, and the base cuboid contains k distinct tuples.

(a) Present a formula to calculate the minimum number of cells that the cube, C, may contain.

(b) Present a formula to calculate the maximum number of cells that C may contain.

(c) Answer parts (a) and (b) as if the count in each cube cell must be no less than a threshold, v.

(d) Answer parts (a) and (b) as if only closed cells are considered (with the minimum count threshold, v).

5.4 Suppose that a base cuboid has three dimensions, A, B, C, with the following number of cells: $|A| = 1,000,000$, $|B| = 100$, and $|C| = 1000$. Suppose that each dimension is evenly partitioned into 10 portions for *chunking*.

(a) Assuming each dimension has only one level, draw the complete lattice of the cube.

(b) If each cube cell stores one measure with four bytes, what is the total size of the computed cube if the cube is *dense*?

(c) State the order for computing the chunks in the cube that requires the least amount of space, and compute the total amount of main memory space required for computing the 2-D planes.

5.5 Often, the aggregate *count* value of many cells in a large data cuboid is zero, resulting in a huge, yet sparse, multidimensional matrix.

(a) Design an implementation method that can elegantly overcome this sparse matrix problem. Note that you need to explain your data structures in detail and discuss the space needed, as well as how to retrieve data from your structures.

(b) Modify your design in (a) to handle *incremental data updates*. Give the reasoning behind your new design.

5.6 When computing a cube of high dimensionality, we encounter the inherent *curse of dimensionality* problem: There exists a huge number of subsets of combinations of dimensions.

(a) Suppose that there are only two base cells, $\{(a_1, a_2, a_3, \ldots, a_{100})$ and $(a_1, a_2, b_3, \ldots, b_{100})\}$, in a 100-D base cuboid. Compute the number of nonempty aggregate cells. Comment on the storage space and time required to compute these cells.

(b) Suppose we are to compute an iceberg cube from (a). If the minimum support count in the iceberg condition is 2, how many aggregate cells will there be in the iceberg cube? Show the cells.

(c) Introducing iceberg cubes will lessen the burden of computing trivial aggregate cells in a data cube. However, even with iceberg cubes, we could still end up having to compute a large number of trivial uninteresting cells (i.e., with small counts). Suppose that a database has 20 tuples that map to (or cover) the two following base cells in a 100-D base cuboid, each with a cell count of 10: $\{(a_1, a_2, a_3, \ldots, a_{100}) : 10, (a_1, a_2, b_3, \ldots, b_{100}) : 10\}$.

i. Let the minimum support be 10. How many distinct aggregate cells will there be like the following: $\{(a_1, a_2, a_3, a_4, \ldots, a_{99}, *) : 10, \ldots, (a_1, a_2, *, a_4, \ldots, a_{99}, a_{100}) : 10, \ldots, (a_1, a_2, a_3, *, \ldots, *, *) : 10\}$?

ii. If we ignore all the aggregate cells that can be obtained by replacing some constants with $*$'s while keeping the same measure value, how many distinct cells remain? What are the cells?

5.7 Propose an algorithm that computes *closed iceberg cubes* efficiently.

5.8 Suppose that we want to compute an iceberg cube for the dimensions, A, B, C, D, where we wish to materialize all cells that satisfy a minimum support count of at least v, and where *cardinality(A)* < *cardinality(B)* < *cardinality(C)* < *cardinality(D)*. Show the *BUC* processing tree (which shows the order in which the BUC algorithm explores a data cube's lattice, starting from **all**) for the construction of this iceberg cube.

5.9 Discuss how you might extend the *Star-Cubing* algorithm to compute iceberg cubes where the iceberg condition tests for an **avg** that is no bigger than some value, v.

5.10 A flight data warehouse for a travel agent consists of six dimensions: *traveler, departure (city), departure_time, arrival, arrival_time*, and *flight*; and two measures: **count()** and **avg_fare()**, where **avg_fare()** stores the concrete fare at the lowest level but the average fare at other levels.

(a) Suppose the cube is fully materialized. Starting with the *base cuboid* [*traveler*, *departure*, *departure_time*, *arrival*, *arrival_time*, *flight*], what specific OLAP operations (e.g., roll-up *flight* to *airline*) should one perform to list the average fare per month for *each business traveler* who flies American Airlines (*AA*) from Los Angeles in 2009?

(b) Suppose we want to compute a data cube where the condition is that the minimum number of records is 10 and the average fare is over $500. Outline an efficient cube computation method (based on common sense about flight data distribution).

5.11 **(Implementation project)** There are four typical data cube computation methods: MultiWay [ZDN97], BUC [BR99], H-Cubing [HPDW01], and Star-Cubing [XHLW03].

(a) Implement any one of these cube computation algorithms and describe your implementation, experimentation, and performance. Find another student who has implemented a different algorithm on the same platform (e.g., C++ on Linux) and compare your algorithm performance with his or hers.

Input:

i. An n-dimensional base cuboid table (for $n < 20$), which is essentially a relational table with n attributes.

ii. An iceberg condition: count $(C) \geq k$, where k is a positive integer as a parameter.

Output:

i. The set of computed cuboids that satisfy the iceberg condition, in the order of your output generation.

ii. Summary of the set of cuboids in the form of "*cuboid ID*: the number of nonempty cells," sorted in alphabetical order of cuboids (e.g., *A*: 155, *AB*: 120, *ABC*: 22, *ABCD*: 4, *ABCE*: 6, *ABD*: 36), where the number after : represents the number of nonempty cells. (This is used to quickly check the correctness of your results.)

(b) Based on your implementation, discuss the following:

i. What challenging computation problems are encountered as the number of dimensions grows large?

ii. How can iceberg cubing solve the problems of part (a) for some data sets (and characterize such data sets)?

iii. Give one simple example to show that sometimes iceberg cubes cannot provide a good solution.

(c) Instead of computing a high-dimensionality data cube, we may choose to materialize the cuboids that have only a small number of dimension combinations. For example, for a 30-D data cube, we may only compute the 5-D cuboids for every possible 5-D combination. The resulting cuboids form a *shell cube*. Discuss how easy or hard it is to modify your cube computation algorithm to facilitate such computation.

5.12 The *sampling cube* was proposed for multidimensional analysis of sampling data (e.g., survey data). In many real applications, sampling data can be of high dimensionality (e.g., it is not unusual to have more than 50 dimensions in a survey data set).

(a) How can we construct an efficient and scalable high-dimensional sampling cube in large sampling data sets?

(b) Design an efficient incremental update algorithm for such a high-dimensional sampling cube.

(c) Discuss how to support quality drill-down given that some low-level cells may be empty or contain too few data for reliable analysis.

5.13 The *ranking cube* was proposed for efficient computation of top-k (ranking) queries in relational databases. Recently, researchers have proposed another kind of query, called a *skyline query*. A *skyline query* returns all the objects p_i such that p_i is not dominated by any other object p_j, where dominance is defined as follows. Let the value of p_i on dimension d be $v(p_i, d)$. We say p_i is dominated by p_j if and only if for each preference dimension d, $v(p_j, d) \leq v(p_i, d)$, and there is at least one d where the equality does not hold.

(a) Design a ranking cube so that skyline queries can be processed efficiently.

(b) Skyline queries are sometimes too strict to be desirable to some users. One may generalize the concept of skyline into *generalized skyline* as follows: *Given a d-dimensional database and a query q, the* ***generalized skyline*** *is the set of the following objects: (1) the skyline objects and (2) the nonskyline objects that are ϵ-neighbors of a skyline object, where r is an ϵ-neighbor of an object p if the distance between p and r is no more than ϵ*. Design a ranking cube to process generalized skyline queries efficiently.

5.14 The ranking cube was designed to support top-k (ranking) queries in relational database systems. However, ranking queries are also posed to data warehouses, where ranking is on multidimensional aggregates instead of on measures of base facts. For example, consider a product manager who is analyzing a sales database that stores the nationwide sales history, organized by location and time. To make investment decisions, the manager may pose the following query: "*What are the top-10 (state, year) cells having the largest total product sales?*" He may further drill down and ask, "*What are the top-10 (city, month) cells?*" Suppose the system can perform such partial materialization to derive two types of materialized cuboids: a *guiding cuboid* and a *supporting cuboid*, where the former contains a number of guiding cells that provide concise, high-level data statistics to guide the ranking query processing, whereas the latter provides inverted indices for efficient online aggregation.

(a) Derive an efficient method for computing such aggregate ranking cubes.

(b) Extend your framework to handle more advanced measures. One such example could be as follows. Consider an organization donation database, where donors are grouped by "*age*," "*income*," and other attributes. Interesting questions include: "*Which age and income groups have made the top*-k *average amount of donation (per donor)?*" and "*Which income group of donors has the largest standard deviation in the donation amount?*"

5.15 The *prediction cube* is a good example of multidimensional data mining in cube space.

(a) Propose an efficient algorithm that computes prediction cubes in a given multidimensional database.

(b) For what kind of classification models can your algorithm be applied? Explain.

5.16 *Multifeature cubes* allow us to construct interesting data cubes based on rather sophisticated query conditions. Can you construct the following multifeature cube by translating the following user requests into queries using the form introduced in this textbook?

(a) Construct a smart shopper cube where a shopper is smart if at least 10% of the goods she buys in each shopping trip are on sale.

(b) Construct a data cube for best-deal products where best-deal products are those products for which the price is the lowest for this product in the given month.

5.17 *Discovery-driven cube exploration* is a desirable way to mark interesting points among a large number of cells in a data cube. Individual users may have different views on whether a point should be considered interesting enough to be marked. Suppose one would like to mark those objects of which the absolute value of z score is over 2 in every row and column in a d-dimensional plane.

(a) Derive an efficient computation method to identify such points during the data cube computation.

(b) Suppose a partially materialized cube has $(d-1)$-dimensional and $(d+1)$-dimensional cuboids materialized but not the d-dimensional one. Derive an efficient method to mark those $(d-1)$-dimensional cells with d-dimensional children that contain such marked points.

5.7 Bibliographic Notes

Efficient computation of multidimensional aggregates in data cubes has been studied by many researchers. Gray, Chaudhuri, Bosworth, et al. [GCB+97] proposed *cube-by* as a relational aggregation operator generalizing group-by, crosstabs, and subtotals, and categorized data cube measures into three categories: *distributive*, *algebraic*, and *holistic*. Harinarayan, Rajaraman, and Ullman [HRU96] proposed a greedy algorithm for the partial materialization of cuboids in the computation of a data cube. Sarawagi and Stonebraker [SS94] developed a chunk-based computation technique for the efficient organization of large multidimensional arrays. Agarwal, Agrawal, Deshpande, et al. [AAD+96] proposed several guidelines for efficient computation of multidimensional aggregates for ROLAP servers.

The chunk-based MultiWay array aggregation method for data cube computation in MOLAP was proposed in Zhao, Deshpande, and Naughton [ZDN97]. Ross and Srivastava [RS97] developed a method for computing sparse data cubes. Iceberg queries are first described in Fang, Shivakumar, Garcia-Molina, et al. [FSGM+98]. BUC, a scalable method that computes iceberg cubes from the apex cuboid downwards, was introduced by Beyer and Ramakrishnan [BR99]. Han, Pei, Dong, and Wang [HPDW01] introduced an H-Cubing method for computing iceberg cubes with complex measures using an H-tree structure.

The Star-Cubing method for computing iceberg cubes with a dynamic star-tree structure was introduced by Xin, Han, Li, and Wah [XHLW03]. MM-Cubing, an efficient